Graphs

1. Introduction to Graph

What is a graph data structure?

There are two types of data structures

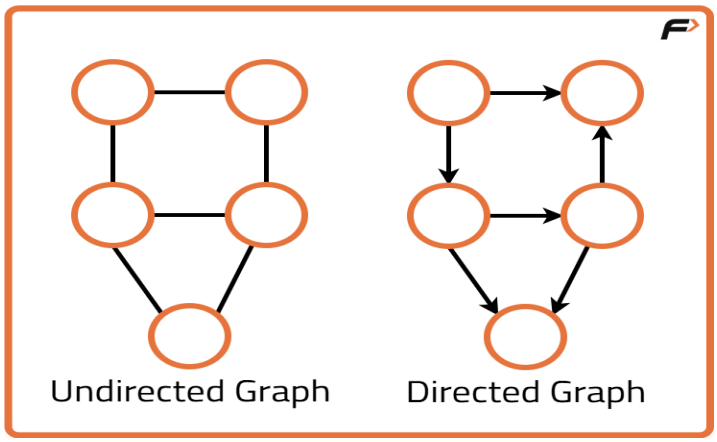
- Linear
- Non - linear

We are aware of linear data structures such as arrays, stacks, queues, and linked lists. They are called linear because data elements are arranged in a linear or sequential manner.

The only non-linear data structure that we've seen so far is Tree. In fact, a tree is a special type of graph with some restrictions. Graphs are data structures that have a wide-ranging application in real life. These include analysis of electrical circuits, finding the shortest routes between two places, building navigation systems like Google Maps, even social media using graphs to store data about each user, etc. To understand and use the graph data structure, let's get familiar with the definitions and terms associated with graphs.

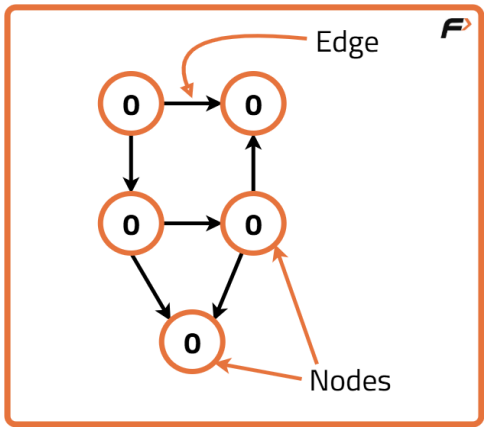
Definitions and Terminology

A graph is a non-linear data structure consisting of nodes that have data and are connected to other nodes through edges.



Nodes are circles represented by numbers. Nodes are also referred to as vertices. They store the data. The numbering of the nodes can be done in any order, no specific order needs to be followed.

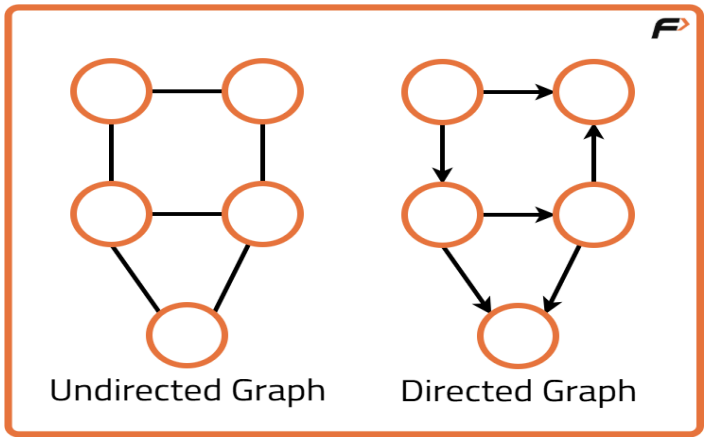
In the following example, the number of nodes or vertices = 5



Two nodes are connected by a horizontal line called **Edge**. Edge can be directed or undirected. Basically, pairs of vertices are called edges.

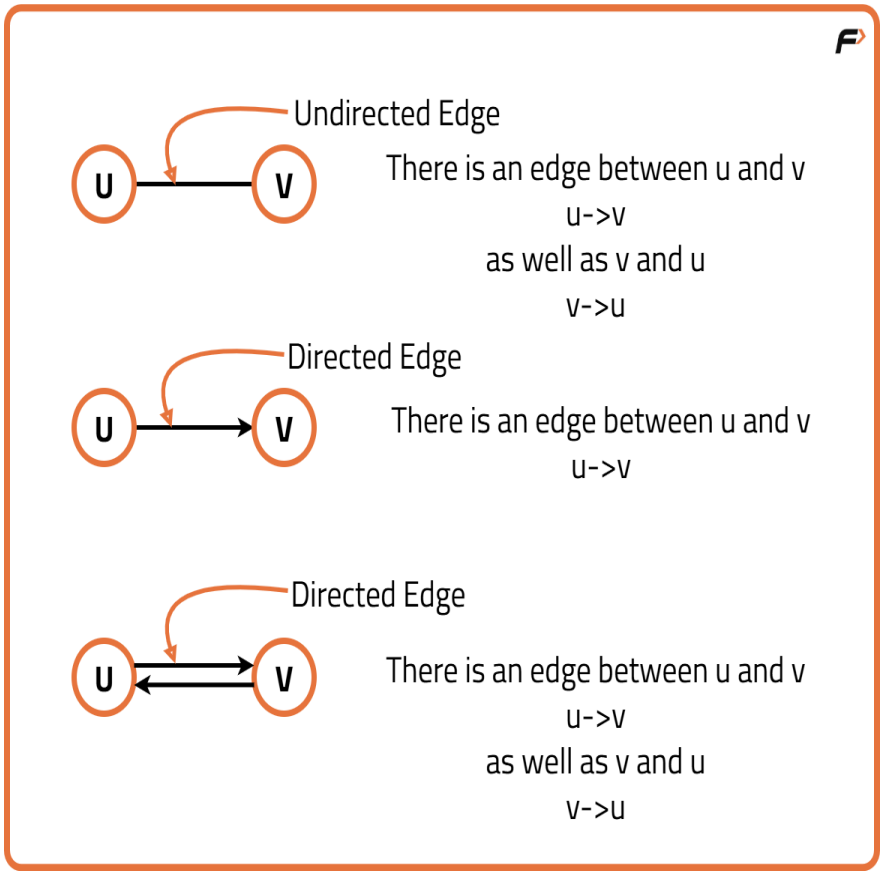
In the above example, the edge can go from 1 to 4 or from 4 to 1, i.e. a bidirectional edge can be in both directions, hence called an **undirected edge**. Thus, the pairs (1,4) and (4,1) represent the same edge.

Types of Graphs



- An undirected graph is a graph where edges are bidirectional, with no direction associated with them, i.e, there will be an undirected edge. In an undirected graph, the pair of vertices representing any edge is unordered. Thus, the pairs (u, v) and (v, u) represent the same edge.
- A directed graph is a graph where all the edges are directed from one vertex to another, i.e, there will be a directed edge. It contains an ordered pair of vertices. It implies each edge is represented by a directed pair <u, v>. Therefore, <u, v> and <v, u> represent two different edges.

There can be multi-directed edges, hence bidirectional edges, as shown in the example below.



Structure of Graph

Does every graph have a cycle?

The answer is No! Let us consider the following examples to understand this.

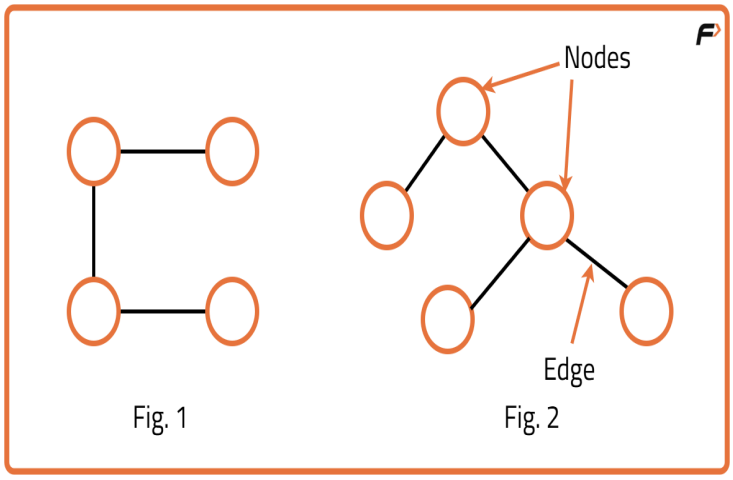
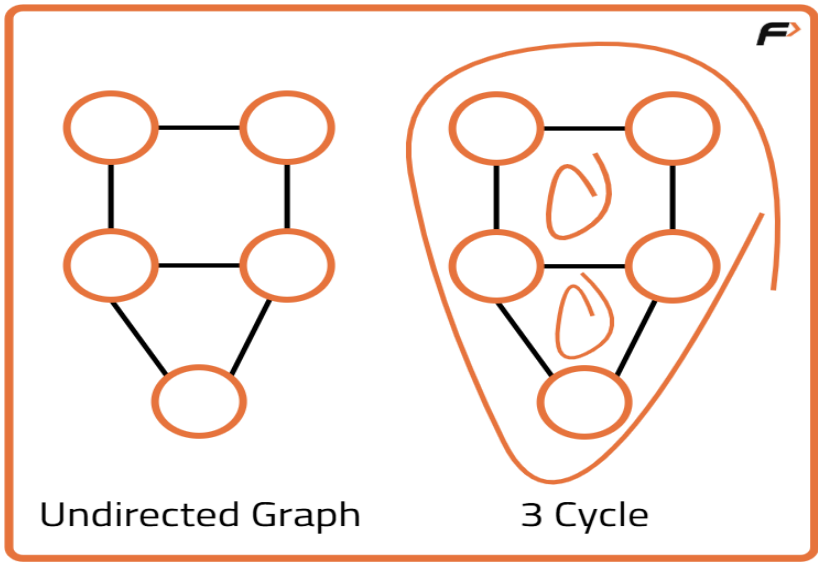


Fig. 1 does not form a cycle but still, it is a graph.

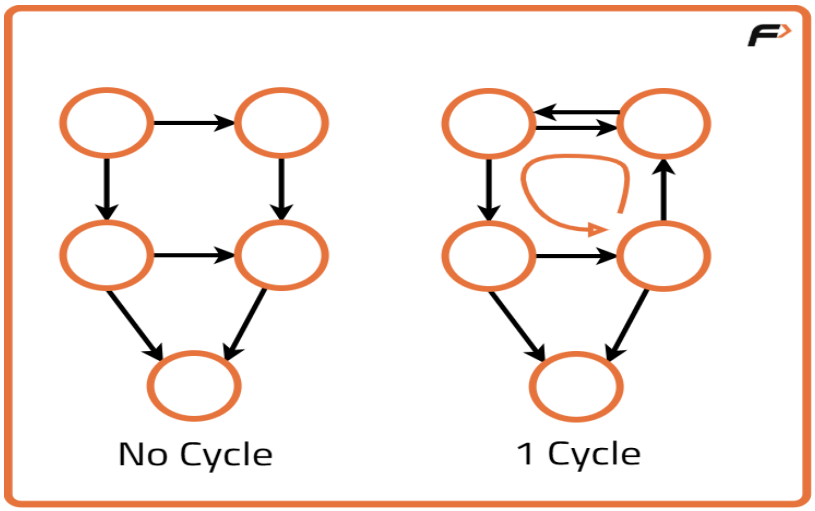
Fig. 2 is an example of a binary tree. It can also be called a graph because it follows all the rules. We've nodes and edges, and this is the minimal condition to be called a graph.

So a graph does not necessarily mean to be an enclosed structure, it can be an open structure as well. A graph is said to have a cycle if it starts from a node and ends at the same node. There can be multiple cycles in a graph.



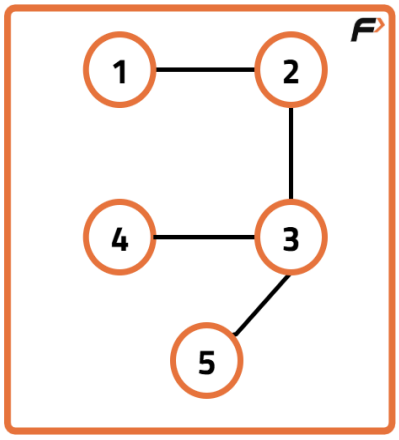
If there is at least one cycle present in the graph then it is called an **Undirected Cyclic Graph.**

In the following examples of directed graphs, the first directed graph is not cyclic as we can't start from a node and end at the same node. Hence it is called **Directed Acyclic Graph,** commonly called **DAG.**



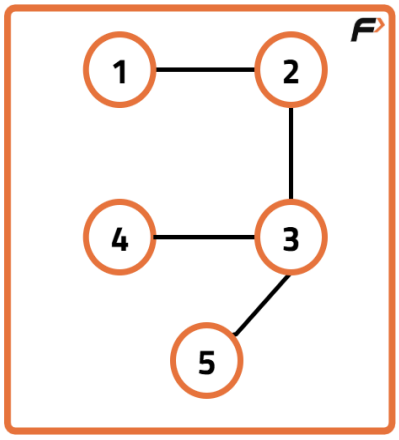
If we just add an edge to the directed graph, then at least one cycle is present in the graph, hence it becomes **Directed Cyclic Graph**.

Path in a Graph



The path contains a lot of nodes and each of them is reachable.

Consider the given graph,



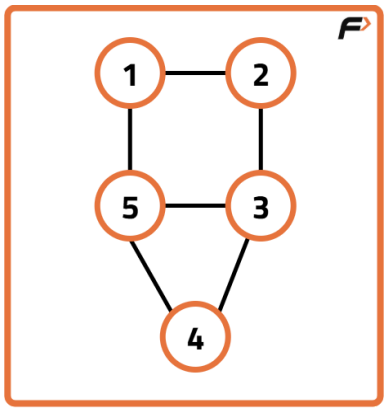

```
1 2 3 5 is a path.
1 2 3 2 1 is not a path, because a node can't appear
twice in a path.
1 3 5 is not a path, as adjacent nodes must have an edge
and there is no edge between 1 and 3.
```

Degree of Graph

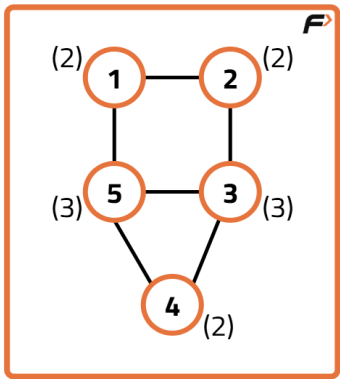
It is the number of edges that go inside or outside that node.

For **undirected graphs**, the degree is the number of edges attached to a node.

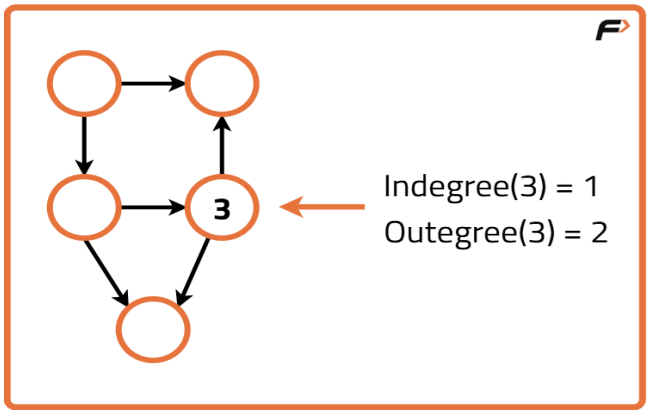
```
Example,
D(3) = 3
D(4) = 2
```



Property: It states that the total degree of a graph is equal to twice the number of edges. This is because every edge is associated/ connected to two nodes.



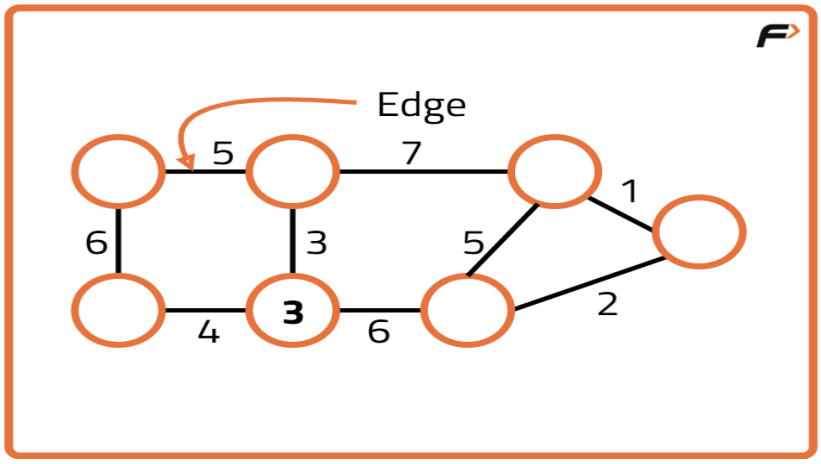

```
Total Degree of a graph = 2 x E
Example, (2+2+3+2+3) = 2 x 6 => 12 = 12
```



For **directed graphs,** we've Indegree and Outdegree. The indegree of a node is the number of incoming edges. The **outdegree** of a node is the number of outgoing edges.

Edge Weight

A graph may have weights assigned on its edges. It is often referred to as the cost of the edge.



If weights are not assigned then we assume the unit weight, i.e, 1. In applications, weight may be a measure of the cost of a route. For example, if vertices A and B represent towns in a road network, then weight on edge AB may represent the cost of moving from A to B, or vice versa.

2. Graph Representation in C++

A graph is made of **nodes (vertices)** and **edges** connecting them.
The question gives whether the graph is **directed or undirected**.

Input format:

- First line: two integers `n` and `m`
  - `n` = number of nodes
  - `m` = number of edges
- Next `m` lines: two integers `u` and `v` representing an edge between nodes `u` and `v`

There is **no fixed limit on the number of edges**. If more edges are added, `m` increases.

Nodes can be **1-based indexed**, so we usually create storage of size `n+1`.

The two most common ways to store a graph are:

1. Adjacency Matrix
2. Adjacency List

Approach 1: Adjacency Matrix

An adjacency matrix is a **2D array of size n x n**.
If there is an edge between node `i` and node `j`, then:

- `adj[i][j] = 1`
- Otherwise, `adj[i][j] = 0`

For an **undirected graph**, if there is an edge between `u` and `v`, then both `(u, v)` and `(v, u)` are marked.

Example Explanation

Input:

```
5 6
1 2
1 3
2 4
3 4
3 5
4 5
```

Here:

- Number of nodes = 5
- Number of edges = 6

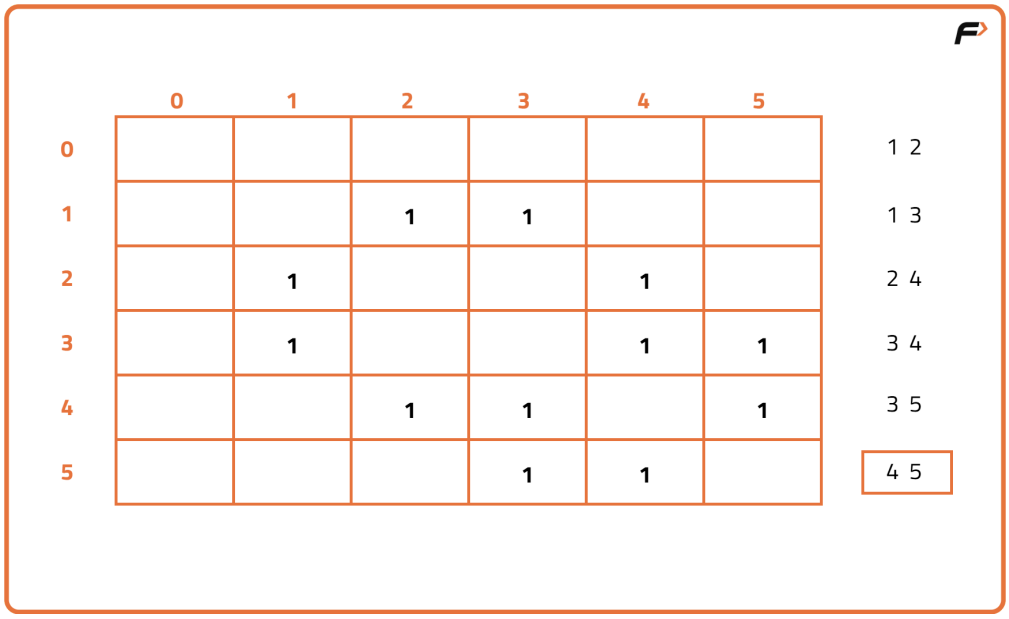
If there is an edge between 1 and 2:

- `adj[1][2] = 1`
- `adj[2][1] = 1`

If there is **no edge** between 5 and 1:

- `adj[5][1] = 0`

This matrix helps quickly check whether two nodes are directly connected.



Algorithm

1. Read n and m
2. Create a 2D array adj of size (n+1) x (n+1)
3. For each edge (u, v):
    - Mark adj[u][v] = 1
    - Mark adj[v][u] = 1 (only for undirected graph)

Code

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n, m;
    cin >> n >> m;
    int adj[n+1][n+1];
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
```

```
            adj[i][j] = 0;
    for(int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u][v] = 1;
        adj[v][u] = 1; // remove this line for directed graph
    }
    return 0;
}
```

Complexity Analysis

- **Space Complexity:** `O(n*n)`
  Reason: A full matrix of size n x n is stored.

- **Time Complexity:** `O(m)`
  Reason: Each edge is processed once.

Approach 2: Adjacency List (Undirected Graph)

In this method, for every node we store a **list of its adjacent nodes**.
This uses much less space compared to adjacency matrix.

Each edge in an undirected graph is stored **twice**:

- v is stored in the list of u

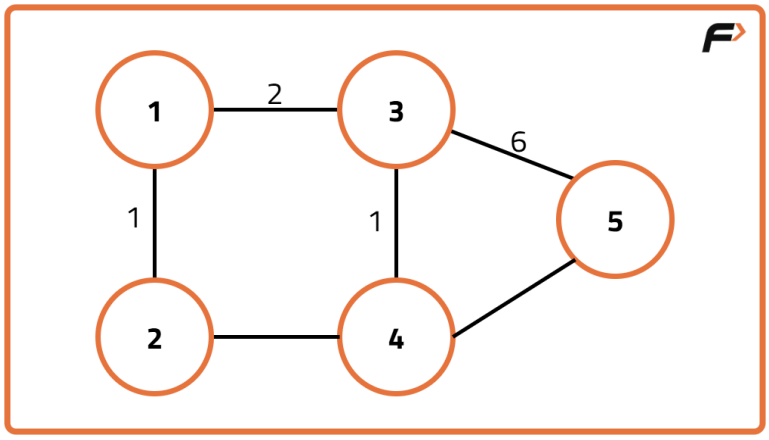
- u is stored in the list of v

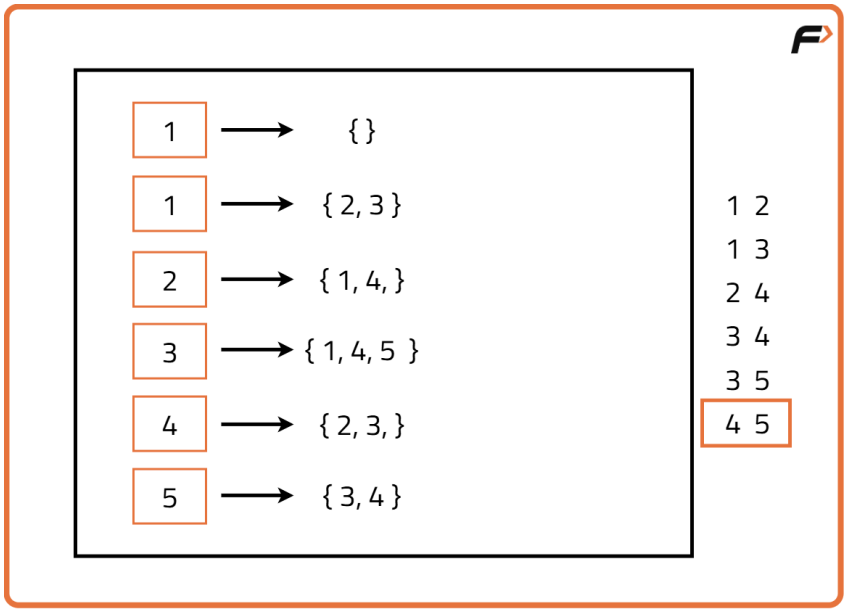
Example Explanation

From the same graph:

- Node 4 is connected to 2, 3, and 5

- So `adj[4] = {2, 3, 5}`





Algorithm

1. Read `n` and `m`
2. Create an array of vectors `adj` of size n+1
3. For each edge `(u, v)`:
    - Add `v` to `adj[u]`

- Add u to `adj[v]`

Code

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n, m;
    cin >> n >> m;
    vector<int> adj[n+1];  //Array of vectors
    for(int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    return 0;
}
```

Complexity Analysis

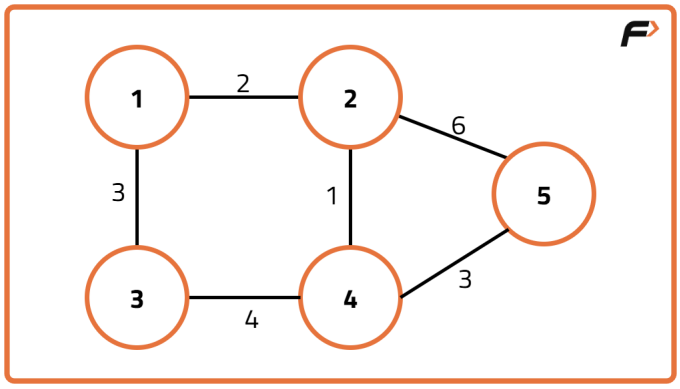
- **Space Complexity:** `O(2E)`
  Reason: Each edge is stored twice in an undirected graph.

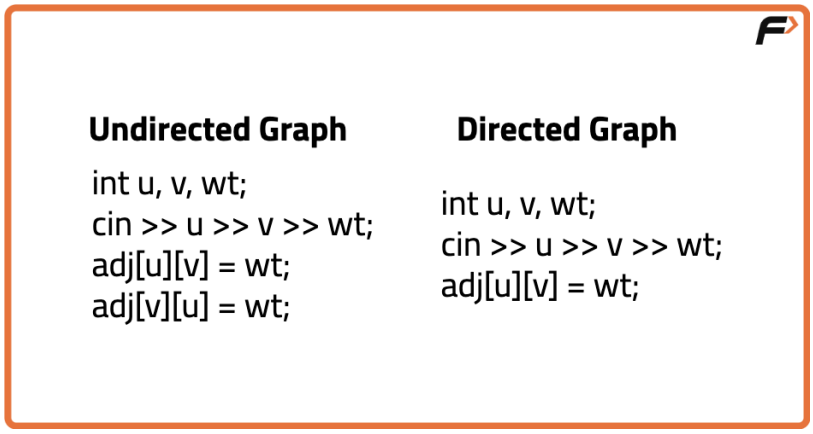
- **Time Complexity:** `O(m)`
  Reason: Each edge insertion is done once.

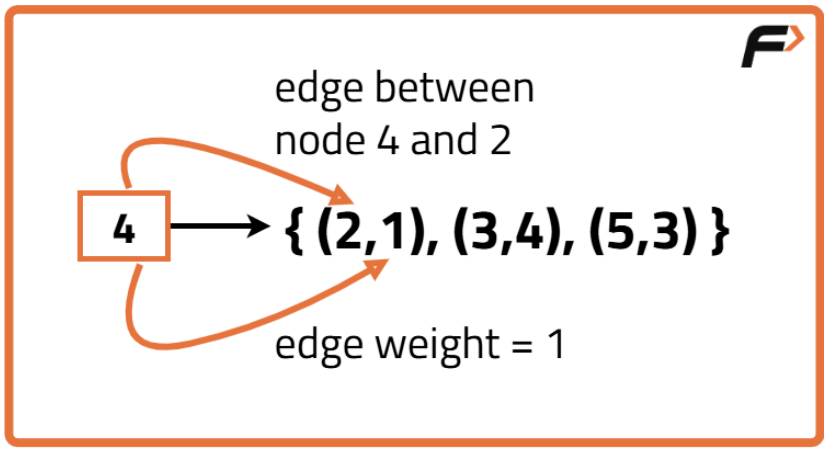
Approach 3: Adjacency List (Directed Graph)

In a directed graph, an edge `(u, v)` means **u → v only**.
So we store v only in the list of u.

Each edge is stored **only once**.







Algorithm

1. Read `n` and `m`
2. Create an array of vectors `adj` of size n+1
3. For each edge `(u, v)`:

- Add v to `adj[u]`

Code

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n, m;
    cin >> n >> m;
    vector<int> adj[n+1];
    for(int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
    }
    return 0;
}
```

Complexity Analysis

- **Space Complexity:** `O(E)`
  Reason: Each directed edge is stored only once.

- **Time Complexity:** `O(m)`
  Reason: Each edge is processed once.

Approach 4: Weighted Graph Representation (Adjacency List)

In weighted graphs, every edge has a **weight**.
So instead of storing only the node, we store a **pair**:

- `(neighbor, weight)`

Algorithm

1. Read `n` and `m`
2. Create an array of vectors of pairs
3. For each edge `(u, v, wt)`:
    - Store `(v, wt)` in `adj[u]`
    - Also store `(u, wt)` for undirected graph

Code

```
#include <bits/stdc++.h>
using namespace std;

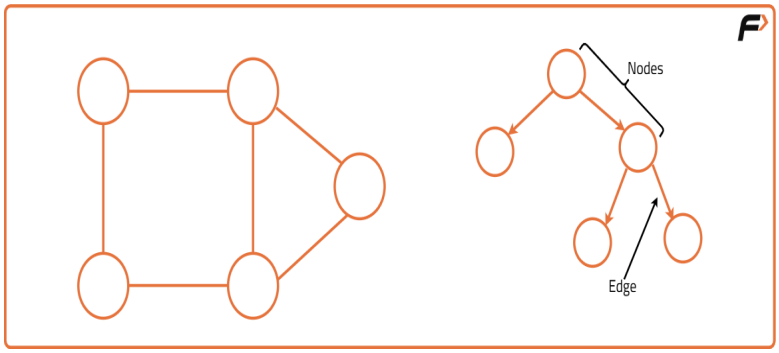
int main()
{
    int n, m;
    cin >> n >> m;
    vector<pair<int,int>> adj[n+1];
    for(int i = 0; i < m; i++)
    {
        int u, v, wt;
        cin >> u >> v >> wt;
        adj[u].push_back({v, wt});
        adj[v].push_back({u, wt}); // remove for directed weighted
graph
    }
    return 0;
}
```

- **Space Complexity:** `O(2E)` for undirected, `O(E)` for directed
  Reason: Each edge stores a pair (node, weight).

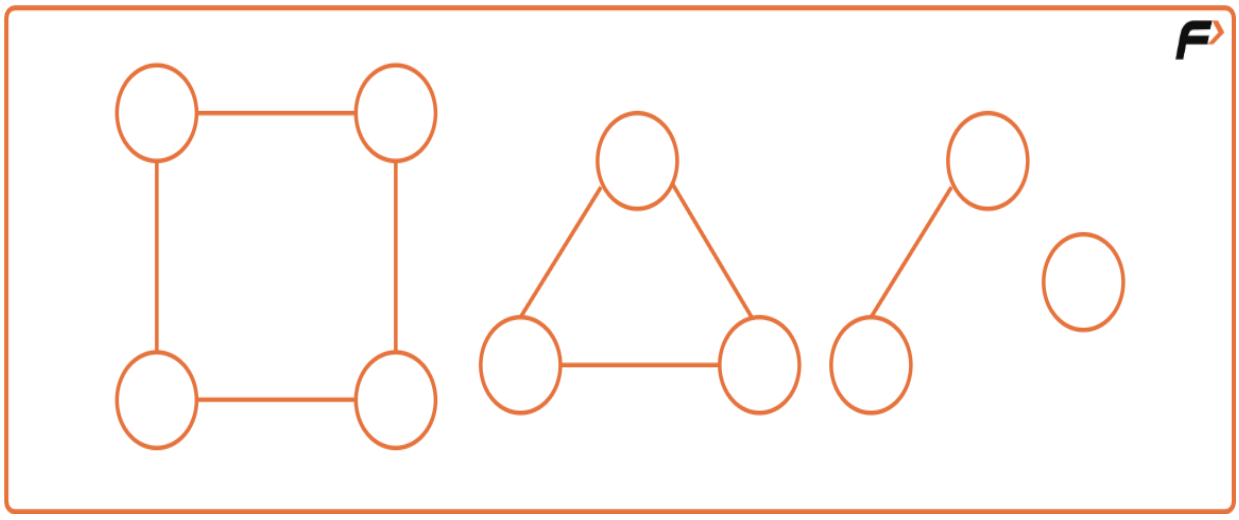
- **Time Complexity:** `O(m)`
  Reason: Each edge is inserted once.

3. Connected Components in Graphs

So far we've seen different types of graphs. Graphs can be connected or can be like a binary tree (as we know all trees are graphs with some restrictions) as shown in the following figure.



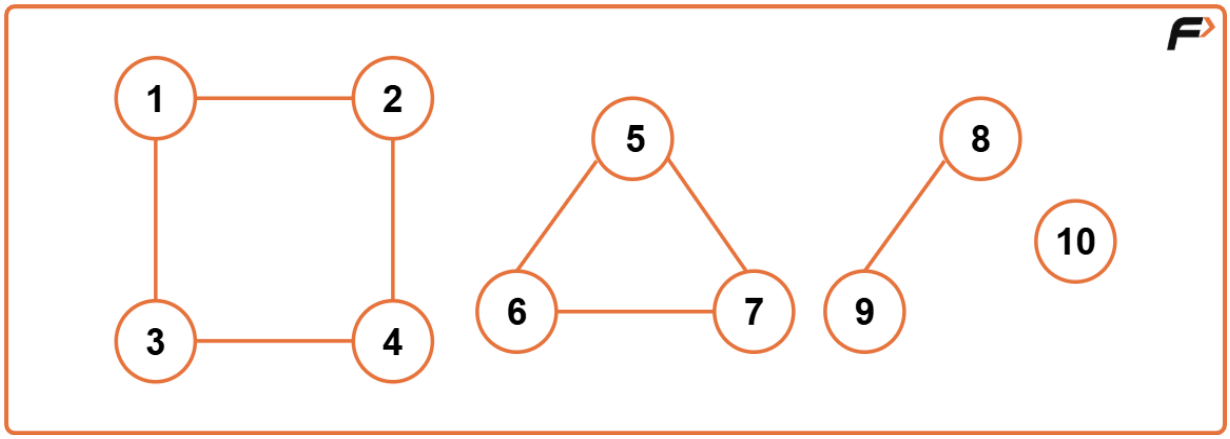
But what would you call the following figure?



The most common answer would be these are 4 different graphs as they are not connected.

But is it possible to call them a single graph? To answer this, let us consider the question given:

Given an undirected graph with 10 nodes and 8 edges. The edges are (1,2), (1,3), (2,4), (4,3), (5,6), (5,7), (6,7), (8,9) .The graph that can be formed with the given information is as follows:



Apparently, it's a graph, which is in 4 pieces, the last one being a single node. In this case, we can say, the graph has been broken down into 4 different **connected components**. So next time if you see two different parts of a graph and they are not connected, then do not say that it cannot be a single graph. In the above example, they can be 4 different graphs but according to the given question and the input, we can call them parts of a single graph.

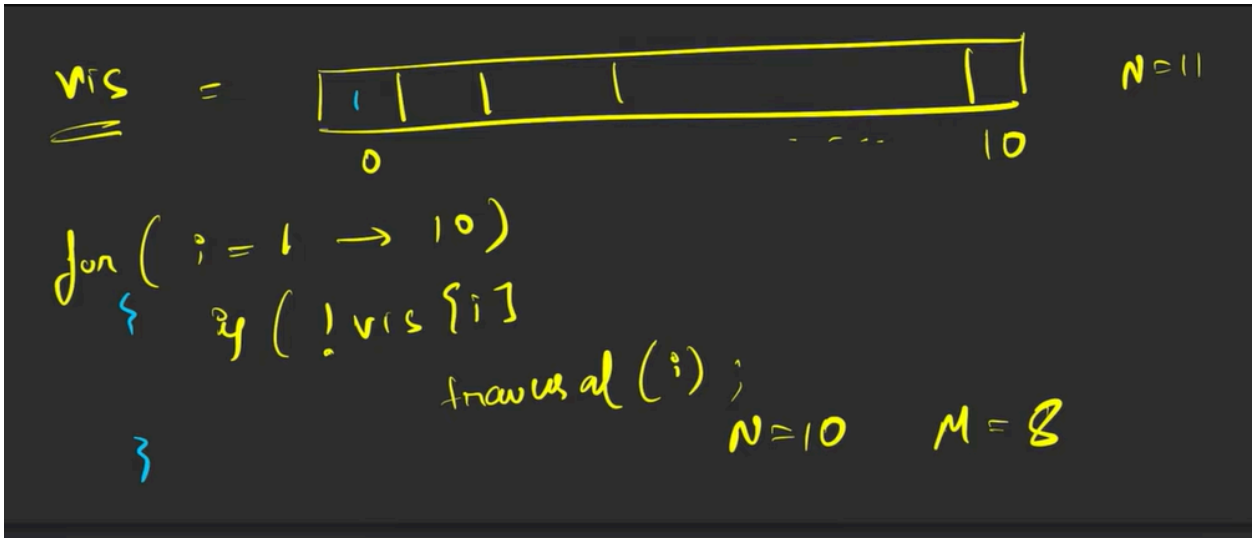
Graph Traversal

In the upcoming topics, we'll be learning about a lot of algorithms. Now, assume a traversal algorithm. Any traversal algorithm will always use a **visited array**.

For the same example, we will create an array of size 11 (n+1) starting with the zeroth index. Initialize this visited array to zero, indicating that all the nodes are unvisited. Then follow the following algorithm. If a node is not visited, then call the traversal algorithm.

Why can't we just call traversal(1)?

We cannot just call traversal(node) because a graph can have multiple components and traversal algorithms are designed in such a way that they will traverse the entire connected portion of the graph. For example, traversal(1) will traverse only the connected nodes, i.e., nodes 2, 3, and 4, but not the connected components.



4. Breadth First Search (BFS): Level Order Traversal

Given an **undirected graph**, return a vector containing all the nodes traversed using **Breadth First Search (BFS)**.
BFS starts from a starting node and explores all its neighbors first before moving to the next level of nodes.
The traversal is done **level by level** using a **queue**.

The graph is represented using an **adjacency list**, and nodes are assumed to be **0-based indexed**.
The traversal always starts from node `0`.

Algorithm

1. Create a `visited` array of size `V` and initialize all values to `0`.

2. Create a queue data structure.

3. Mark the starting node `0` as visited and push it into the queue.
4. While the queue is not empty:
    - Take the front element of the queue and remove it.
    - Add this node to the BFS result vector.
    - Traverse all adjacent nodes of this node.
    - If an adjacent node is not visited:
        - Mark it as visited.
        - Push it into the queue.
5. Continue until the queue becomes empty.
6. Return the BFS traversal vector.

This ensures nodes are visited in **breadthwise (level order)** manner.

Example Explanation

For the graph:

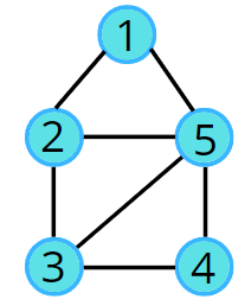
```
0 - 1 - 2
|   |
4   3
```

Starting from node `0`:

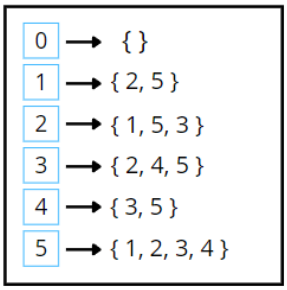
- Visit `0`, push its neighbors 1 and 4
- Visit 1, push its neighbors 2 and 3
- Visit 4, then 2, then 3

Output order:

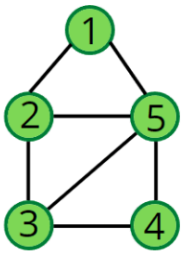
```
0 1 4 2 3
```



Undirected Graph



Adjacency List



0	1	2	3	4	5
0	1	1	1	1	1

Visited Array

Print: 1 2 5 3



Queue

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
  public:
    vector<int> bfsOfGraph(int V, vector<int> adj[]) {
        int vis[V] = {0};
        queue<int> q;
        vector<int> bfs;
        vis[0] = 1;
        q.push(0);
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            bfs.push_back(node);
            for(auto it : adj[node]) {
                if(!vis[it]) {
```

```
                    vis[it] = 1;
                    q.push(it);
                }
            }
        }
        return bfs;
    }
};

void addEdge(vector<int> adj[], int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void printAns(vector<int> &ans) {
    for(int i = 0; i < ans.size(); i++)
        cout << ans[i] << " ";
}

int main() {
    vector<int> adj[6];
    addEdge(adj, 0, 1);
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 0, 4);
    Solution obj;
    vector<int> ans = obj.bfsOfGraph(5, adj);
    printAns(ans);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N) + O(2E)$
  Reason: $O(N)$ for visiting all nodes and $O(2E)$ for traversing adjacency lists in an undirected graph.

- **Space Complexity:** `O(N)`
  Reason: Space used by visited array, queue, and result vector.

✅ Updated BFS for disconnected graph

```
vector<int> bfsOfGraph(int V, vector<int> adj[]) {
    vector<int> bfs;
    vector<int> vis(V, 0);
    queue<int> q;

    for(int i = 0; i < V; i++) {
        if(!vis[i]) {
            vis[i] = 1;
            q.push(i);

            while(!q.empty()) {
                int node = q.front();
                q.pop();
                bfs.push_back(node);

                for(auto it : adj[node]) {
                    if(!vis[it]) {
                        vis[it] = 1;
                        q.push(it);
                    }
                }
            }
        }
    }
    return bfs;
}
```

5. Depth First Search (DFS)

Given an **undirected graph**, return a vector of all nodes by traversing the graph using **Depth First Search (DFS)**.
DFS explores the graph by going **as deep as possible** along one path before backtracking and

exploring other paths.
The graph is represented using an **adjacency list**, and traversal starts from a chosen node.

Approach

Algorithm

1. Use a `visited` array to keep track of visited nodes.
2. Start DFS from a given node.
3. Mark the current node as visited and store it in the result vector.
4. Traverse all adjacent nodes of the current node using the adjacency list.
5. For each adjacent node that is not visited:
    - Recursively apply DFS on that node.
6. When a node has no unvisited neighbors, backtrack to the previous node.
7. Repeat until all reachable nodes are visited.

DFS uses **recursion and backtracking** to explore the graph fully.

Example Explanation

Example:

```
V = 5
adj = [[1,2], [0,3], [0,4], [1], [2]]
```

Start DFS from node `0`:

- Visit `0`
- Go to 1

- Go to 3 (no more neighbors, backtrack)
- Back to `0`, go to 2
- Go to 4

DFS traversal:

```
0 1 3 2 4
```

This order shows how DFS goes deep first, then backtracks.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void dfs(int v, vector<int> adj[], vector<int>& visited,
vector<int>& result) {
        visited[v] = 1;
        result.push_back(v);
        for(int u : adj[v]) {
            if(!visited[u]) {
                dfs(u, adj, visited, result);
            }
        }
    }
};

int main() {
    int V = 5;
    vector<int> adj[V];
    adj[0] = {1, 2};
    adj[1] = {0, 3};
    adj[2] = {0, 4};
```

```
    adj[3] = {1};
    adj[4] = {2};
    vector<int> visited(V, 0);
    vector<int> result;
    Solution sol;
    sol.dfs(0, adj, visited, result);
    for(int x : result) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex is visited once and each edge is checked once through the adjacency list.

- **Space Complexity:** `O(V)`
  Reason: Extra space is used by the visited array and the recursion stack.

If Node fix na ho to:

```
vector<int> dfsOfGraph(int V, vector<int> adj[]) {
      vector<int> visited(V, 0);
      vector<int> result;

      for(int i = 0; i < V; i++) {
         if(!visited[i]) {
            dfs(i, adj, visited, result);
         }
      }
      return result;
   }
```

6. Number of Provinces

Given an **undirected graph** with `V` vertices, find the number of **provinces**.
Two vertices u and `v` belong to the same province if there exists a **path** between them, directly or indirectly.

The graph is given as an **adjacency matrix** `adj` of size `V x V`:

- `adj[i][j] = 1` means city `i` and city `j` are directly connected
- `adj[i][j] = 0` means there is no direct connection

A province is simply a **connected component** of the graph.

Algorithm

A province contains all cities that can be reached from one city using traversal.

1. Create a `visited` array to track visited cities.
2. Convert the adjacency matrix into an adjacency list for easy traversal.
3. Loop through all cities from `0` to `V-1`.
4. If a city is not visited:
    - It means we found a new province.
    - Increase the province counter.
    - Run DFS from that city to mark all cities in this province as visited.
5. The number of times DFS is started gives the number of provinces.

DFS ensures that all directly or indirectly connected cities are visited in one call.

Example Explanation

Example 1:

```
adj = [
 [1,0,0,1],
 [0,1,1,0],
 [0,1,1,0],
 [1,0,0,1]
]
```

- Cities 1 and 4 are connected → one province
- Cities 2 and 3 are connected → another province
- No connection between these groups

Answer = 2

Example 2:

```
adj = [
 [1,0,1],
 [0,1,0],
 [1,0,1]
]
```

- Cities 1 and 3 are connected
- City 2 is isolated

Answer = 2

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void dfs(int node, vector<int> adjList[], int visited[]) {
        visited[node] = 1;
```

```
        for(auto it : adjList[node]) {
            if(!visited[it]) {
                dfs(it, adjList, visited);
            }
        }
    }

public:
    int numProvinces(vector<vector<int>> adj, int V) {
        vector<int> adjList[V];
        for(int i = 0; i < V; i++) {
            for(int j = 0; j < V; j++) {
                if(adj[i][j] == 1 && i != j) {
                    adjList[i].push_back(j);
                    adjList[j].push_back(i);
                }
            }
        }

        int visited[V] = {0};
        int count = 0;

        for(int i = 0; i < V; i++) {
            if(!visited[i]) {
                count++;
                dfs(i, adjList, visited);
            }
        }
        return count;
    }
};

int main() {
    vector<vector<int>> adj = {
        {1,0,1},
        {0,1,0},
        {1,0,1}
    };
```

```
    int V = 3;
    Solution obj;
    cout << obj.numProvinces(adj, V);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex is visited once, and all its edges are traversed during DFS.

- **Space Complexity:** `O(V)`
  Reason: Space used for visited array and recursion stack.

7. Connected Components

Given an **undirected graph** with `V` vertices numbered from `0` to `V-1` and a list of `E` edges, find the **number of connected components** in the graph.
Two vertices belong to the same connected component if there exists a **path** between them.

A connected component is a group of vertices where:

- Every vertex can reach every other vertex in the group.

- No vertex in the group is connected to a vertex outside the group.

Approach

To count connected components, we need to make sure every vertex is visited exactly once and grouped correctly.

1. Build an **adjacency list** from the given edge list.
2. Create a `visited` array of size `V` and initialize all values to `0`.
3. Initialize a counter `components = 0`.
4. Loop through all vertices from `0` to `V-1`:
    - If a vertex is not visited:
        - This means we found a new connected component.
        - Increment the component counter.
        - Perform **BFS** starting from this vertex.
        - During BFS, mark all reachable vertices as visited.
5. After all vertices are processed, return the component count.

Each BFS traversal covers exactly one connected component.

Example Explanation

Example 1:

```
V = 4
edges = [[0,1],[1,2]]
```

- Vertices `{0,1,2}` are connected → one component
- Vertex `{3}` is isolated → another component

Answer = 2

Example 2:

```
V = 7
edges = [[0,1],[1,2],[2,3],[4,5]]
```

- {0,1,2,3} form one component
- {4,5} form another component
- {6} is isolated

Answer = 3

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countComponents(int V, vector<vector<int>>& edges) {
        vector<vector<int>> adj(V);
        for(auto &e : edges) {
            adj[e[0]].push_back(e[1]);
            adj[e[1]].push_back(e[0]);
        }

        vector<int> visited(V, 0);
        int components = 0;

        for(int i = 0; i < V; i++) {
            if(!visited[i]) {
                components++;
                queue<int> q;
                q.push(i);
                visited[i] = 1;
                while(!q.empty()) {
                    int node = q.front();
                    q.pop();
                    for(auto nbr : adj[node]) {
                        if(!visited[nbr]) {
                            visited[nbr] = 1;
```

```
                            q.push(nbr);
                        }
                    }
                }
            }
        }
        return components;
    }
};

int main() {
    int V = 5;
    vector<vector<int>> edges = {{0,1},{1,2},{3,4}};
    Solution sol;
    cout << sol.countComponents(V, edges);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex is visited once and each edge is checked at most twice.

- **Space Complexity:** `O(V + E)`
  Reason: Space used for adjacency list, visited array, and queue.

8. Rotten Oranges : Min time to rot all oranges : BFS

Given an `n x m` grid where:

- 2 represents a **rotten orange**

- 1 represents a **fresh orange**
- 0 represents an **empty cell**

Every minute, a fresh orange becomes rotten if it is **adjacent (up, down, left, right)** to a rotten orange.
Return the **minimum number of minutes** required so that **no fresh orange remains**.
If it is **not possible** to rot all oranges, return −1.

Approach

Algorithm

The problem is solved using **Breadth First Search (BFS)** because rotting happens **level by level (minute by minute)**.

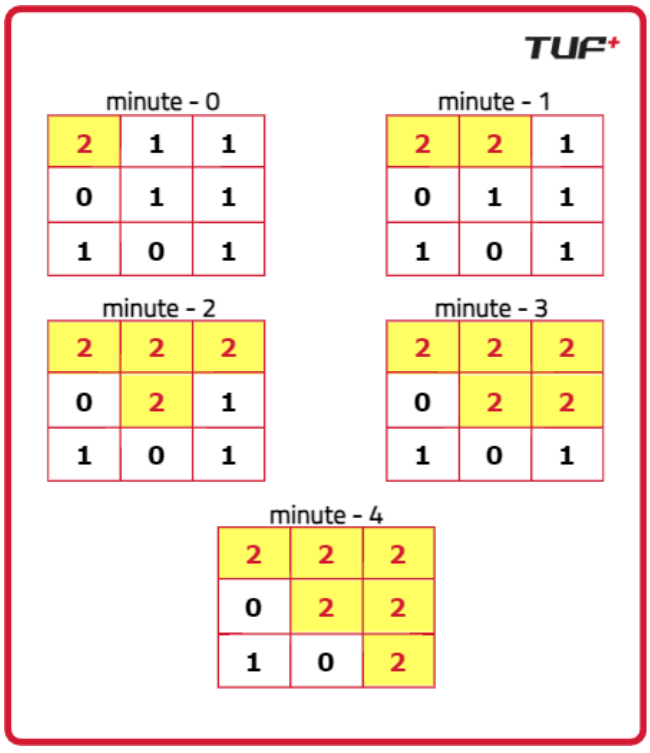
1. Create a **queue** to store positions of all initially rotten oranges.
2. Traverse the grid:
    - Count total number of oranges (`tot`) (fresh + rotten).
    - Push all rotten oranges into the queue.
3. Maintain:
    - `cnt` → number of oranges that become rotten during BFS
    - `days` → total minutes taken
4. Perform BFS:
    - For each level (one minute), process all currently rotten oranges.
    - For every rotten orange, check its 4 directions.
    - If a fresh orange is found, mark it rotten and push it into the queue.
    - Increase the rotten count.

5. After processing one level, if the queue is not empty, increment `days`.
6. After BFS ends:
    - If `cnt == tot`, return `days`
    - Otherwise, return `-1` (some fresh oranges are unreachable)

Example Explanation

Example 1:

```
grid = [
 [2,1,1],
 [0,1,1],
 [1,0,1]
]
```

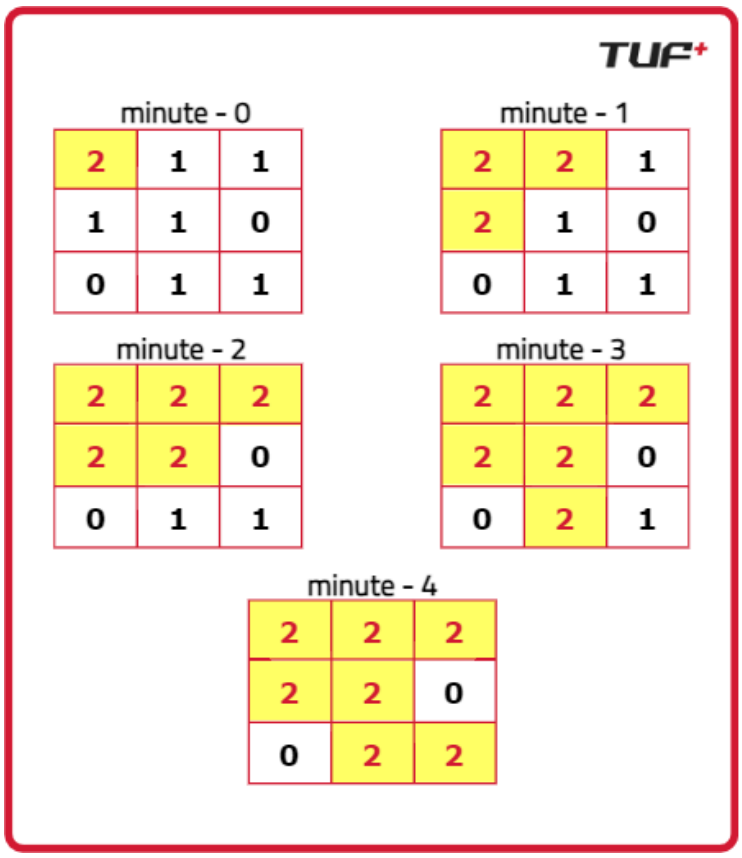


Orange at position `(2,0)` can never be reached by any rotten orange.
So answer is `-1`.

Example 2:

```
grid = [
 [2,1,1],
 [1,1,0],
 [0,1,1]
]
```

All oranges can be rotted level by level.
Total time required = 4.



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int orangesRotting(vector<vector<int>>& grid) {
    if(grid.empty()) return 0;

    int m = grid.size();
    int n = grid[0].size();
    int days = 0;
    int tot = 0;
    int cnt = 0;

    queue<pair<int,int>> rotten;

    for(int i = 0; i < m; i++) {
        for(int j = 0; j < n; j++) {
            if(grid[i][j] != 0) tot++;
            if(grid[i][j] == 2)
                rotten.push({i,j});
        }
    }

    int dx[4] = {0,0,1,-1};
    int dy[4] = {1,-1,0,0};

    while(!rotten.empty()) {
        int k = rotten.size();
        cnt += k;
        while(k--) {
            int x = rotten.front().first;
            int y = rotten.front().second;
            rotten.pop();
            for(int i = 0; i < 4; i++) {
                int nx = x + dx[i];
                int ny = y + dy[i];
                if(nx < 0 || ny < 0 || nx >= m || ny >= n ||
grid[nx][ny] != 1)
                    continue;
                grid[nx][ny] = 2;
                rotten.push({nx, ny});
            }
```

```
        }
        if(!rotten.empty())
            days++;
    }
    return tot == cnt ? days : -1;
}

int main() {
    vector<vector<int>> grid = {
        {2,1,1},
        {1,1,0},
        {0,1,1}
    };
    cout << orangesRotting(grid);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n × m)`
  Reason: Each cell is processed once and each time we check 4 directions.

- **Space Complexity:** `O(n × m)`
  Reason: In the worst case, all oranges are stored in the queue at once.

9. Flood Fill Algorithm - Graphs

An image is represented as a **2D grid of integers**, where each integer represents a pixel color. Given a starting pixel `(sr, sc)` and a `newColor`, perform **flood fill** such that all pixels **connected in 4 directions** (up, right, down, left) having the **same initial color** as the starting pixel are changed to `newColor`.

Approach

Algorithm

1. Store the **initial color** of the starting pixel `(sr, sc)`.
2. Create a **copy of the image** so the original image is not modified directly.
3. Use **DFS traversal** starting from `(sr, sc)`.
4. At each pixel:
    - Change its color to `newColor`.
    - Explore its 4 neighbors (up, right, down, left).
5. For each neighbor:
    - Check if it is inside the grid.
    - Check if it has the **same initial color**.
    - Check if it is **not already colored** with `newColor`.
6. Recursively apply DFS on valid neighbors.
7. Continue until all connected pixels with the initial color are filled.
8. Return the updated image.

Example Explanation

For the image:

```
1 1 1
1 1 0
1 0 1
```

Starting at `(1,1)` with `newColor = 2`:

- Initial color = 1
- All connected 1s reachable from $(1, 1)$ are changed to 2
- Cells with `0` block the fill

Result:

```
2 2 2
2 2 0
2 0 1
```

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void dfs(int row,int col,vector<vector<int>>& ans,
             vector<vector<int>>& image,int newColor,
             int delRow[],int delCol[],int iniColor) {
        ans[row][col] = newColor;
        int n = image.size();
        int m = image[0].size();
        for(int i = 0; i < 4; i++) {
            int nrow = row + delRow[i];
            int ncol = col + delCol[i];
            if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m &&
               image[nrow][ncol] == iniColor &&
               ans[nrow][ncol] != newColor) {
                dfs(nrow, ncol, ans, image, newColor, delRow, delCol,
iniColor);
            }
        }
    }
public:
```

```
    vector<vector<int>> floodFill(vector<vector<int>>& image,
                                  int sr,int sc,int newColor) {
        int iniColor = image[sr][sc];
        vector<vector<int>> ans = image;
        int delRow[] = {-1,0,1,0};
        int delCol[] = {0,1,0,-1};
        dfs(sr, sc, ans, image, newColor, delRow, delCol, iniColor);
        return ans;
    }
};

int main() {
    vector<vector<int>> image = {
        {1,1,1},
        {1,1,0},
        {1,0,1}
    };
    Solution obj;
    vector<vector<int>> ans = obj.floodFill(image, 1, 1, 2);
    for(auto i : ans) {
        for(auto j : i)
            cout << j << " ";
        cout << "\n";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N \times M)$
  Reason: In the worst case, DFS visits every cell once and checks 4 directions.

- **Space Complexity:** $O(N \times M)$
  Reason: Extra space for the copied image and recursion stack in the worst case.

10. Detect Cycle in an Undirected Graph (using BFS)

Given an **undirected graph** with `V` vertices and `E` edges, check whether the graph contains **any cycle**.
A cycle exists if we can start from a node and come back to the **same node through a different path**.

Algorithm

We use **Breadth First Search (BFS)** with **parent tracking**.

1. Create a `visited` array of size `V` and initialize all values to `0`.
2. Since the graph can have multiple connected components, loop through all vertices.
3. For every unvisited vertex, start a BFS:
    - Push a pair `(node, parent)` into the queue.
    - Mark the node as visited.
4. While the queue is not empty:
    - Pop the front element `(node, parent)`.
    - Traverse all adjacent nodes of `node`.
    - If an adjacent node is **not visited**:
        - Mark it visited.
        - Push `(adjacentNode, node)` into the queue.
    - Else if the adjacent node is visited **and it is not the parent**:
        - A cycle is detected, return `true`.

5. If BFS finishes for all components and no cycle is found, return `false`.

The parent is required to avoid detecting the same edge in reverse direction as a cycle.

Example Explanation

Example:

```
Edges: 4 -> 5 -> 6 -> 4
```

- Start BFS from node 4
- Visit 5, then 6
- From 6, we again reach 4 which is already visited and not the parent
- This confirms the presence of a cycle

If no such situation occurs, then the graph has no cycle.

If you visit a vertex that's already visited AND it's not your parent → cycle!

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    bool detect(int src, vector<int> adj[], int vis[]) {
        vis[src] = 1;
        queue<pair<int,int>> q;
        q.push({src, -1}); //(node, parent)
        while(!q.empty()) {
            int node = q.front().first;
            int parent = q.front().second;
```

```
            q.pop();
            for(auto adjacentNode : adj[node]) {
                if(!vis[adjacentNode]) {
                    vis[adjacentNode] = 1;
                    q.push({adjacentNode, node});
                }
                else if(adjacentNode != parent) {
                    return true;
                }
            }
        }
        return false;
    }

public:
    bool isCycle(int V, vector<int> adj[]) {
        int vis[V] = {0};
        for(int i = 0; i < V; i++) {
            if(!vis[i]) {
                if(detect(i, adj, vis))
                    return true;
            }
        }
        return false;
    }
};

int main() {
    vector<int> adj[4] = {{}, {2}, {1, 3}, {2}};
    Solution obj;
    bool ans = obj.isCycle(4, adj);
    if(ans) cout << "1";
    else cout << "0";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N + 2E) + O(N)`
  Reason: BFS traverses all nodes and edges, and an extra loop handles disconnected components.

- **Space Complexity:** `O(N)`
  Reason: Space used by visited array and queue.

11. Detect Cycle in an Undirected Graph (using DFS)

Given an **undirected graph** with `V` vertices and `E` edges, check whether the graph contains **any cycle** using **Depth First Search (DFS)**.

In an undirected graph, a cycle exists if during traversal we reach a **visited node that is not the parent** of the current node.

Approach

Algorithm

1. Create a `visited` array to mark visited vertices.

2. Since the graph can have **multiple connected components**, iterate over all vertices.

3. For every unvisited vertex:

    - Start a DFS with its parent as `-1`.

4. In DFS:

    - Mark the current node as visited.

- Traverse all its adjacent nodes.
- If an adjacent node is not visited, recursively call DFS with current node as parent.
- If an adjacent node is already visited and is **not the parent**, a cycle exists.

5. If DFS finishes without detecting such a case, no cycle exists.
6. If any DFS call detects a cycle, return `true`. Otherwise, return `false`.

Example Explanation

Example:

`4 -> 5 -> 6 -> 4`

- Start DFS from node 4
- Visit 5, then 6
- From 6, we reach 4 again
- 4 is visited and is not the parent of 6
- Hence, a cycle is detected

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool dfs(int node, int parent, vector<int> adj[], vector<int>&
visited) {
        visited[node] = 1;
        for(int neighbor : adj[node]) {
```

```
            if(!visited[neighbor]) {
                if(dfs(neighbor, node, adj, visited))
                    return true;
            }
            else if(neighbor != parent) {
                return true;
            }
        }
        return false;
    }

    bool isCycle(int V, vector<int> adj[]) {
        vector<int> visited(V, 0);
        for(int i = 0; i < V; i++) {
            if(!visited[i]) {
                if(dfs(i, -1, adj, visited)) // -1 is parent
                    return true;
            }
        }
        return false;
    }
};

int main() {
    int V = 5;
    vector<int> adj[V];
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[2].push_back(3);
    adj[3].push_back(2);
    adj[3].push_back(4);
    adj[4].push_back(3);
    adj[4].push_back(1);

    Solution sol;
    if(sol.isCycle(V, adj))
```

```
        cout << "Cycle detected";
    else
        cout << "No cycle found";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex is visited once and each edge is explored during DFS.

- **Space Complexity:** `O(V + E)`
  Reason: Space used for adjacency list, visited array, and recursion stack.

12. Distance of Nearest Cell having 1

Given a binary grid of size `N x M`, find the **distance of the nearest cell having value 1 for every cell** in the grid.
Distance is measured using **4-directional movement** (up, right, down, left).

Approach

Algorithm

We use **Breadth First Search (BFS)** because BFS naturally expands **level by level**, which directly represents distance.

1. Create:
    - `visited` matrix to mark visited cells

- `dist` matrix to store distance of nearest 1
- Queue to store `((row, col), steps)`

2. Traverse the grid:
    - If a cell contains 1, push it into the queue with distance `0`
    - Mark it as visited
3. Start BFS:
    - Pop a cell from the queue
    - Store its distance in the `dist` matrix
    - Explore its 4 neighbors
4. For every valid, unvisited neighbor:
    - Mark it visited
    - Push it into the queue with `steps + 1`
5. Continue until the queue becomes empty.
6. Return the distance matrix.

All 1 cells act as **multiple BFS sources**, ensuring the shortest distance is found.

Example Explanation

Input:

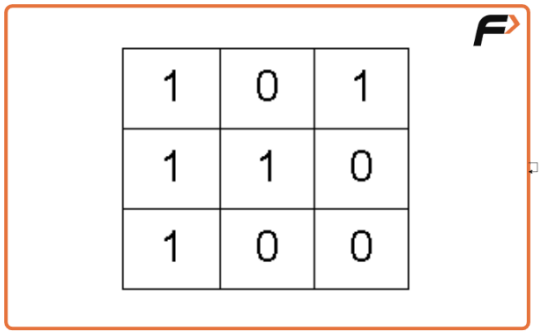
```
0 1 1 0
1 1 0 0
0 0 1 1
```

- All cells containing 1 are pushed into the queue with distance `0`

- BFS spreads to nearby `0` cells level by level
- Each `0` cell gets the distance of the nearest 1

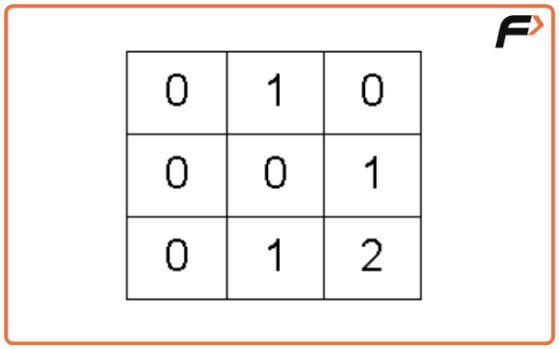
`Ex 2:`

`Input`



1	0	1
1	1	0
1	0	0

`Output`



0	1	0
0	0	1
0	1	2

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<int>> nearest(vector<vector<int>> grid) {
        int n = grid.size();
        int m = grid[0].size();
        vector<vector<int>> vis(n, vector<int>(m, 0));
```

```
        vector<vector<int>> dist(n, vector<int>(m, 0));
        queue<pair<pair<int,int>, int>> q;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1) {
                    q.push({{i, j}, 0});
                    vis[i][j] = 1;
                }
            }
        }

        int delrow[] = {-1, 0, 1, 0};
        int delcol[] = {0, 1, 0, -1};

        while(!q.empty()) {
            int row = q.front().first.first;
            int col = q.front().first.second;
            int steps = q.front().second;
            q.pop();
            dist[row][col] = steps;

            for(int i = 0; i < 4; i++) {
                int nrow = row + delrow[i];
                int ncol = col + delcol[i];
                if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m &&
                   vis[nrow][ncol] == 0) {
                    vis[nrow][ncol] = 1;
                    q.push({{nrow, ncol}, steps + 1});
                }
            }
        }
        return dist;
    }
};

int main() {
    vector<vector<int>> grid = {
```

```
        {0,1,1,0},
        {1,1,0,0},
        {0,0,1,1}
    };
    Solution obj;
    vector<vector<int>> ans = obj.nearest(grid);
    for(auto i : ans) {
        for(auto j : i)
            cout << j << " ";
        cout << "\n";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N × M)`
  Reason: Each cell is visited once and each visit checks 4 neighbors.

- **Space Complexity:** `O(N × M)`
  Reason: Space used for visited matrix, distance matrix, and queue.

13. Surrounded Regions | Replace O's with X's

Given a matrix `mat` of size `N x M` where each cell contains either `'O'` or `'X'`.
Replace all `'O'` with `'X'` **only if the `'O'` is completely surrounded by `'X'` from all 4 directions** (up, down, left, right).

An `'O'` (or group of `'O'`) **connected to the boundary** of the matrix can **never** be surrounded, so it must **remain `'O'`**.

Example

Example 1

Input:

```
X X X X
X 0 X X
X 0 0 X
X 0 X X
X X 0 0
```

Explanation:

- The `'0'` at the bottom-right is connected to the boundary → cannot be replaced.
- The group of `'0'` in the middle is fully surrounded by `'X'`.

Output:

```
X X X X
X X X X
X X X X
X X X X
X X 0 0
```

This example shows why **boundary-connected `'0'` must be protected first**.

Approach

Algorithm

1. Any `'0'` connected to the **boundary** is **safe** and cannot be replaced.

2. Start DFS from all boundary cells that contain `'O'`.
3. Mark all `'O'` cells reachable from the boundary as **visited**.
4. Traverse the entire matrix:
    - If a cell is `'O'` and **not visited**, replace it with `'X'`.
    - Visited `'O'` remain unchanged.

This ensures only **fully surrounded regions** are replaced.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    void dfs(int row,int col,vector<vector<int>>& vis,
             vector<vector<char>>& mat,int delrow[],int delcol[]) {
        vis[row][col] = 1;
        int n = mat.size(), m = mat[0].size();
        for(int k = 0; k < 4; k++) {
            int nrow = row + delrow[k];
            int ncol = col + delcol[k];
            if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m &&
               !vis[nrow][ncol] && mat[nrow][ncol] == 'O') {
                dfs(nrow, ncol, vis, mat, delrow, delcol);
            }
        }
    }

public:
    vector<vector<char>> fill(int n,int m,vector<vector<char>> mat) {
        vector<vector<int>> vis(n, vector<int>(m, 0));
        int delrow[4] = {-1, 0, 1, 0};
        int delcol[4] = {0, 1, 0, -1};
```

```
        // first and last row
        for(int j = 0; j < m; j++) {
            if(mat[0][j] == 'O' && !vis[0][j])
                dfs(0, j, vis, mat, delrow, delcol);
            if(mat[n-1][j] == 'O' && !vis[n-1][j])
                dfs(n-1, j, vis, mat, delrow, delcol);
        }

        // first and last column
        for(int i = 0; i < n; i++) {
            if(mat[i][0] == 'O' && !vis[i][0])
                dfs(i, 0, vis, mat, delrow, delcol);
            if(mat[i][m-1] == 'O' && !vis[i][m-1])
                dfs(i, m-1, vis, mat, delrow, delcol);
        }

        // replace surrounded O's
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(mat[i][j] == 'O' && !vis[i][j])
                    mat[i][j] = 'X';
            }
        }
        return mat;
    }
};

int main() {
    vector<vector<char>> mat{
        {'X','X','X','X'},
        {'X','O','X','X'},
        {'X','O','O','X'},
        {'X','O','X','X'},
        {'X','X','O','O'}
    };
    Solution ob;
```

```
    vector<vector<char>> ans = ob.fill(mat.size(), mat[0].size(),
mat);
    for(auto i : ans) {
        for(auto j : i) cout << j << " ";
        cout << "\n";
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N × M)`
  Reason: Each cell is visited at most once in DFS and once in final traversal.

- **Space Complexity:** `O(N × M)`
  Reason: Visited matrix and recursion stack in the worst case.

14. Number of Enclaves

You are given an `N x M` binary grid:

- `0` represents **sea**

- 1 represents **land**

A move is allowed **only between adjacent land cells (up, down, left, right)** or you can walk **out of the boundary** of the grid.

A **land cell is called an enclave** if **it is not possible to walk off the boundary starting from that cell**, no matter how many moves you make.

Your task is to **count the number of such land cells**.

Example

Example 1

Input:

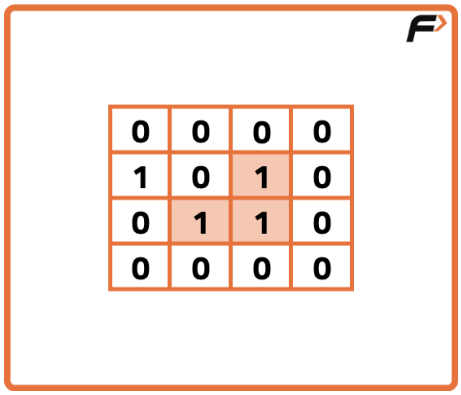
```
0 0 0 0
1 0 1 0
0 1 1 0
0 0 0 0
```

Explanation:

- Land cells connected to the boundary are **not enclaves**
- The land cells `(1,2)`, `(2,1)`, `(2,2)` are **completely surrounded**
- From these cells, we cannot reach the boundary

Output:

3



This example shows that **only land cells not connected to the boundary are counted**.

Approach

Algorithm

The key observation is:

- **Any land cell connected to the boundary cannot be an enclave**

Steps:

1. Create a `visited` matrix to track visited land cells.
2. Traverse all **boundary cells**:
    - If a boundary cell is land (1), mark it visited and push it into a queue.
3. Perform **BFS** from these boundary land cells:
    - Move in 4 directions
    - Mark all reachable land cells as visited
4. After BFS:
    - Traverse the grid
    - Count land cells (1) that are **not visited**
5. Return the count

Unvisited land cells are exactly the **enclaves**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    int numberOfEnclaves(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();

        vector<vector<int>> vis(n, vector<int>(m, 0));
        queue<pair<int,int>> q;

        // push all boundary land cells
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(i == 0 || j == 0 || i == n-1 || j == m-1) {
                    if(grid[i][j] == 1) {
                        vis[i][j] = 1;
                        q.push({i, j});
                    }
                }
            }
        }

        int delrow[] = {-1, 0, 1, 0};
        int delcol[] = {0, 1, 0, -1};

        // BFS from boundary land cells
        while(!q.empty()) {
            auto it = q.front();
            q.pop();
            int row = it.first;
            int col = it.second;

            for(int k = 0; k < 4; k++) {
                int nrow = row + delrow[k];
                int ncol = col + delcol[k];

                if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m &&
                   !vis[nrow][ncol] && grid[nrow][ncol] == 1) {
                    vis[nrow][ncol] = 1;
                    q.push({nrow, ncol});
```

```
                }
            }
        }

        // count enclaves
        int cnt = 0;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(grid[i][j] == 1 && vis[i][j] == 0)
                    cnt++;
            }
        }
        return cnt;
    }
};

int main() {
    vector<vector<int>> grid{
        {0,0,0,0},
        {1,0,1,0},
        {0,1,1,0},
        {0,0,0,0}
    };
    Solution obj;
    cout << obj.numberOfEnclaves(grid);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N \times M)$
  Reason: Each cell is visited at most once during BFS and counting.

- **Space Complexity:** $O(N \times M)$
  Reason: Visited matrix and BFS queue can store all cells in the worst case.

15. Word Ladder - I : G-29

Given two **distinct words** `startWord` and `targetWord`, and a list `wordList` containing **unique words of equal length**, find the **length of the shortest transformation sequence** from `startWord` to `targetWord`.

Rules for transformation:

- Only **one character** can be changed at a time.
- Each transformed word **must exist in `wordList`**, including `targetWord`.
- All words contain **only lowercase letters**.
- `startWord` may or may not be present in `wordList`.
- If no such transformation is possible, return `0`.

Example

Example 1

Input:

```
wordList = {"des","der","dfr","dgt","dfs"}
startWord = "der"
targetWord = "dfs"
```

Explanation:

`"der" → "dfr" → "dfs"`

- Change e to f → `"dfr"`
- Change r to s → `"dfs"`

All intermediate words exist in `wordList`.

Output:

3

Example 2

Input:

```
wordList = {"geek","gefk"}
startWord = "gedk"
targetWord = "geek"
```

Explanation:

`"gedk" → "geek"`

- Change d to e

Output:

2

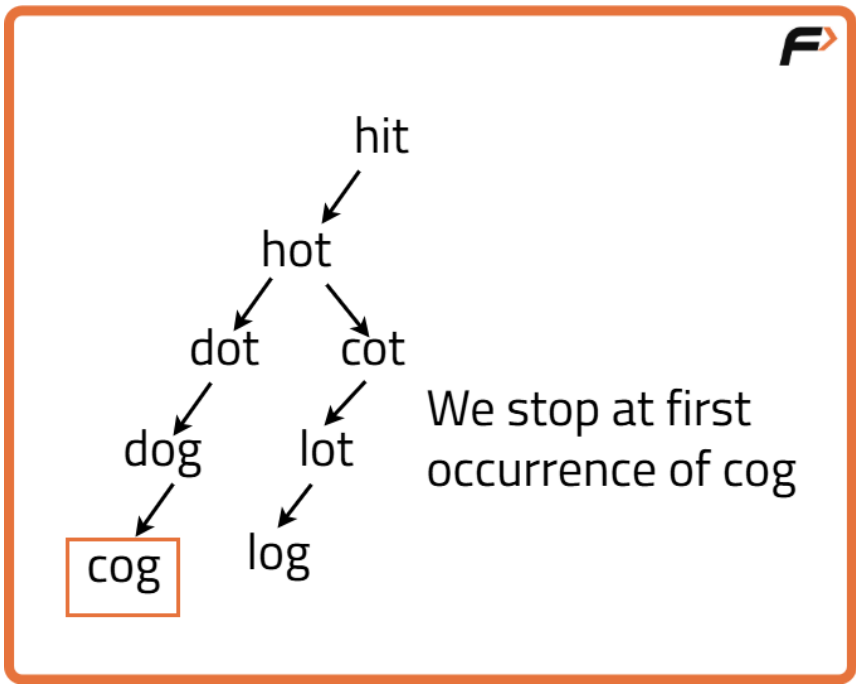
These examples show that we must find the **minimum number of valid transformations**.

Approach

This problem can be solved using **Breadth First Search (BFS)** because BFS always finds the **shortest path**.

1. Treat each word as a **node**.
2. Two words are connected if they differ by **exactly one character**.

3. Push `{startWord, 1}` into a queue where 1 is the starting step.
4. Store all words from `wordList` in an `unordered_set` for fast lookup.
5. While the queue is not empty:
    - Pop the front word and its current step count.
    - If the word equals `targetWord`, return the step count.
    - For each character in the word:
        - Try replacing it with all characters from `'a'` to `'z'`.
        - If the new word exists in the set:
            - Remove it from the set (to avoid revisiting).
            - Push it into the queue with `steps + 1`.
6. If BFS ends without reaching `targetWord`, return `0`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int wordLadderLength(string startWord, string targetWord,
vector<string>& wordList) {
        queue<pair<string,int>> q;
        q.push({startWord, 1});

        unordered_set<string> st(wordList.begin(), wordList.end());
        st.erase(startWord);

        while(!q.empty()) {
            string word = q.front().first;
            int steps = q.front().second;
            q.pop();

            if(word == targetWord) return steps;

            for(int i = 0; i < word.size(); i++) {
                char original = word[i];
                for(char ch = 'a'; ch <= 'z'; ch++) {
                    word[i] = ch;
                    if(st.find(word) != st.end()) {
                        st.erase(word);
                        q.push({word, steps + 1});
                    }
                }
                word[i] = original;
            }
        }
        return 0;
    }
};

int main() {
```

```
    vector<string> wordList = {"des","der","dfr","dgt","dfs"};
    string startWord = "der";
    string targetWord = "dfs";

    Solution obj;
    cout << obj.wordLadderLength(startWord, targetWord, wordList);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N × L × 26)`
  Reason: For each word, we try changing all L characters to 26 possible letters.

- **Space Complexity:** `O(N × L)`
  Reason: Space used by the `unordered_set` and BFS queue.

16. G-30 : Word Ladder-II

Given two **distinct words** `startWord` and `targetWord`, and a list `wordList` containing **unique words of equal length**, find **all the shortest transformation sequences** from `startWord` to `targetWord`.

Rules:

- Only **one letter** can be changed at a time.
- Each transformed word **must exist in `wordList`**, including `targetWord`.
- All words contain **only lowercase letters**.
- `startWord` may or may not be present in `wordList`.

- If no valid transformation exists, return an **empty list**.

Ek BFS level ke andar words delete nahi karte
Level complete hone ke baad delete karte hain

Examples

Example 1

Input:

```
startWord = "der"
targetWord = "dfs"
wordList = {"des","der","dfr","dgt","dfs"}
```

Explanation:

Shortest transformation length = **3**

All shortest sequences are:

```
der → des → dfs
der → dfr → dfs
```

So the output is:

```
[ ["der","dfr","dfs"], ["der","des","dfs"] ]
```

This example shows that **multiple shortest paths** can exist.

Example 2

Input:

```
startWord = "gedk"
targetWord = "geek"
wordList = {"geek","gefk"}
```

Explanation:

`gedk → geek`

Only one shortest sequence exists.

Output:

`[ ["gedk","geek"] ]`

Algorithm

This problem is an extension of **Word Ladder I**.
Instead of finding only the length, we must find **all shortest transformation sequences**.

We use **BFS with paths**:

1. Insert all words from `wordList` into an `unordered_set` for fast lookup.
2. Use a **queue of vector<string>**, where each vector represents a transformation sequence.
3. Push `{startWord}` as the initial sequence into the queue.
4. Maintain:
    - `level` → current BFS depth
    - `usedOnLevel` → words used at the current level
5. While the queue is not empty:
    - Pop a sequence.
    - If the sequence length increases, remove all words used in the previous level from the set.
    - Take the **last word** of the sequence.

- If it equals `targetWord`:
    - Store it if it is the first found sequence
    - Or if its length equals already found shortest sequences
- Try changing each character of the word from `'a'` to `'z'`:
    - If the new word exists in the set:
        - Add it to the sequence
        - Push the new sequence into the queue
        - Mark the word as used on this level

6. After BFS ends, return all stored sequences.

BFS guarantees that we only collect **shortest sequences**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<string>> findSequences(string beginWord, string
endWord,
                                         vector<string>& wordList) {
        unordered_set<string> st(wordList.begin(), wordList.end());
        queue<vector<string>> q;
        q.push({beginWord});

        vector<string> usedOnLevel;
        usedOnLevel.push_back(beginWord);
        int level = 0;

        vector<vector<string>> ans;
```

```
        while(!q.empty()) {
            vector<string> vec = q.front();
            q.pop();

            if(vec.size() > level) {
                level++;
                for(auto it : usedOnLevel)
                    st.erase(it);
                usedOnLevel.clear();
            }

            string word = vec.back();

            if(word == endWord) {
                if(ans.size() == 0)
                    ans.push_back(vec);
                else if(ans[0].size() == vec.size())
                    ans.push_back(vec);
            }

            for(int i = 0; i < word.size(); i++) {
                char original = word[i];
                for(char c = 'a'; c <= 'z'; c++) {
                    word[i] = c;
                    if(st.count(word)) {
                        vec.push_back(word);
                        q.push(vec);
                        usedOnLevel.push_back(word);
                        vec.pop_back();
                    }
                }
                word[i] = original;
            }
        }
        return ans;
    }
};
```

```
bool comp(vector<string> a, vector<string> b) {
    string x = "", y = "";
    for(string i : a) x += i;
    for(string i : b) y += i;
    return x < y;
}

int main() {
    vector<string> wordList = {"des","der","dfr","dgt","dfs"};
    string startWord = "der", targetWord = "dfs";

    Solution obj;
    vector<vector<string>> ans = obj.findSequences(startWord,
targetWord, wordList);

    if(ans.size() == 0)
        cout << -1 << endl;
    else {
        sort(ans.begin(), ans.end(), comp);
        for(auto seq : ans) {
            for(auto word : seq)
                cout << word << " ";
            cout << endl;
        }
    }
    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
  $O(N \times L \times 26 + S \times L)$
  Reason:
    - For each word, we try L × 26 transformations

    - S shortest sequences of length L are stored

- **Space Complexity:**
  `O(N × L + S × L)`
  Reason:
    - Queue stores full paths
    - Set stores unused words
    - Result stores all shortest sequences

17. Number of Islands

Given a grid of size `N x M` consisting of:

- `'1'` → Land
- `'0'` → Water

Find the **number of islands**.

An island is formed by connecting **adjacent land cells (`'1'`) in all 8 directions**:

- Up, Down, Left, Right
- All 4 diagonals

Water (`'0'`) separates islands.

Example

Example 1

Input:

```
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
```

Explanation:

- Top-left block of 1s → Island 1
- Single 1 in middle → Island 2
- Bottom-right connected 1s → Island 3

Output:

3

Example 2

Input:

```
1 1 1
1 1 1
1 1 1
```

Explanation:

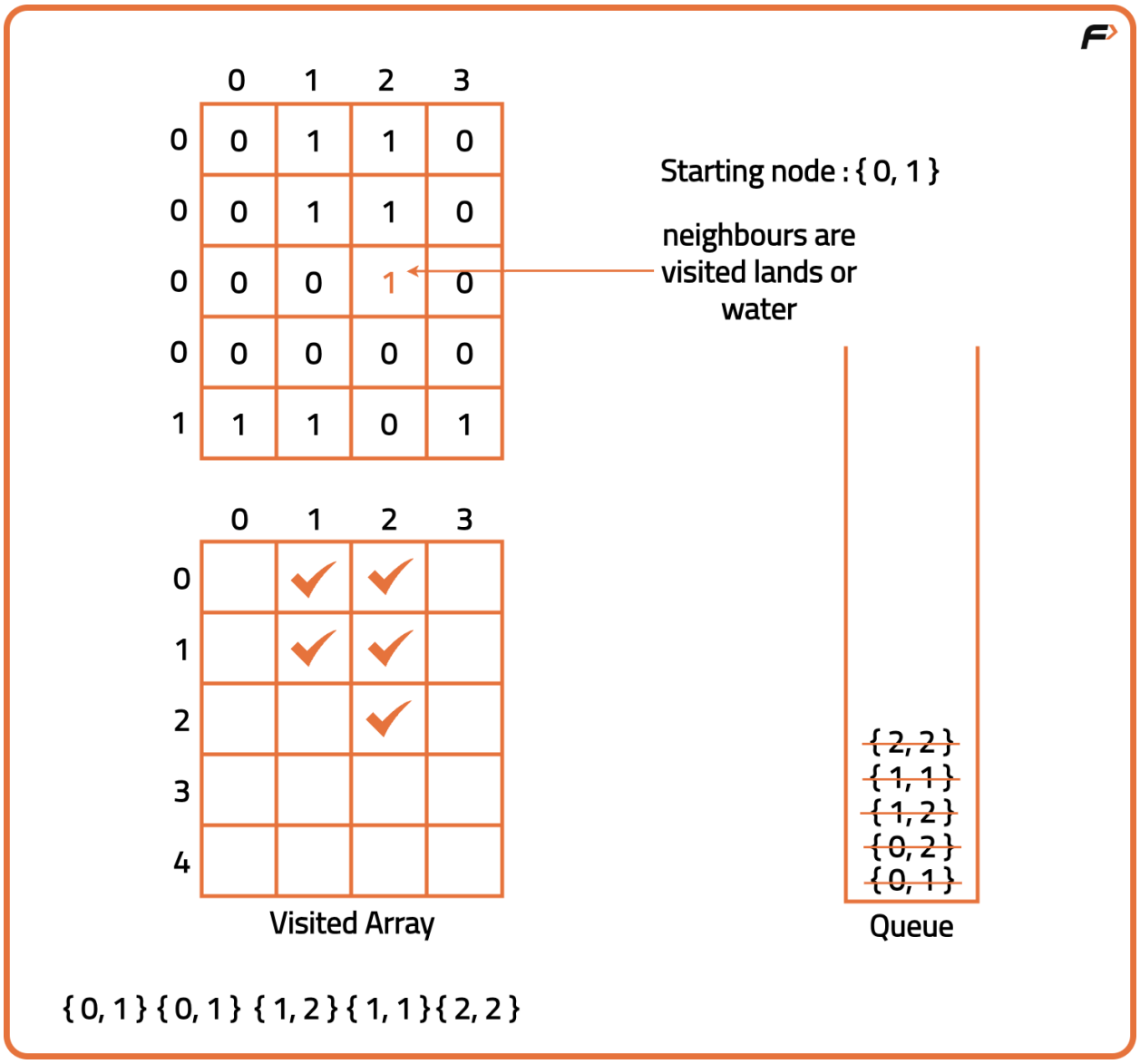
- All land cells are connected (including diagonals)

Output:

1

These examples show that **diagonal connections also matter**.

Approach



Algorithm

An island is a **connected component** of land cells (' 1 ') considering **8-directional connectivity**.

We use **Breadth First Search (BFS)**.

1. Create a `visited` matrix to mark visited cells.
2. Traverse the entire grid.
3. Whenever an **unvisited land cell (`'1'`)** is found:
    - Increment island count.
    - Start BFS from that cell.
4. BFS will:
    - Visit all land cells connected in 8 directions.
    - Mark them as visited.
5. Continue scanning the grid.
6. The number of BFS calls equals the number of islands.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void bfs(int row, int col, vector<vector<int>>& vis,
             vector<vector<char>>& grid) {

        queue<pair<int,int>> q;
        q.push({row, col});
        vis[row][col] = 1;

        int delRow[] = {-1,-1,-1,0,1,1,1,0};
        int delCol[] = {-1,0,1,1,1,0,-1,-1};
```

```
        while(!q.empty()) {
            int r = q.front().first;
            int c = q.front().second;
            q.pop();

            for(int i = 0; i < 8; i++) {
                int nrow = r + delRow[i];
                int ncol = c + delCol[i];

                if(nrow >= 0 && nrow < grid.size() &&
                   ncol >= 0 && ncol < grid[0].size() &&
                   !vis[nrow][ncol] &&
                   grid[nrow][ncol] == '1') {

                    vis[nrow][ncol] = 1;
                    q.push({nrow, ncol});
                }
            }
        }
    }

    int numIslands(vector<vector<char>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        vector<vector<int>> vis(n, vector<int>(m, 0));
        int count = 0;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(!vis[i][j] && grid[i][j] == '1') {
                    count++;
                    bfs(i, j, vis, grid);
                }
            }
        }
        return count;
    }
};
```

```
int main() {
    vector<vector<char>> grid = {
        {'1','1','0','0','0'},
        {'1','1','0','0','0'},
        {'0','0','1','0','0'},
        {'0','0','0','1','1'}
    };

    Solution obj;
    cout << obj.numIslands(grid);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N \times M)$
  Reason: Every cell is visited at most once.

- **Space Complexity:** $O(N \times M)$
  Reason: Visited matrix and BFS queue in the worst case.

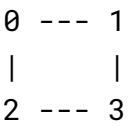
18. Bipartite Graph | DFS Implementation

Given an **undirected graph** with V vertices (0-based indexing) represented using an **adjacency list**, check whether the graph is **bipartite**.

A graph is called **bipartite** if we can color all vertices using **only two colors** such that **no two adjacent vertices have the same color**.

Example 1

Input:



Explanation:

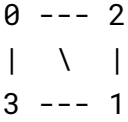
- We can color the graph using 2 colors:
    - Color nodes `{0,2}` with color 0
    - Color nodes `{1,3}` with color 1
- No adjacent nodes have the same color

Output:

`1`

Example 2

Input:

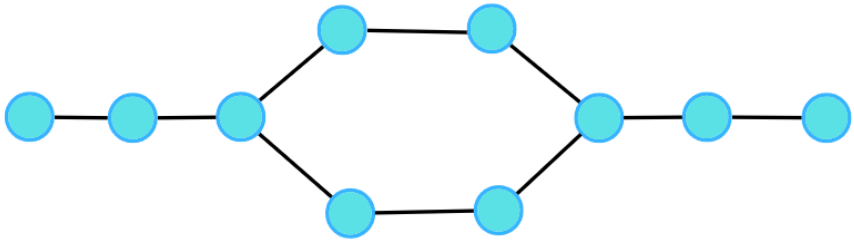


Explanation:

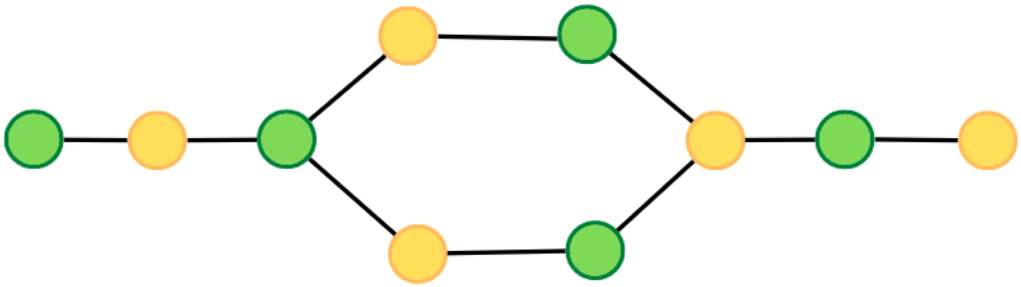
- This graph contains an **odd-length cycle**
- While coloring, two adjacent nodes are forced to have the same color
- Hence, it cannot be bipartite

Output:

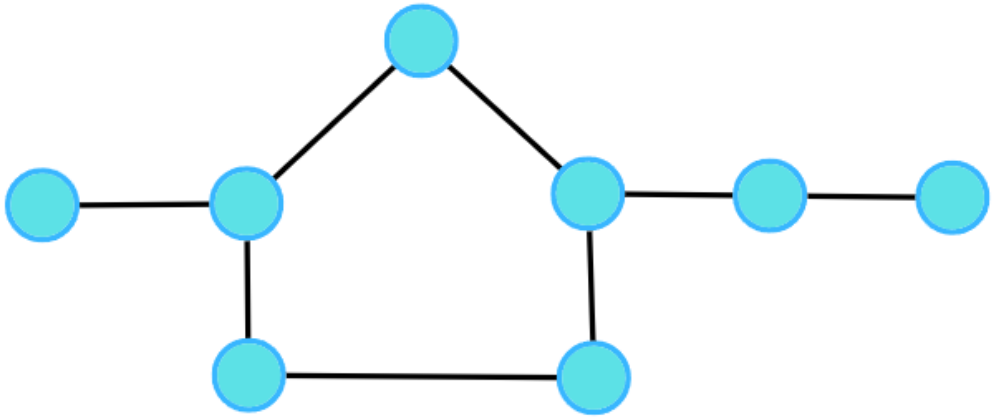
0



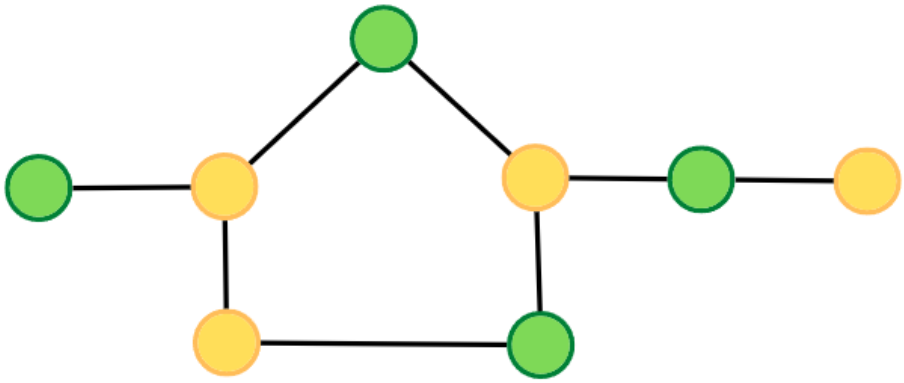
Output 1



Input



Output 0



These examples show that **graphs with odd cycles are not bipartite**, while graphs without odd cycles can be bipartite.

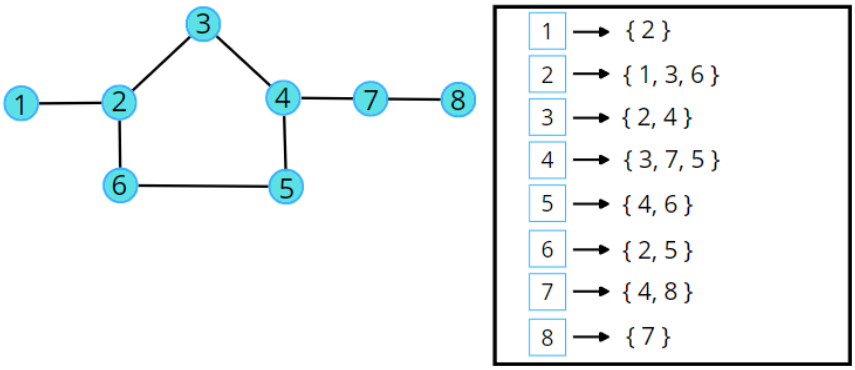
Approach

Algorithm

We use **Depth First Search (DFS)** with coloring.

1. Create a `color` array of size `V`, initialize all values to `-1` (uncolored).
2. Since the graph can have **multiple connected components**, iterate over all vertices.
3. For every uncolored vertex:
    - Start DFS and assign it color `0`.
4. In DFS:
    - Assign the current node the given color.
    - Traverse all adjacent nodes:
        - If an adjacent node is uncolored, assign it the **opposite color** and continue DFS.
        - If an adjacent node is already colored with the **same color**, return `false`.
5. If DFS completes successfully for all components, return `true`.

DFS ensures we try to color the graph consistently. Any conflict means the graph is **not bipartite**.



Adjacency List

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    bool dfs(int node, int col, int color[], vector<int> adj[]) {
        color[node] = col;
        for(auto it : adj[node]) {
            if(color[it] == -1) {
                if(!dfs(it, !col, color, adj))
                    return false;
            }
            else if(color[it] == col) {
                return false;
            }
        }
        return true;
    }
```

```
public:
    bool isBipartite(int V, vector<int> adj[]) {
        int color[V];
        for(int i = 0; i < V; i++) color[i] = -1;

        for(int i = 0; i < V; i++) {
            if(color[i] == -1) {
                if(!dfs(i, 0, color, adj))
                    return false;
            }
        }
        return true;
    }
};

void addEdge(vector<int> adj[], int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    vector<int> adj[4];
    addEdge(adj, 0, 2);
    addEdge(adj, 0, 3);
    addEdge(adj, 2, 3);
    addEdge(adj, 3, 1);

    Solution obj;
    bool ans = obj.isBipartite(4, adj);
    if(ans) cout << "1\n";
    else cout << "0\n";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + 2E)`
  Reason: Each vertex and each edge is visited once during DFS.

- **Space Complexity:** `O(V)`
  Reason: Space used by color array and recursion stack.

19. Detect cycle in a directed graph (using DFS) : G-19

Given a **directed graph** with V vertices and E edges, check whether the graph contains **any cycle** or not.

In a directed graph, a cycle exists **only if during traversal we visit a node again on the same DFS path**.

Example 1

Input:

`8 → 9 → 10 → 8`

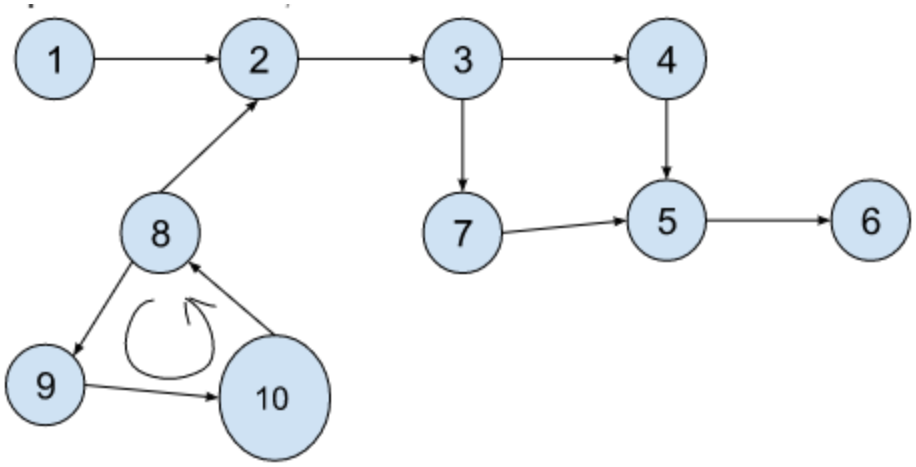
Explanation:

- Starting from node 8, we reach 9, then 10, and again come back to 8
- The node 8 is revisited on the **same path**

Output:

`true`

Example:



`N = 10, E = 11`

`Output:` `true`

`Explanation:` `8->9->10 is a cycle.`

Example 2

Input:

1 → 2 → 3 → 4

Explanation:

- No node is revisited on the same DFS path
- The graph has no cycle

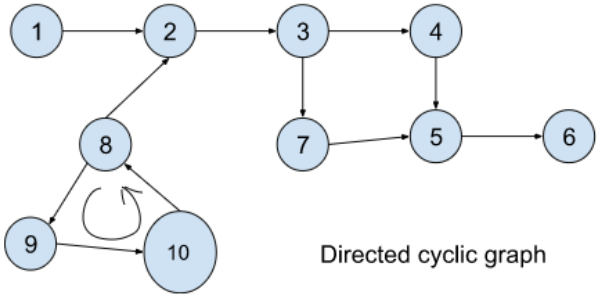
Output:

`false`

These examples show that **revisiting a node via a different path is not a cycle**, but revisiting it **on the same path is**.

Approach

Algorithm



To detect a cycle in a **directed graph**, we use **DFS with two visited arrays**.

1. Create two arrays:
    - `vis[ ]` → marks nodes that are completely visited
    - `pathVis[ ]` → marks nodes visited in the **current DFS path**
2. Traverse the graph component-wise using DFS.
3. In DFS for a node:
    - Mark it as visited in both `vis` and `pathVis`.
4. For every adjacent node:
    - **Case 1:** If the node is not visited, call DFS recursively.
    - **Case 2:** If the node is visited **and also marked in `pathVis`**, a cycle exists → return `true`.
    - **Case 3:** If the node is visited but not in `pathVis`, ignore it.
5. After exploring all neighbors:
    - Unmark the node from `pathVis` (backtracking).
6. If no cycle is found in any DFS call, return `false`.

The key idea is:
👉 **Cycle exists only if a node is revisited on the same DFS path**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    bool dfsCheck(int node, vector<int> adj[], int vis[], int
pathVis[]) {
        vis[node] = 1;
        pathVis[node] = 1;

        for(auto it : adj[node]) {
            if(!vis[it]) {
                if(dfsCheck(it, adj, vis, pathVis))
                    return true;
            }
            else if(pathVis[it]) {
                return true;
            }
        }

        pathVis[node] = 0;
        return false;
    }

public:
    bool isCyclic(int V, vector<int> adj[]) {
        int vis[V] = {0};
        int pathVis[V] = {0};

        for(int i = 0; i < V; i++) {
            if(!vis[i]) {
                if(dfsCheck(i, adj, vis, pathVis))
                    return true;
```

```
            }
        }
        return false;
    }
};

int main() {
    vector<int> adj[11] = {
        {}, {2}, {3}, {4,7}, {5}, {6}, {}, {5}, {9}, {10}, {8}
    };
    int V = 11;

    Solution obj;
    bool ans = obj.isCyclic(V, adj);

    if(ans) cout << "True\n";
    else cout << "False\n";

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each node and edge is visited once during DFS.

- **Space Complexity:** `O(V)`
  Reason: Two visited arrays and recursion stack space.

20. Topological Sort Algorithm | DFS : G-21

Given a **Directed Acyclic Graph (DAG)** with `V` vertices numbered from `0` to `V-1`, represented using an **adjacency list**, find **any valid topological ordering** of the graph.

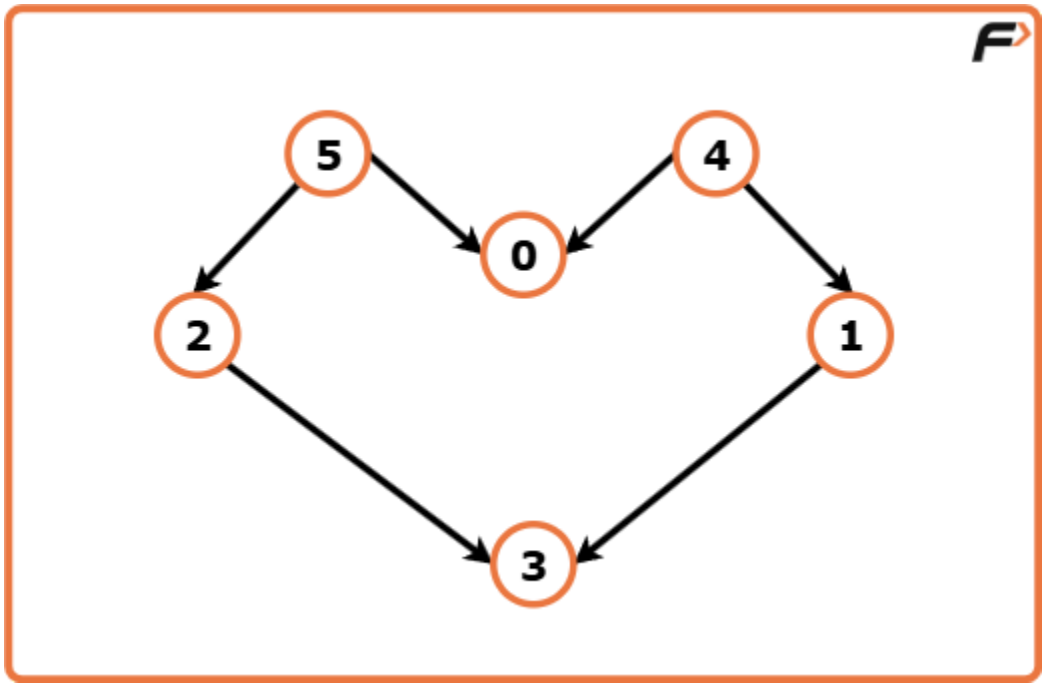
In **topological sorting**, for every directed edge u → v, node u must appear **before** node v in the ordering.

Examples

Example 1

Input:

```
V = 6
adj = [[], [], [3], [1], [0,1], [0,2]]
```



One valid Output:

```
[5, 4, 2, 3, 1, 0]
```

Explanation:
The ordering satisfies all edge conditions:

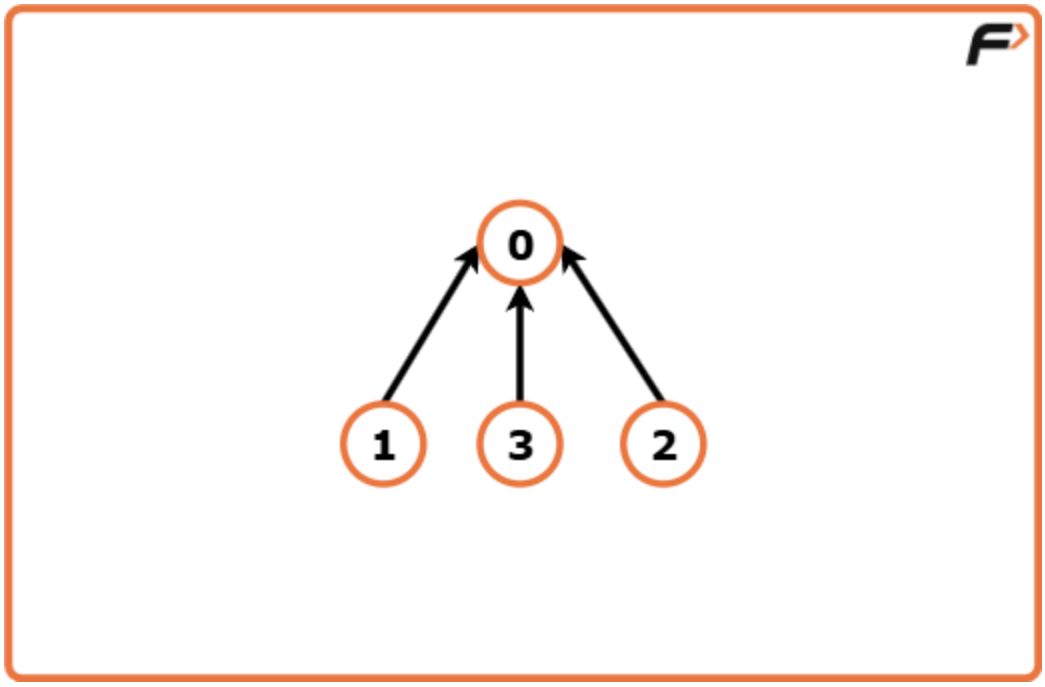
- 5 → 0 → 5 comes before 0
- 4 → 0 → 4 comes before 0
- 5 → 2 → 5 comes before 2
- 2 → 3 → 2 comes before 3
- 3 → 1 → 3 comes before 1
- 4 → 1 → 4 comes before 1

Multiple answers are possible for a DAG.

Example 2

Input:

```
V = 4
adj = [[], [0], [0], [0]]
```



Output:

`[3, 2, 1, 0]`

Explanation:

- `1 → 0, 2 → 0, 3 → 0`
- All nodes `{1,2,3}` must appear before `0`

Approach 1: Using DFS (Depth First Search)

Algorithm

Topological sorting using DFS works on the idea of **finishing times**.

1. Create a `visited` array of size `V`, initialized to `0`.
2. Create a stack to store nodes.

3. For every vertex:
    - If it is not visited, call DFS on it.
4. In DFS:
    - Mark the current node as visited.
    - For every adjacent node:
        - If not visited, call DFS recursively.
    - After visiting all adjacent nodes, push the current node into the stack.
5. After DFS is done for all vertices:
    - Pop elements from the stack to get the topological order.

Pushing a node after exploring all its dependencies ensures correct ordering.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void dfs(int node, vector<int> adj[], vector<int>& vis,
stack<int>& st) {
        vis[node] = 1;
        for(auto it : adj[node]) {
            if(!vis[it]) {
                dfs(it, adj, vis, st);
            }
        }
        st.push(node);
    }

    vector<int> topoSort(int V, vector<int> adj[]) {
```

```
        vector<int> vis(V, 0);
        stack<int> st;

        for(int i = 0; i < V; i++) {
            if(!vis[i]) {
                dfs(i, adj, vis, st);
            }
        }

        vector<int> ans;
        while(!st.empty()) {
            ans.push_back(st.top());
            st.pop();
        }
        return ans;
    }
};

int main() {
    int V = 6;
    vector<int> adj[V];
    adj[5].push_back(0);
    adj[5].push_back(2);
    adj[4].push_back(0);
    adj[4].push_back(1);
    adj[2].push_back(3);
    adj[3].push_back(1);

    Solution obj;
    vector<int> res = obj.topoSort(V, adj);

    for(auto it : res) cout << it << " ";
    return 0;
}
```

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex and each edge is visited exactly once during DFS.

- **Space Complexity:** `O(V + E)`
  Reason: Adjacency list, visited array, recursion stack, and result stack.

Approach 2: Using BFS (Kahn's Algorithm)

Algorithm

This approach is based on **in-degrees** of vertices.

1. Create an `indegree` array of size `V`, initialized to `0`.
2. For every edge `u → v`, increment `indegree[v]`.
3. Push all vertices with `indegree = 0` into a queue.
4. While the queue is not empty:
    - Pop a node and add it to the result.
    - For all its adjacent nodes:
        - Decrease their in-degree by 1.
        - If in-degree becomes `0`, push into the queue.
5. The order in which nodes are removed from the queue gives a topological sort.

This works because nodes with zero in-degree have no dependencies.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> topologicalSort(int V, vector<int> adj[]) {
```

```
        vector<int> indegree(V, 0);

        for(int i = 0; i < V; i++) {
            for(auto it : adj[i]) {
                indegree[it]++;
            }
        }

        queue<int> q;
        for(int i = 0; i < V; i++) {
            if(indegree[i] == 0)
                q.push(i);
        }

        vector<int> topo;
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            topo.push_back(node);

            for(auto it : adj[node]) {
                indegree[it]--;
                if(indegree[it] == 0)
                    q.push(it);
            }
        }
        return topo;
    }
};

int main() {
    int V = 6;
    vector<int> adj[V];
    adj[5].push_back(0);
    adj[5].push_back(2);
    adj[4].push_back(0);
    adj[4].push_back(1);
    adj[2].push_back(3);
```

```
    adj[3].push_back(1);

    Solution obj;
    vector<int> ans = obj.topologicalSort(V, adj);

    for(auto it : ans) cout << it << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(V + E)`
  Reason: Each vertex is processed once and each edge reduces indegree once.

- **Space Complexity:** `O(V + E)`
  Reason: Adjacency list, indegree array, queue, and result vector.

21. Kahn's Algorithm | Topological Sort Algorithm | BFS : G-22

Given a **Directed Acyclic Graph (DAG)** with V vertices and E edges, find **any valid topological sorting** of the graph.

In topological sorting, for every directed edge u → v, node u must always appear **before** node v in the ordering.

Examples

Example 1

Input:

`V = 6`

```
adj = [ [], [], [3], [1], [0,1], [0,2] ]
```

Explanation:

Edges present:

```
5 → 0
5 → 2
4 → 0
4 → 1
2 → 3
3 → 1
```

One valid topological ordering:

```
[5, 4, 2, 3, 1, 0]
```

Another valid ordering:

```
[4, 5, 2, 3, 1, 0]
```

Multiple answers are possible because the graph is a DAG.

Example 2

Input:

```
V = 4
adj = [ [], [0], [0], [0] ]
```

Explanation:

Edges present:

```
1 → 0
2 → 0
3 → 0
```

All nodes `{1,2,3}` must come before `0`.

One valid output:

```
[3, 2, 1, 0]
```

Approach

Algorithm (Kahn's Algorithm – BFS)

Kahn's Algorithm uses **Breadth First Search (BFS)** and **in-degree counting**.

1. Create an array `inDegree[]` to store the number of incoming edges for each node.
2. Traverse the adjacency list and compute in-degrees of all nodes.
3. Push all nodes with `inDegree = 0` into a queue (these nodes have no dependencies).
4. While the queue is not empty:
    - Pop a node from the queue and add it to the answer.
    - For every adjacent node:
        - Decrease its in-degree by 1.
        - If in-degree becomes 0, push it into the queue.
5. The order in which nodes are removed from the queue forms a valid topological sort.

This works because a node can only be processed **after all its parent nodes are processed**.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    vector<int> topoSort(int V, vector<int> adj[]) {
        vector<int> ans;
        vector<int> inDegree(V, 0);

        // Calculate in-degree of each node
        for(int i = 0; i < V; i++) {
            for(auto it : adj[i]) {
                inDegree[it]++;
            }
        }

        // Queue for BFS
        queue<int> q;

        // Push nodes with in-degree 0
        for(int i = 0; i < V; i++) {
            if(inDegree[i] == 0)
                q.push(i);
        }

        // BFS
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            ans.push_back(node);

            for(auto it : adj[node]) {
                inDegree[it]--;
                if(inDegree[it] == 0)
                    q.push(it);
            }
        }

        return ans;
    }
};
```

```
int main() {
    int V = 6;
    vector<int> adj[V] = {
        {},
        {},
        {3},
        {1},
        {0,1},
        {0,2}
    };

    Solution obj;
    vector<int> ans = obj.topoSort(V, adj);

    for(int x : ans)
        cout << x << " ";

    return 0;
}
```

Complexity Analysis

Time Complexity: `O(V + E)`
Reason:

- Each vertex is processed once
- Each edge is processed once while reducing in-degrees

Space Complexity: `O(V)`
Reason:

- In-degree array and queue can store up to `V` nodes

22. Detect a Cycle in a Directed Graph

Given a **directed graph** with `V` vertices labeled from `0` to `V-1`, represented using an **adjacency list**, determine whether the graph contains **any cycle**.

A cycle exists if we can start from a node and come back to the same node by following the **direction of edges**.

Examples

Example 1

Input:

```
V = 6
adj = [[1], [2,5], [3], [4], [1], []]
```

Explanation:

`1 → 2 → 3 → 4 → 1`

We can start from node 1 and return back to 1 following directed edges.
So, a **cycle exists**.

Output:

`True`

Example 2

Input:

```
V = 4
adj = [[1,2], [2], [], [0,2]]
```

Explanation:

- No path forms a closed loop following edge directions.
- Every path ends without coming back to the starting node.

Output:

```
False
```

These examples show that **not all directed graphs have cycles**, and direction of edges matters.

Approach

Algorithm (Using BFS – Kahn's Algorithm)

A cycle in a directed graph can be detected using **Kahn's Algorithm**, which is based on **Topological Sorting**.

Key idea:

- A **Directed Acyclic Graph (DAG)** always has a topological ordering containing **all vertices**.
- If a graph contains a cycle, **some vertices will never get in-degree 0**, so they will never be processed.

Steps:

1. Create an `indegree` array to store the number of incoming edges for each vertex.
2. Traverse the adjacency list and compute indegree of every vertex.

3. Push all vertices with `indegree = 0` into a queue.
4. Initialize a counter `count = 0` to count processed nodes.
5. While the queue is not empty:
    - Pop a node from the queue.
    - Increment `count`.
    - For each neighbor of this node:
        - Decrease its indegree by 1.
        - If indegree becomes 0, push it into the queue.
6. After BFS:
    - If `count == V`, all nodes were processed → **no cycle**.
    - If `count < V`, some nodes were not processed → **cycle exists**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isCyclic(int V, vector<vector<int>>& adj) {
        vector<int> indegree(V, 0);

        // Calculate indegree of each vertex
        for(int i = 0; i < V; i++) {
            for(auto it : adj[i]) {
                indegree[it]++;
            }
        }
```

```
        queue<int> q;

        // Push all vertices with indegree 0
        for(int i = 0; i < V; i++) {
            if(indegree[i] == 0)
                q.push(i);
        }

        int count = 0;

        // BFS using Kahn's Algorithm
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            count++;

            for(auto it : adj[node]) {
                indegree[it]--;
                if(indegree[it] == 0)
                    q.push(it);
            }
        }

        // If all nodes are not processed, cycle exists
        return count != V;
    }
};

int main() {
    int V = 6;
    vector<vector<int>> adj = {
        {1}, {2,5}, {3}, {4}, {1}, {}
    };

    Solution obj;
    if(obj.isCyclic(V, adj))
        cout << "True";
    else
```

```
        cout << "False";

    return 0;
}
```

Complexity Analysis

Time Complexity: `O(V + E)`
Reason:

- Each vertex is processed once.
- Each edge is considered once while updating indegrees.

Space Complexity: `O(V + E)`
Reason:

- Adjacency list stores edges.
- Indegree array and queue take up to `O(V)` space.

23. Course Schedule I and II | Pre-requisite Tasks | Topological Sort : G-24

There are tasks (or courses) labeled from `0` to `N-1`.
Some tasks depend on other tasks and must be completed **after** their prerequisites.

A prerequisite pair `[a, b]` means:

- **Task b must be completed before task a**

These problems are solved using **Topological Sort on a Directed Graph**.

Examples

Example 1 (Possible)

Input:

```
N = 4
P = 3
prerequisites = {{1,0},{2,1},{3,2}}
```

Explanation:

```
0 → 1 → 2 → 3
```

One valid order:

```
3 2 1 0
```

All tasks can be completed.

Output:

```
Yes
```

Example 2 (Not Possible)

Input:

```
N = 4
P = 4
prerequisites = {{1,2},{4,3},{2,4},{4,1}}
```

Explanation:

```
1 → 2 → 4 → 1   (cycle)
```

Because of the cycle, tasks depend on each other circularly.

Output:

No

Course Schedule I

(Check if it is possible to finish all tasks)

Approach 1

Algorithm

The problem reduces to **cycle detection in a directed graph**.

If the graph has a cycle, tasks cannot be finished.
If the graph is a **DAG**, tasks can be finished.

We use **Kahn's Algorithm (BFS Topological Sort)**:

1. Treat each task as a node.
2. Create a directed edge `b → a` for prerequisite `[a, b]`.
3. Compute `inDegree` for every task.
4. Push all tasks with `inDegree = 0` into a queue.
5. Process the queue:
    - Remove a task.
    - Reduce inDegree of its neighbors.
    - Push neighbors whose inDegree becomes 0.
6. Count processed tasks.
7. If processed count == N → possible, else not possible.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites)
{
        vector<vector<int>> adj(numCourses);
        vector<int> inDegree(numCourses, 0);

        for(auto &p : prerequisites) {
            int a = p[0], b = p[1];
            adj[b].push_back(a);
            inDegree[a]++;
        }

        queue<int> q;
        for(int i = 0; i < numCourses; i++) {
            if(inDegree[i] == 0) q.push(i);
        }

        int cnt = 0;
        while(!q.empty()) {
            int node = q.front(); q.pop();
            cnt++;

            for(int nei : adj[node]) {
                inDegree[nei]--;
                if(inDegree[nei] == 0)
                    q.push(nei);
            }
        }

        return cnt == numCourses;
    }
```

```
};

int main() {
    Solution sol;
    vector<vector<int>> prerequisites = {{1,0},{0,1}};
    int numCourses = 2;
    cout << (sol.canFinish(numCourses, prerequisites) ? "true" :
"false");
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(V + E)$
  Each task and dependency is processed once.

- **Space Complexity:** $O(V + E)$
  Adjacency list, inDegree array, queue.

Course Schedule II

(Return one valid order of tasks)

Approach 2

Algorithm

This is an extension of Course Schedule I.

Instead of only checking feasibility, we **store the topological order**.

Steps are the same as Kahn's Algorithm:

1. Build graph and inDegree array.

2. Push all nodes with inDegree = 0 into queue.

3. While queue is not empty:
    - Pop node and add it to order.
    - Reduce inDegree of neighbors.
4. If order size == N, return order.
5. Otherwise, return empty array (cycle exists).

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>&
prerequisites) {
        vector<vector<int>> adj(numCourses);
        vector<int> inDegree(numCourses, 0);

        for(auto &p : prerequisites) {
            int a = p[0], b = p[1];
            adj[b].push_back(a);
            inDegree[a]++;
        }

        queue<int> q;
        for(int i = 0; i < numCourses; i++) {
            if(inDegree[i] == 0) q.push(i);
        }

        vector<int> order;
        while(!q.empty()) {
            int node = q.front(); q.pop();
            order.push_back(node);
```

```
            for(int nei : adj[node]) {
                inDegree[nei]--;
                if(inDegree[nei] == 0)
                    q.push(nei);
            }
        }

        if(order.size() == numCourses)
            return order;
        return {};
    }
};

int main() {
    Solution sol;
    vector<vector<int>> prerequisites = {{1,0},{2,0},{3,1},{3,2}};
    int numCourses = 4;

    vector<int> ans = sol.findOrder(numCourses, prerequisites);
    for(int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(V + E)$
- **Space Complexity:** $O(V + E)$

24. Find Eventual Safe States – BFS – Topological Sort : G-25

Given a **directed graph** with `V` vertices labeled from `0` to `V-1`, represented using an **adjacency list**, find all **eventual safe nodes**.

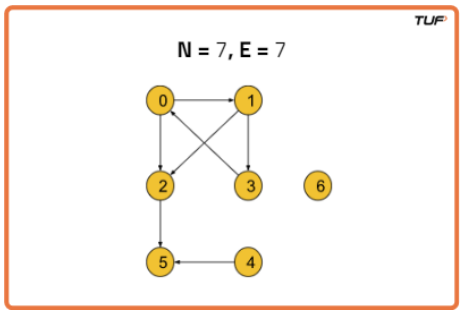
- A **terminal node** is a node with **no outgoing edges**.
- A **safe node** is a node from which **every possible path** eventually leads to a terminal node.
- If a node is part of a **cycle** or can reach a cycle, it is **not safe**.
- Return all safe nodes in **ascending order**.

Examples

Example 1

Input:

```
N = 7
Edges = 7
```



Explanation (conceptual):

- There is a cycle: `0 → 1 → 3 → 0`

- Node 7 leads into this cycle
- Nodes `0, 1, 3, 7` are **not safe**
- Nodes `2, 4, 5, 6` eventually lead to terminal nodes

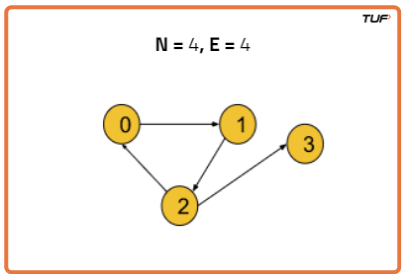
Output:

```
{2, 4, 5, 6}
```

Example 2

Input:

```
N = 4
Edges = 4
```



Explanation:

- Only node 3 has no outgoing edges
- All other nodes either form or lead into a cycle

Output:

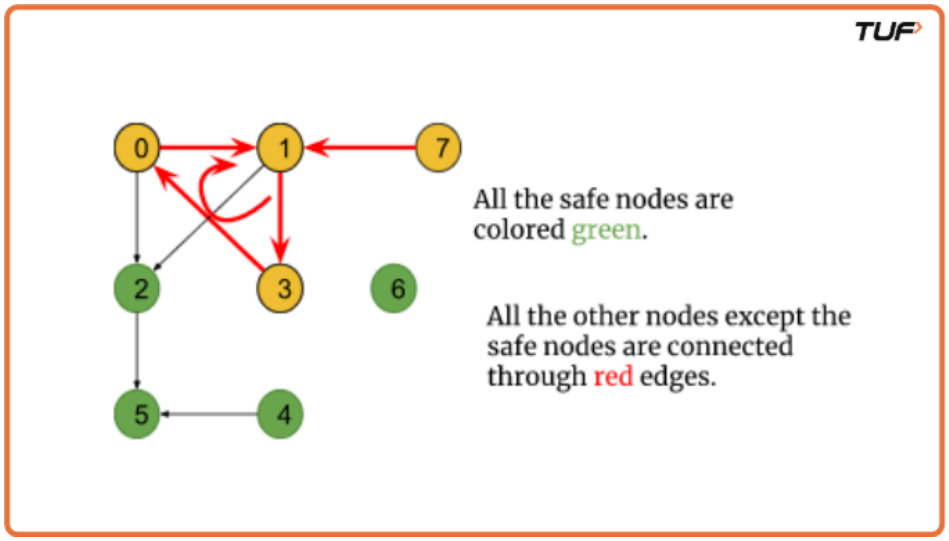
```
{3}
```

These examples show that **nodes involved in cycles or leading to cycles are unsafe**.

Approach

Algorithm (BFS + Topological Sort)

The idea is to **eliminate all nodes that are part of cycles or lead to cycles**.



Key observations:

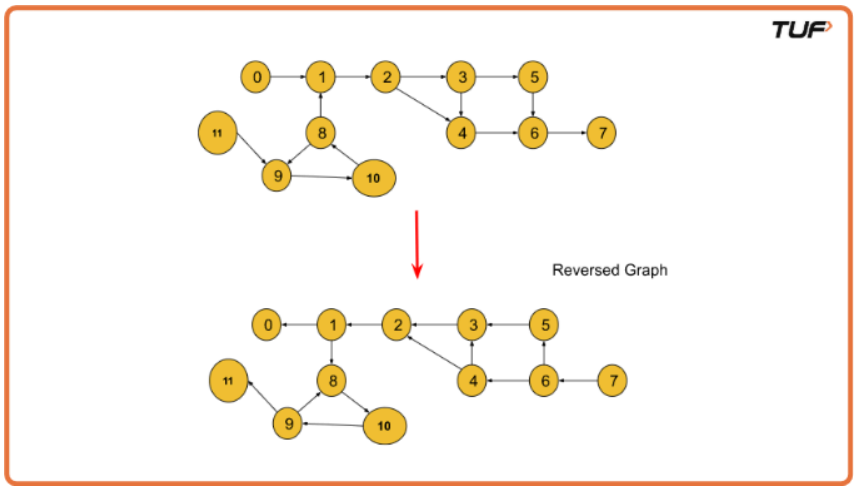
- Terminal nodes (outdegree = 0) are always **safe**.
- If a node points only to safe nodes, it is also safe.
- Nodes in cycles will never become terminal.

To use **topological sort logic**, we do the following:

1. **Reverse the graph**:
    - Original edge: u → v
    - Reversed edge: v → u
2. In the reversed graph:
    - Terminal nodes become nodes with **indegree = 0**.
3. Create an `indegree[]` array based on the reversed graph.

4. Push all nodes with `indegree = 0` into a queue.
5. Perform BFS:
    - Pop a node → it is safe.
    - For each neighbor in the reversed graph:
        - Decrease its indegree.
        - If indegree becomes 0, push it into the queue.
6. All nodes processed by BFS are **safe nodes**.
7. Sort the result as required.

Nodes that are part of cycles will **never reach indegree 0**, so they are excluded automatically.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> eventualSafeNodes(int V, vector<int> adj[]) {
        vector<int> adjRev[V];
```

```
        vector<int> indegree(V, 0);

        // Reverse the graph and compute indegree
        for(int i = 0; i < V; i++) {
            for(auto it : adj[i]) {
                adjRev[it].push_back(i);
                indegree[i]++;
            }
        }

        queue<int> q;
        vector<int> safeNodes;

        // Push all nodes with indegree 0
        for(int i = 0; i < V; i++) {
            if(indegree[i] == 0)
                q.push(i);
        }

        // BFS traversal
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            safeNodes.push_back(node);

            for(auto it : adjRev[node]) {
                indegree[it]--;
                if(indegree[it] == 0)
                    q.push(it);
            }
        }

        sort(safeNodes.begin(), safeNodes.end());
        return safeNodes;
    }
};

int main() {
```

```
    vector<int> adj[12] = {
        {1}, {2}, {3,4}, {4,5}, {6}, {6}, {7}, {},
        {1,9}, {10}, {8}, {9}
    };
    int V = 12;

    Solution obj;
    vector<int> ans = obj.eventualSafeNodes(V, adj);

    for(int x : ans)
        cout << x << " ";
    return 0;
}
```

Time Complexity

`O(V + E) + O(N log N)`

- `O(V + E)` for BFS traversal
- `O(N log N)` for sorting safe nodes

Space Complexity

`O(V + E)`

- Reversed adjacency list
- Indegree array
- Queue for BFS

25. Alien Dictionary – Topological Sort : G-26

You are given a **sorted dictionary of an alien language**.
Your task is to **determine the order of characters** in that alien language.

- The dictionary contains N words.
- Each word is made of characters from an alien language.
- There are exactly K unique characters (assumed to be the first K lowercase letters).
- The dictionary is already sorted according to the alien language rules.

You need to return **one valid ordering of the characters**.

Example 1

Input:

```
N = 5, K = 4
dict = {"baa","abcd","abca","cab","cad"}
```

Explanation (compare consecutive words):

- "baa" vs "abcd" → b comes before a
- "abcd" vs "abca" → d comes before a
- "abca" vs "cab" → a comes before c
- "cab" vs "cad" → b comes before d

So one valid alien character order is:

```
b d a c
```

Example 2

Input:

```
N = 3, K = 3
dict = {"caa","aaa","aab"}
```

Explanation:

- "caa" vs "aaa" → c comes before a
- "aaa" vs "aab" → a comes before b

So one valid order is:

```
c a b
```

These examples show that **character order is derived by comparing adjacent words**.

Approach

Algorithm

This problem can be solved using **Topological Sort (BFS / Kahn's Algorithm)**.

Step 1: Build the Graph

- Treat each character as a node.
- Compare every pair of **adjacent words**.
- For the first position where characters differ:
    - If `s1[i] != s2[i]`, then add a directed edge `s1[i] → s2[i]`

- Break, because only the first mismatch matters.

Step 2: Topological Sort (BFS)

- Create an `indegree` array for all K characters.
- Push all characters with `indegree = 0` into a queue.
- While the queue is not empty:
  - Pop a character and add it to the result.
  - Reduce indegree of its neighbors.
  - If any neighbor's indegree becomes 0, push it into the queue.

Step 3: Final Answer

- Convert numeric nodes back to characters.
- Return the ordering as a string.

This works because the character dependency graph is a **Directed Acyclic Graph (DAG)**.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    vector<int> topoSort(int V, vector<int> adj[]) {
        vector<int> indegree(V, 0);

        for(int i = 0; i < V; i++) {
            for(auto it : adj[i]) {
                indegree[it]++;
            }
```

```
        }

        queue<int> q;
        for(int i = 0; i < V; i++) {
            if(indegree[i] == 0)
                q.push(i);
        }

        vector<int> topo;
        while(!q.empty()) {
            int node = q.front();
            q.pop();
            topo.push_back(node);

            for(auto it : adj[node]) {
                indegree[it]--;
                if(indegree[it] == 0)
                    q.push(it);
            }
        }
        return topo;
    }

public:
    string findOrder(string dict[], int N, int K) {
        vector<int> adj[K];

        for(int i = 0; i < N - 1; i++) {
            string s1 = dict[i];
            string s2 = dict[i + 1];
            int len = min(s1.size(), s2.size());

            for(int j = 0; j < len; j++) {
                if(s1[j] != s2[j]) {
                    adj[s1[j] - 'a'].push_back(s2[j] - 'a');
                    break;
                }
            }
```

```
        }

        vector<int> topo = topoSort(K, adj);

        string ans = "";
        for(auto it : topo) {
            ans += char(it + 'a');
        }
        return ans;
    }
};

int main() {
    int N = 5, K = 4;
    string dict[] = {"baa","abcd","abca","cab","cad"};

    Solution obj;
    string res = obj.findOrder(dict, N, K);

    for(char c : res)
        cout << c << " ";
    return 0;
}
```

Time Complexity

`O(N * L) + O(K + E)`

- `N * L` for comparing adjacent words (up to first mismatch)
- `K + E` for topological sort

Space Complexity

`O(K)`

- Adjacency list
- Indegree array
- Queue
- Result array

26. Shortest Path in Undirected Graph with Unit Distance : G-28

Problem Statement

Given an **undirected graph** with **unit weight edges**, find the **shortest distance from the source node (0)** to **all other nodes**.

- If a node is **unreachable**, return `-1` for that node.
- All edges have weight = 1.

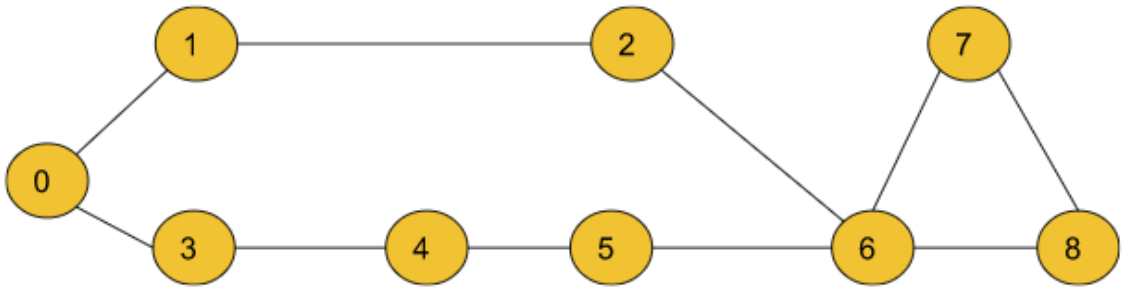
Examples (Important)

Example 1

Input

```
n = 9, m = 10
```

```
edges = [[0,1],[0,3],[3,4],[4,5],[5,6],
         [1,2],[2,6],[6,7],[7,8],[6,8]]
src = 0
```



Output

```
0 1 2 1 2 3 3 4 4
```

Explanation

- Distance from 0 → 0 = 0
- Distance from 0 → 1 = 1
- Distance from 0 → 3 = 1
- Distance from 0 → 2 = 2 (0 → 1 → 2)
- Distance from 0 → 6 = 3 (0 → 1 → 2 → 6)
- Distance from 0 → 8 = 4

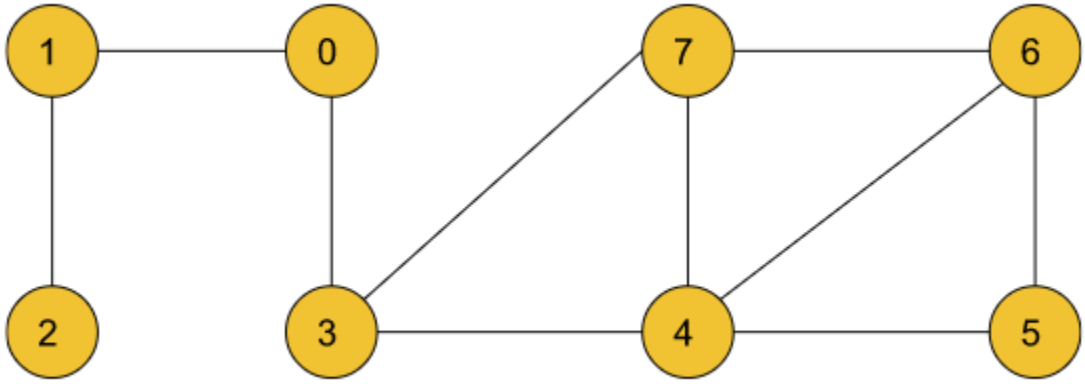
This array represents the **shortest number of edges** needed to reach each node from source `0`.

Example 2

Input

```
n = 8, m = 10
```

```
edges = [[1,0],[2,1],[0,3],[3,7],[3,4],
         [7,4],[7,6],[4,5],[4,6],[6,5]]
src = 0
```



Output

```
0 1 2 1 2 3 3 2
```

Explanation

- Nodes closer to source appear with smaller distances.
- If a node were unreachable, it would be -1.

Approach

Why BFS?

- The graph is **undirected**
- All edges have **unit weight (1)**
- **Breadth First Search (BFS)** always explores nodes level by level
  → guarantees the **shortest path in terms of edges**

So **BFS is optimal** here (no need for Dijkstra).

Algorithm

1. Convert edge list into an **adjacency list**
2. Create a `dist[]` array initialized with a large value (`1e9`)
3. Set `dist[src] = 0`
4. Push `src` into a queue
5. Perform BFS:
    - For each popped node, explore its neighbors
    - If `dist[node] + 1 < dist[neighbor]`
        - Update distance
        - Push neighbor into queue
6. After BFS:
    - If distance is still `1e9`, mark it as `-1`

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> shortestPath(vector<vector<int>>& edges, int N, int M,
int src) {
```

```
        // Adjacency list
        vector<int> adj[N];
        for (auto it : edges) {
            adj[it[0]].push_back(it[1]);
            adj[it[1]].push_back(it[0]);
        }

        // Distance array
        vector<int> dist(N, 1e9);
        dist[src] = 0;

        // BFS queue
        queue<int> q;
        q.push(src);

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            for (auto it : adj[node]) {
                if (dist[node] + 1 < dist[it]) {
                    dist[it] = dist[node] + 1;
                    q.push(it);
                }
            }
        }

        // Convert unreachable nodes to -1
        for (int i = 0; i < N; i++) {
            if (dist[i] == 1e9)
                dist[i] = -1;
        }

        return dist;
    }
};

int main() {
```

```
    int N = 9, M = 10;
    vector<vector<int>> edges = {
        {0,1},{0,3},{3,4},{4,5},{5,6},
        {1,2},{2,6},{6,7},{7,8},{6,8}
    };

    Solution obj;
    vector<int> ans = obj.shortestPath(edges, N, M, 0);

    for (int x : ans)
        cout << x << " ";

    return 0;
}
```

Complexity Analysis

Time Complexity

$O(N + M)$

- BFS visits every node once → $O(N)$
- Traverses every edge once → $O(M)$

Space Complexity

$O(N + M)$

- Adjacency list
- Distance array
- BFS queue

27. Shortest Path in Directed Acyclic Graph (DAG) using Topological Sort : G-27

You are given a **Directed Acyclic Graph (DAG)** with

- **N vertices** numbered from `0` to `N-1`

M directed edges, where each edge is of the form

`u → v with weight w`

-

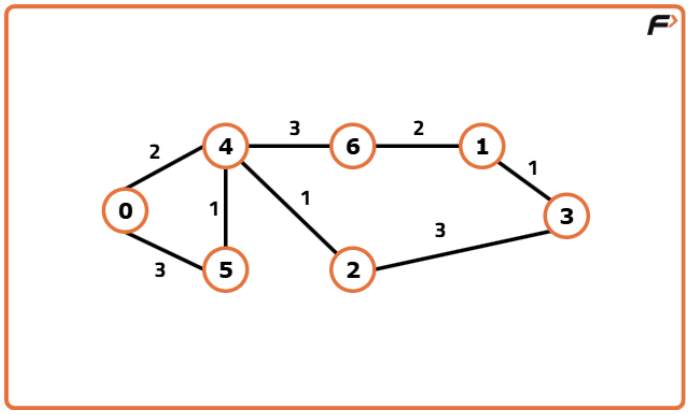
You are required to find the **shortest distance from the source vertex `0`** to **all other vertices**.

- If a vertex is **not reachable** from the source, return `-1` for that vertex.

Example 1

Input

```
n = 7, m = 8
Edges = [
  [0,4,2], [0,5,3], [5,4,1], [4,6,3],
  [4,2,1], [6,1,2], [2,3,3], [1,3,1]
]
```



Output

```
0 7 3 6 2 3 5
```

Explanation

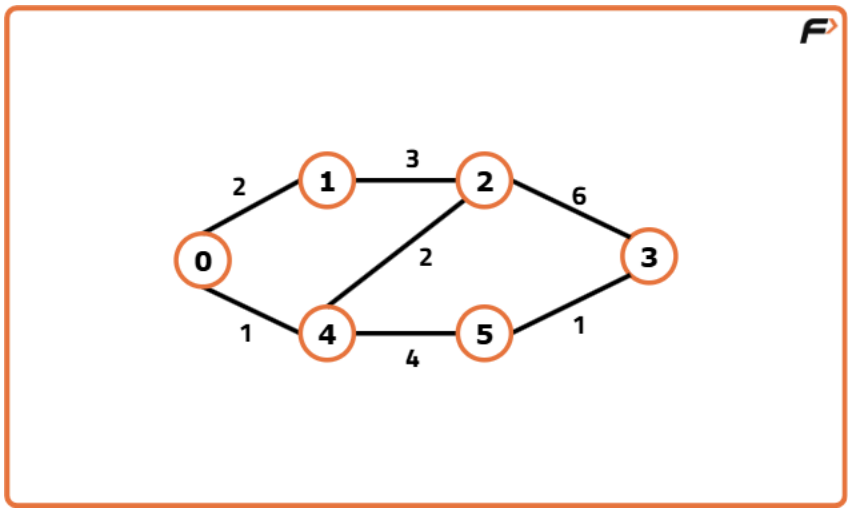
Node	Shortest Distance from 0	Path Example
0	0	source
4	2	0 → 4
5	3	0 → 5
2	3	0 → 4 → 2
6	5	0 → 4 → 6
1	7	0 → 4 → 6 → 1
3	6	0 → 4 → 2 → 3

All distances are computed using **directed edges only**.

Example 2

Input

```
n = 6, m = 7
edges = [
  [0,1,2], [0,4,1], [4,5,4],
  [4,2,2], [1,2,3], [2,3,6], [5,3,1]
]
```



Output

```
0 2 3 6 1 5
```

Explanation

Node	Shortest Distance
0	0
1	2
4	1
2	3
5	5
3	6

Why Topological Sort Works Here

- The graph is a **DAG** → no cycles
- In a DAG, **topological order guarantees** that when we process a node, all nodes that can reach it have already been processed.
- This allows us to **relax edges only once**, unlike Dijkstra.

👉 **Dijkstra is required only when cycles may exist**.

Approach

Key Idea

If we already know the shortest distances to all nodes that come **before** a node in topological order, we can easily compute the shortest distance for the current node.

Algorithm

Build adjacency list
Store graph as:

```
adj[u] = { (v, weight) }
```

1.
2. **Topological Sort using DFS**
    - Maintain a `visited[]` array
    - Push nodes into a stack after DFS completion

Initialize distance array

```
dist[i] = INF
dist[0] = 0
```

3.
4. **Process nodes in topological order**

- Pop nodes from stack

For each edge `u → v (wt)`:

```
if dist[u] + wt < dist[v]
    dist[v] = dist[u] + wt
```

5. **Convert unreachable nodes**
    - Replace `INF` with `-1`

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    // DFS for Topological Sort
    void topoSort(int node, vector<pair<int,int>> adj[],
                  vector<int>& vis, stack<int>& st) {

        vis[node] = 1;

        for (auto it : adj[node]) {
            int v = it.first;
            if (!vis[v]) {
                topoSort(v, adj, vis, st);
            }
        }
        st.push(node);
    }

public:
    vector<int> shortestPath(int N, int M, vector<vector<int>>& edges)
{
```

```
// Adjacency list
vector<pair<int,int>> adj[N];
for (auto &e : edges) {
    adj[e[0]].push_back({e[1], e[2]});
}

// Topological sort
vector<int> vis(N, 0);
stack<int> st;
for (int i = 0; i < N; i++) {
    if (!vis[i]) {
        topoSort(i, adj, vis, st);
    }
}

// Distance array
vector<int> dist(N, 1e9);
dist[0] = 0;

// Relax edges in topo order
while (!st.empty()) {
    int node = st.top();
    st.pop();

    if (dist[node] != 1e9) {
        for (auto it : adj[node]) {
            int v = it.first;
            int wt = it.second;
            if (dist[node] + wt < dist[v]) {
                dist[v] = dist[node] + wt;
            }
        }
    }
}

// Mark unreachable nodes
for (int i = 0; i < N; i++) {
```

```
            if (dist[i] == 1e9)
                dist[i] = -1;
        }

        return dist;
    }
};

int main() {
    int N = 6, M = 7;
    vector<vector<int>> edges = {
        {0,1,2}, {0,4,1}, {4,5,4},
        {4,2,2}, {1,2,3}, {2,3,6}, {5,3,1}
    };

    Solution obj;
    vector<int> ans = obj.shortestPath(N, M, edges);

    for (int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity

$O(N + M)$

- Topological sort → $O(N + M)$
- Edge relaxation → $O(N + M)$

Space Complexity

$O(N + M)$

- Adjacency list
- Stack for topo sort
- Distance & visited arrays

28. Dijkstra's Algorithm (Using Set) : G-33

Problem Statement

You are given a **weighted, undirected, and connected graph** with

- **V vertices**
- An **adjacency list** `adj`, where each entry contains `{neighbor, weight}`
- A **source vertex S**

Your task is to find the **shortest distance from the source vertex S to every other vertex**.

If a vertex is unreachable (not possible here since graph is connected), its distance would remain infinity.

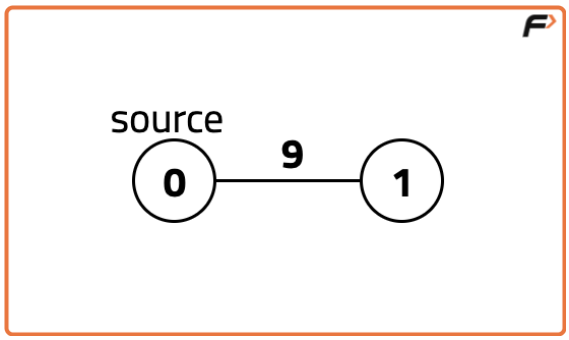
Note:

- The graph **does not contain negative weight edges**
- Dijkstra's Algorithm **fails** if negative weights exist

Example 1

Input

```
V = 2
adj = [
  {{1, 9}},
  {{0, 9}}
]
S = 0
```



Output

```
0 9
```

Explanation

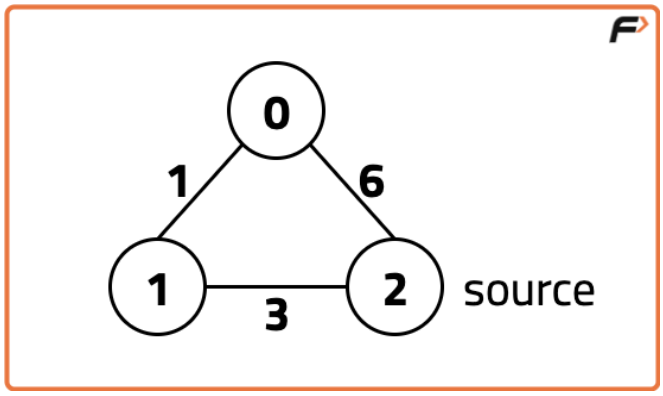
- Distance from node `0` to itself = `0`
- Distance from node `0` to node 1 = 9

Example 2

Input

```
V = 3
adj = {
  {{1,1}, {2,6}},
```

```
  {{2,3}, {0,1}},
  {{1,3}, {0,6}}
}
S = 2
```



Output

```
4 3 0
```

Explanation

Shortest paths from source 2:

Destination	Path	Distance
2	2	0
1	2 → 1	3
0	2 → 1 → 0	4

Direct path 2 → 0 has weight 6, but 2 → 1 → 0 = 3 + 1 = 4, which is shorter.

Intuition

Dijkstra's Algorithm is a **greedy algorithm**:

- Always pick the node with the **smallest known distance**

- Try to improve (relax) distances of its neighbors
- Once a node is picked with minimum distance, that distance is **final**

Using a **set**:

- Keeps `{distance, node}` pairs in **sorted order**
- Allows **deleting outdated entries**, unlike priority queue

---

Approach

Key Data Structures

- **Distance Array**
  `dist[i]` → shortest distance from source to node `i`
- **Set `{distance, node}`**
  Always gives the node with the smallest distance

---

1. Initialize:
    - `dist[i] = INF` for all nodes
    - `dist[S] = 0`
    - Insert `{0, S}` into the set
2. While the set is not empty:
    - Extract the node with **minimum distance**
    - For each adjacent node:
        - If `currentDist + edgeWeight < dist[adjNode]`

- Remove old {dist[adjNode], adjNode} from set (if exists)
- Update dist[adjNode]
- Insert updated pair into set

3. Return the distance array

C++ Code (Using Set)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<int>> adj[], int S) {

        // Set stores {distance, node}
        set<pair<int,int>> st;

        // Distance array
        vector<int> dist(V, 1e9);

        // Source initialization
        dist[S] = 0;
        st.insert({0, S});

        while (!st.empty()) {
            auto it = *st.begin();
            st.erase(it);

            int node = it.second;
            int currDist = it.first;

            // Traverse neighbors
            for (auto nbr : adj[node]) {
                int adjNode = nbr[0];
```

```
                int wt = nbr[1];

                if (currDist + wt < dist[adjNode]) {

                    // Remove old entry if exists
                    if (dist[adjNode] != 1e9)
                        st.erase({dist[adjNode], adjNode});

                    // Update distance
                    dist[adjNode] = currDist + wt;
                    st.insert({dist[adjNode], adjNode});
                }
            }
        }

        return dist;
    }
};

int main() {
    int V = 3, S = 2;
    vector<vector<int>> adj[V];

    adj[0].push_back({1,1});
    adj[0].push_back({2,6});
    adj[1].push_back({2,3});
    adj[1].push_back({0,1});
    adj[2].push_back({1,3});
    adj[2].push_back({0,6});

    Solution obj;
    vector<int> ans = obj.dijkstra(V, adj, S);

    for (int x : ans) cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity

`O(E log V)`

- Each edge may cause insertion/deletion in set → `log V`

Space Complexity

`O(V + E)`

- Distance array + adjacency list + set

29. Dijkstra's Algorithm – Using Priority Queue : G-32

Problem Statement

You are given a **weighted, undirected, and connected graph** with

- **V vertices**
- **E edges**
- A **source vertex S**

Your task is to find the **shortest distance from the source vertex S to all other vertices**.

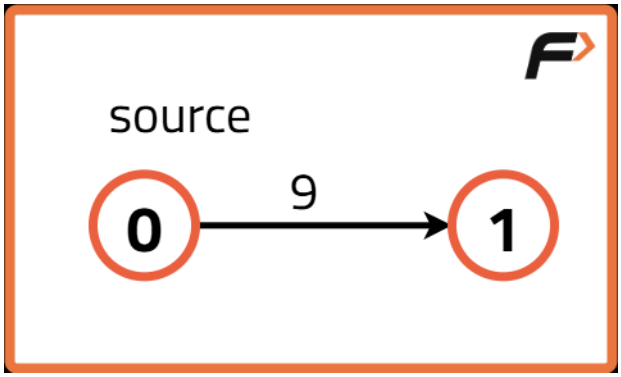
Note:

- The graph does **not** contain negative weight edges or cycles
- If a node is unreachable, its distance would be `-1` (not applicable here since graph is connected)

Example 1

Input

```
V = 2, E = 1
Edges: 0 --9--> 1
S = 0
```



Output

```
[0, 9]
```

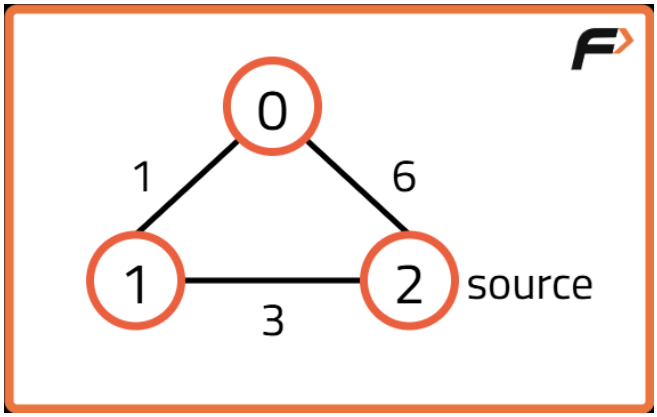
Explanation

- Distance from source `0` to itself = `0`
- Distance from `0` to `1` = `9`

Example 2

Input

```
V = 3, E = 3
Edges:
2 --1--> 1
1 --3--> 0
2 --6--> 0
S = 2
```



Output

```
[4, 3, 0]
```

Explanation

Shortest paths from source 2:

Node	Path	Distance
2	2	0
1	2 → 1	1
0	2 → 1 → 0	1 + 3 = 4

Direct path 2 → 0 has weight 6, which is **not optimal**.

Intuition

Dijkstra's Algorithm is a **greedy algorithm**:

- Always expand the **closest unprocessed node**
- Once a node is popped with minimum distance, that distance is **final**
- Works only when all edge weights are **non-negative**

Using a **Priority Queue (Min-Heap)** helps us:

- Efficiently extract the node with the **minimum distance**
- Avoid scanning all vertices repeatedly

Algorithm

1. Create a `dist[]` array initialized with a very large value (`INF`)
2. Set `dist[source] = 0`
3. Use a **min-heap priority queue** storing `{distance, node}`
4. Push `{0, source}` into the queue
5. While the queue is not empty:
    - Pop the node with the smallest distance
    - If this distance is **greater than the stored distance**, skip it
    - For each adjacent node:
        - If `dist[curr] + weight < dist[adj]`
            - Update `dist[adj]`
            - Push `{dist[adj], adj}` into the queue
6. After completion, `dist[]` contains shortest distances from the source

C++ Code (Priority Queue – Min Heap)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<pair<int,int>>>& adj,
int src) {

        // Distance array
        vector<int> dist(V, 1e9);

        // Min-heap {distance, node}
        priority_queue<
            pair<int,int>,
            vector<pair<int,int>>,
            greater<pair<int,int>>
        > pq;

        // Initialize source
        dist[src] = 0;
        pq.push({0, src});

        while (!pq.empty()) {
            auto top = pq.top();
            pq.pop();

            int node = top.second;
            int currDist = top.first;

            // Skip outdated entry
            if (currDist > dist[node]) continue;

            // Relax neighbors
            for (auto it : adj[node]) {
                int next = it.first;
```

```
                int wt = it.second;

                if (dist[node] + wt < dist[next]) {
                    dist[next] = dist[node] + wt;
                    pq.push({dist[next], next});
                }
            }
        }

        return dist;
    }
};

int main() {
    int V = 5;
    vector<vector<pair<int,int>>> adj(V);

    adj[0].push_back({1, 2});
    adj[0].push_back({2, 4});
    adj[1].push_back({2, 1});
    adj[1].push_back({3, 7});
    adj[2].push_back({4, 3});
    adj[3].push_back({4, 2});

    Solution obj;
    vector<int> dist = obj.dijkstra(V, adj, 0);

    for (int i = 0; i < V; i++) {
        cout << dist[i] << " ";
    }
}
```

Complexity Analysis

Time Complexity

`O((V + E) log V)`

- Each edge relaxation may push into heap → `log V`

Space Complexity

`O(V + E)`

- Distance array + adjacency list + priority queue

30. G-36: Shortest Distance in a Binary Maze

Given an n × m binary grid where each cell is either 0 or 1. You are given a source cell and a destination cell. You can move only through cells having value 1. Movement is allowed only in four directions: up, down, left, and right.
Your task is to find the shortest distance from the source to the destination. If no path exists, return -1.

If the source and destination are the same cell, the distance is 0.

Example:
Grid
1 1 1 1
1 1 0 1
1 1 1 1
1 1 0 0
1 0 0 1

Source = (0,1), Destination = (2,2)
Shortest path is (0,1) → (1,1) → (2,1) → (2,2)
Total steps = 3

Approach

Algorithm

We use Breadth First Search because each move has equal cost and BFS always gives the shortest path.

1. If source and destination are the same, return 0.
2. Create a queue that stores `{distance, {row, col}}`.
3. Create a distance matrix and initialize all values to a very large number.
4. Set the source distance as 0 and push it into the queue.
5. Define direction arrays for up, right, down, and left.
6. While the queue is not empty:
    - Pop the front element.
    - For each of the 4 directions:
        - Check boundary conditions.
        - Check if the cell value is 1.
        - Check if the new distance is smaller than the stored distance.
        - Update distance and push the cell into the queue.
        - If destination is reached, return the distance.
7. If BFS ends and destination is not reached, return -1.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {
```

```
public:

    int shortestPath(vector<vector<int>> &grid, pair<int,int> source,
pair<int,int> destination) {

        if(source.first==destination.first &&
source.second==destination.second)

            return 0;


        int n=grid.size();

        int m=grid[0].size();


        vector<vector<int>> dist(n,vector<int>(m,1e9));

        queue<pair<int,pair<int,int>>> q;


        dist[source.first][source.second]=0;

        q.push({0,{source.first,source.second}});


        int dr[]={-1,0,1,0};

        int dc[]={0,1,0,-1};


        while(!q.empty()){

            auto it=q.front();

            q.pop();

            int dis=it.first;
```

```
            int r=it.second.first;
            int c=it.second.second;

            for(int i=0;i<4;i++){
                int newr=r+dr[i];
                int newc=c+dc[i];

                if(newr>=0 && newr<n && newc>=0 && newc<m &&
                   grid[newr][newc]==1 && dis+1<dist[newr][newc]){
                    dist[newr][newc]=dis+1;

                    if(newr==destination.first &&
newc==destination.second)
                        return dis+1;

                    q.push({dis+1,{newr,newc}});
                }
            }
        }
        return -1;
    }
};
```

```
int main(){

    vector<vector<int>> grid={{1,1,1,1},

                              {1,1,0,1},

                              {1,1,1,1},

                              {1,1,0,0},

                              {1,0,0,1}};

    pair<int,int> source={0,1};

    pair<int,int> destination={2,2};


    Solution obj;

    cout<<obj.shortestPath(grid,source,destination);

    return 0;

}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each cell is visited at most once, and for every cell we check 4 directions.

Space Complexity: **O(N × M)**
Distance matrix and queue together take space proportional to the number of cells.

31. G-37: Path With Minimum Effort

You are given a 2D grid `heights` where each cell represents the height at that position. You start from the top-left cell `(0,0)` and want to reach the bottom-right cell `(n-1,m-1)`.
You can move only in four directions: up, down, left, and right.

The effort of a path is defined as the **maximum absolute difference in heights between any two consecutive cells** in that path.
Your goal is to find a path such that this effort is minimum.

Example:
For heights =
1 2 2
3 8 2
5 3 5

One path is 1 → 3 → 5 → 3 → 5.
The maximum height difference here is 2, which is smaller than other possible paths.
So the answer is 2.

Approach

Algorithm

This problem is solved using Dijkstra's algorithm because we want to minimize the maximum edge cost along a path.

1. Use a priority queue that stores `{effort, {row, col}}`.
2. Create a distance matrix and initialize all values with a large number.
3. Set distance of source cell `(0,0)` as 0 and push it into the priority queue.
4. While the priority queue is not empty:
    - Pop the cell with the minimum effort.
    - If the destination cell is reached, return the effort.
    - For all 4 directions:

- Check if the new cell is inside the grid.
- Compute new effort as
  `max(current effort, absolute height difference)`
- If this effort is smaller than the stored value, update it and push into the queue.

5. If destination is not reached, return 0.

This works because the priority queue always processes the path with the least current effort first.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int MinimumEffort(vector<vector<int>> &heights) {
        int n = heights.size();
        int m = heights[0].size();

        priority_queue<pair<int,pair<int,int>>,
                       vector<pair<int,pair<int,int>>>,
                       greater<pair<int,pair<int,int>>>> pq;

        vector<vector<int>> dist(n,vector<int>(m,1e9));
        dist[0][0]=0;
        pq.push({0,{0,0}});

        int dr[]={-1,0,1,0};
        int dc[]={0,1,0,-1};

        while(!pq.empty()){
            auto it=pq.top();
            pq.pop();
            int diff=it.first;
            int r=it.second.first;
```

```
            int c=it.second.second;

            if(r==n-1 && c==m-1)
                return diff;

            for(int i=0;i<4;i++){
                int newr=r+dr[i];
                int newc=c+dc[i];

                if(newr>=0 && newr<n && newc>=0 && newc<m){
                    int
newEffort=max(abs(heights[r][c]-heights[newr][newc]),diff);
                    if(newEffort<dist[newr][newc]){
                        dist[newr][newc]=newEffort;
                        pq.push({newEffort,{newr,newc}});
                    }
                }
            }
        }
        return 0;
    }
};

int main(){
    vector<vector<int>> heights={{1,2,2},{3,8,2},{5,3,5}};
    Solution obj;
    cout<<obj.MinimumEffort(heights);
    return 0;
}
```

Complexity Analysis

Time Complexity: `O(N * M * log(N * M))`
Each cell can be inserted into the priority queue, and each insertion or deletion takes `log(N*M)` time.

Space Complexity: `O(N * M)`
Used for the distance matrix and the priority queue in the worst case.

32. G-38: Cheapest Flights Within K Stops

You are given n cities connected by flights. Each flight is represented as `[from, to, price]`.
You are also given a source city `src`, a destination city `dst`, and an integer k which represents the maximum number of stops allowed.

You need to find the **minimum cost** to travel from `src` to `dst` using **at most k stops**.
If no such route exists, return `-1`.

Example:
Cities = 4
Flights = [0→1 (100), 1→3 (600), 1→2 (100), 2→3 (200)]
src = 0, dst = 3, k = 1

Path 0→1→3 has cost 700 and uses 1 stop, so it is valid.
Path 0→1→2→3 is cheaper but uses 2 stops, so it is invalid.
Answer = 700.

Approach

Algorithm

This approach uses **BFS with stop control**.

1. Create an adjacency list from the flights array.
2. Use a queue that stores `{stops, {node, cost}}`.
3. Create a distance array initialized with a very large value.
4. Push the source node with 0 stops and 0 cost into the queue.
5. While the queue is not empty:
    - Pop the front element.
    - If the number of stops is greater than k, skip it.

- Traverse all adjacent nodes.
- If a cheaper cost is found and stops are within limit:
    - Update the distance.
    - Push the adjacent node with incremented stop count.

6. After traversal:
    - If destination cost is still infinite, return -1.
    - Otherwise return the stored minimum cost.

This works because BFS ensures we explore paths level by level based on stops.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int CheapestFLight(int n, vector<vector<int>> &flights,
                       int src, int dst, int K) {

        vector<pair<int,int>> adj[n];
        for(auto it: flights){
            adj[it[0]].push_back({it[1], it[2]});
        }

        queue<pair<int,pair<int,int>>> q;
        q.push({0,{src,0}});

        vector<int> dist(n,1e9);
        dist[src]=0;

        while(!q.empty()){
            auto it=q.front();
            q.pop();
            int stops=it.first;
```

```
            int node=it.second.first;
            int cost=it.second.second;

            if(stops>K) continue;

            for(auto iter: adj[node]){
                int adjNode=iter.first;
                int edW=iter.second;

                if(cost+edW<dist[adjNode] && stops<=K){
                    dist[adjNode]=cost+edW;
                    q.push({stops+1,{adjNode,cost+edW}});
                }
            }
        }

        if(dist[dst]==1e9) return -1;
        return dist[dst];
    }
};

int main(){
    int n=4, src=0, dst=3, K=1;
    vector<vector<int>>
flights={{0,1,100},{1,2,100},{2,0,100},{1,3,600},{2,3,200}};
    Solution obj;
    cout<<obj.CheapestFLight(n,flights,src,dst,K);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(E)$
Each flight edge is processed at most once during BFS traversal.

Space Complexity: $O(E + V)$
Adjacency list stores all flights, and the distance array stores minimum cost for each city.

33. Network Delay Time

You are given a directed weighted graph with n nodes numbered from 1 to n. Each edge `(u, v, w)` means a signal can go from node u to node v in w units of time.
A signal starts from node k at time `0`. Whenever a node receives the signal, it immediately forwards it to all its outgoing neighbors.

You need to find the **minimum time** required for **all nodes** to receive the signal.
If any node cannot be reached from k, return `-1`.

Example:
times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
From node 2:

- Node 1 receives signal in 1 unit
- Node 4 receives signal via 2 → 3 → 4 in 2 units
  Answer = 2

Approach

Algorithm

This problem is a shortest path problem on a directed graph with non-negative weights, so we use **Dijkstra's Algorithm**.

1. Create an adjacency list where each node stores its outgoing edges and weights.
2. Create a distance array initialized to infinity for all nodes.
3. Set distance of source node k to `0`.
4. Use a min-heap (priority queue) storing `{time, node}`.
5. Push `{0, k}` into the priority queue.
6. While the priority queue is not empty:
    - Pop the node with the smallest current time.

- For each neighbor:
    - If going through the current node gives a smaller time, update the distance and push it into the queue.

7. After processing all reachable nodes:
    - Find the maximum value in the distance array.
    - If any node is still unreachable (infinity), return `-1`.
    - Otherwise, return the maximum time.

Dijkstra works here because all edge weights are non-negative and we want minimum arrival times.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int,int>>> adj(n+1);
        for(auto &t: times){
            adj[t[0]].push_back({t[1], t[2]});
        }

        priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<>> pq;
        vector<int> dist(n+1, INT_MAX);

        dist[k]=0;
        pq.push({0,k});

        while(!pq.empty()){
            auto it=pq.top();
            pq.pop();
            int time=it.first;
            int node=it.second;
```

```
            for(auto &nbr: adj[node]){
                int v=nbr.first;
                int wt=nbr.second;
                if(dist[v]>time+wt){
                    dist[v]=time+wt;
                    pq.push({dist[v],v});
                }
            }
        }

        int ans=*max_element(dist.begin()+1, dist.end());
        if(ans==INT_MAX) return -1;
        return ans;
    }
};

int main(){
    Solution sol;
    vector<vector<int>> times={{2,1,1},{2,3,1},{3,4,1}};
    int n=4, k=2;
    cout<<sol.networkDelayTime(times,n,k);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O((V + E) \log V)$
Each edge relaxation and priority queue operation takes $\log V$ time.

Space Complexity: $O(V + E)$
Adjacency list stores all edges, and the distance array plus priority queue store up to V nodes.

34. G-40: Number of Ways to Arrive at Destination

You are given a city with n intersections numbered from `0` to `n-1`. The intersections are connected by **bi-directional roads**, and each road has a travel time.
You start at intersection `0` and want to reach intersection `n-1`.

Your task is to find **how many different ways** you can reach the destination **using the shortest possible time**.
Since the number of ways can be large, return the answer modulo `10^9 + 7`.

Example:
If the shortest time to reach the destination is 7 minutes, and there are 4 different paths that take exactly 7 minutes, then the answer is 4.

Approach

Algorithm

This problem is solved using **Dijkstra's Algorithm with path counting**.

1. Create an adjacency list to store roads between intersections.
2. Use a priority queue (min-heap) that stores `{distance, node}`.
3. Create a distance array `dist[]` initialized with infinity.
4. Create a `ways[]` array to store the number of shortest paths to each node.
5. Set `dist[0] = 0` and `ways[0] = 1` because there is exactly one way to start.
6. Push `{0, 0}` into the priority queue.
7. While the priority queue is not empty:
    - Pop the node with the smallest distance.
    - For every adjacent node:

- If a **shorter distance** is found:
  - Update the distance.
  - Copy the number of ways from the current node.
- If the **same shortest distance** is found:
  - Add the number of ways from the current node.

8. At the end, return `ways[n-1] % (10^9 + 7)`.

This works because Dijkstra ensures we process nodes in increasing order of shortest distance, and we count all paths that achieve that same minimum distance.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countPaths(int n, vector<vector<int>> &roads) {

        vector<pair<int,int>> adj[n];
        for(auto it: roads){
            adj[it[0]].push_back({it[1], it[2]});
            adj[it[1]].push_back({it[0], it[2]});
        }

        priority_queue<pair<long long,int>,
                       vector<pair<long long,int>>,
                       greater<pair<long long,int>>> pq;

        vector<long long> dist(n, LLONG_MAX);
        vector<int> ways(n, 0);

        dist[0]=0;
        ways[0]=1;
```

```
        pq.push({0,0});

        int mod=1e9+7;

        while(!pq.empty()){
            auto it=pq.top();
            pq.pop();
            long long dis=it.first;
            int node=it.second;

            for(auto nbr: adj[node]){
                int adjNode=nbr.first;
                int edW=nbr.second;

                if(dis+edW<dist[adjNode]){
                    dist[adjNode]=dis+edW;
                    ways[adjNode]=ways[node];
                    pq.push({dist[adjNode],adjNode});
                }
                else if(dis+edW==dist[adjNode]){
                    ways[adjNode]=(ways[adjNode]+ways[node])%mod;
                }
            }
        }
        return ways[n-1]%mod;
    }
};

int main(){
    int n=7;
    vector<vector<int>>
roads={{0,6,7},{0,1,2},{1,2,3},{1,3,3},{6,3,3},

{3,5,1},{6,5,1},{2,5,1},{0,4,5},{4,6,2}};
    Solution obj;
    cout<<obj.countPaths(n,roads);
    return 0;
}
```

Complexity Analysis

Time Complexity: `O(E log V)`
Dijkstra's algorithm processes each edge with priority queue operations.

Space Complexity: `O(V + E)`
Used for adjacency list, distance array, ways array, and priority queue.

35. G-39: Minimum Multiplications to Reach End

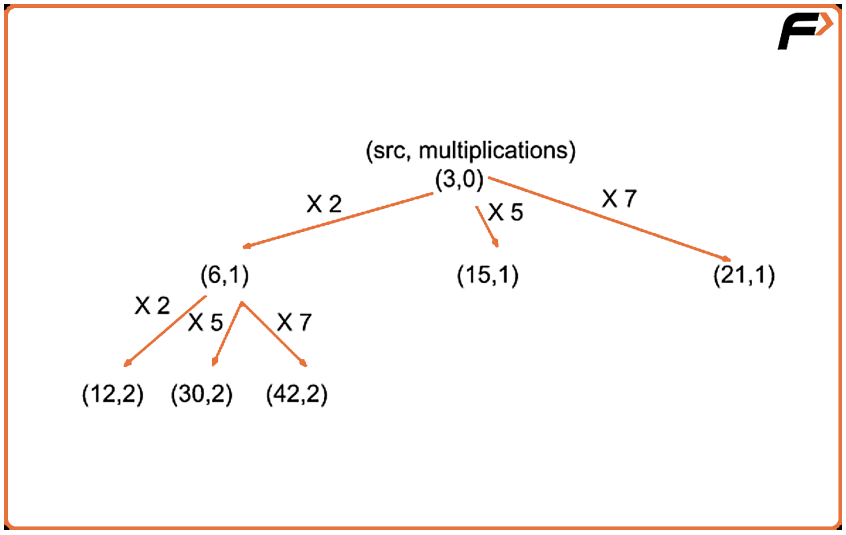
You are given a starting number `start`, a target number `end`, and an array `arr` of numbers.
At each step, you can multiply the current number with **any element of the array**, and then take modulo `100000`.

Your task is to find the **minimum number of multiplications** needed to reach `end` starting from `start`.
If it is not possible to reach `end`, return `-1`.

Example:
arr = {2,5,7}, start = 3, end = 30
3 → 6 → 30
Minimum steps = 2

Approach

Algorithm



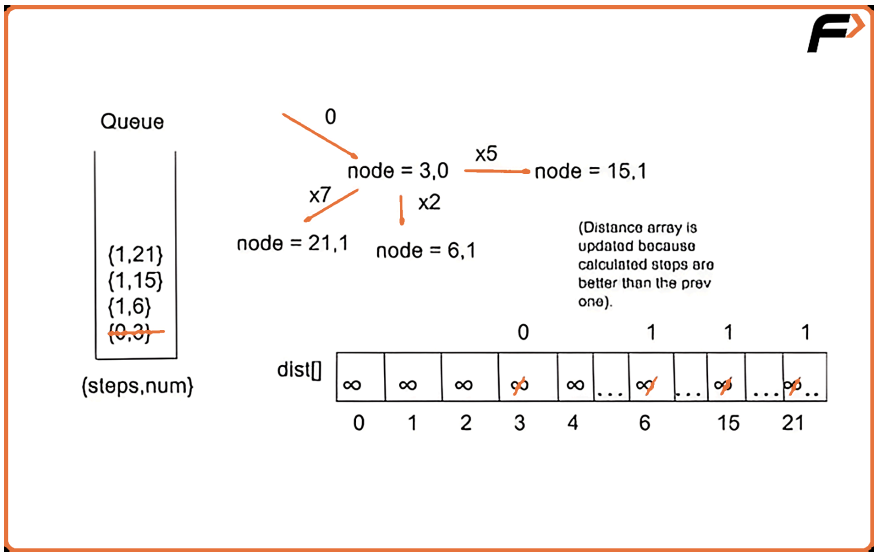
This problem can be treated as a **shortest path problem**.

- Each number (0 to 99999) is considered as a node.
- From a number x, you can go to `(x * arr[i]) % 100000` in one step.
- Since every operation has equal cost (1 step), **BFS** is used.

Steps:

1. Create a queue that stores `{number, steps}`.
2. Create a distance array `dist` of size `100000`, initialized with a very large value.
3. Set `dist[start] = 0` and push `{start, 0}` into the queue.
4. While the queue is not empty:
    - Pop the front element.
    - For each value in `arr`:
        - Compute `newNum = (currentNum * arr[i]) % 100000`.
        - If `steps + 1 < dist[newNum]`, update it and push into the queue.
        - If `newNum == end`, return `steps + 1`.

5. If BFS finishes and end is not reached, return -1.



This guarantees minimum steps because BFS explores level by level.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minimumMultiplications(vector<int> &arr, int start, int end) {

        queue<pair<int,int>> q;
        q.push({start,0});

        vector<int> dist(100000,1e9);
        dist[start]=0;

        int mod=100000;

        while(!q.empty()){
            int node=q.front().first;
            int steps=q.front().second;
```

```
            q.pop();

            for(auto it: arr){
                int num=(node*it)%mod;

                if(steps+1<dist[num]){
                    dist[num]=steps+1;

                    if(num==end)
                        return steps+1;

                    q.push({num,steps+1});
                }
            }
        }
        return -1;
    }
};

int main(){
    vector<int> arr={2,5,7};
    int start=3, end=30;

    Solution obj;
    cout<<obj.minimumMultiplications(arr,start,end);
    return 0;
}
```

Complexity Analysis

Time Complexity: `O(100000 * N)`
Each number from `0` to `99999` can be visited, and for each we try all `N` multipliers.

Space Complexity: `O(100000)`
Distance array stores the minimum steps for each possible number, and the queue can hold at most these many states.

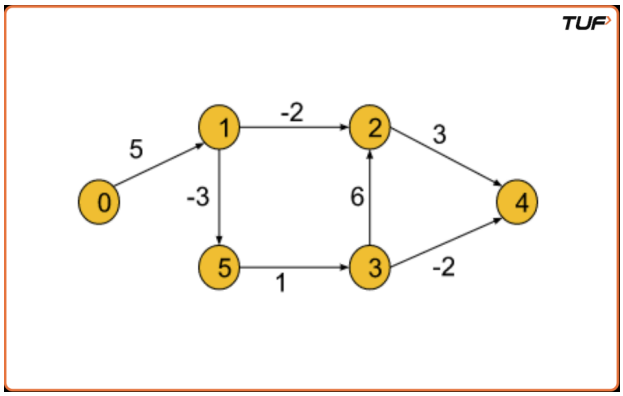
36. Bellman Ford Algorithm: G-41

You are given a **directed weighted graph** with V vertices and a list of edges. Each edge is of the form `[u, v, w]`, meaning there is a directed edge from u to v with weight w.
You are also given a source vertex S.

Your task is to find the **shortest distance from the source S to all other vertices**.

If the graph contains a **negative weight cycle**, return an array containing only `-1`.

Example:
V = 6
Edges =
[3,2,6], [5,3,1], [0,1,5], [1,5,-3], [1,2,-2], [3,4,-2], [2,4,3]
S = 0



Output:
0 5 3 3 1 2

This means the shortest distances from node 0 to all other nodes are as shown.

Approach

Algorithm

Bellman-Ford algorithm is used to find shortest paths from a single source even when **negative edge weights** are present.

1. Create a distance array `dist` of size V.

2. Initialize all distances as infinity except the source S, which is set to 0.

3. Repeat the following process `V-1` times:

    - For every edge `(u, v, wt)`:

        - If `dist[u]` is not infinity and `dist[u] + wt < dist[v]`, update `dist[v]`.

4. After `V-1` relaxations, perform one more iteration over all edges:

    - If any distance can still be reduced, a **negative cycle exists**.

    - In that case, return `{-1}`.

5. If no negative cycle is found, return the distance array.

Why V-1 times?
The shortest path between two vertices can have at most `V-1` edges. More relaxations are only needed to detect negative cycles.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> bellman_ford(int V, vector<vector<int>>& edges, int S)
{

        vector<int> dist(V, 1e8);
        dist[S] = 0;

        // Relax all edges V-1 times
        for(int i = 0; i < V - 1; i++){
            for(auto it : edges){
                int u = it[0];
                int v = it[1];
```

```
                int wt = it[2];

                if(dist[u] != 1e8 && dist[u] + wt < dist[v]){
                    dist[v] = dist[u] + wt;
                }
            }
        }

        // Check for negative cycle
        for(auto it : edges){
            int u = it[0];
            int v = it[1];
            int wt = it[2];

            if(dist[u] != 1e8 && dist[u] + wt < dist[v]){
                return {-1};
            }
        }

        return dist;
    }
};

int main() {
    int V = 6;
    vector<vector<int>> edges = {
        {3,2,6},{5,3,1},{0,1,5},
        {1,5,-3},{1,2,-2},{3,4,-2},{2,4,3}
    };

    int S = 0;
    Solution obj;
    vector<int> ans = obj.bellman_ford(V, edges, S);

    for(int x : ans){
        cout << x << " ";
    }
    return 0;
```

```
}
```

Complexity Analysis

Time Complexity: `O(V * E)`
Each edge is relaxed `V-1` times, plus one extra pass for cycle detection.

Space Complexity: `O(V)`
Only the distance array of size `V` is used.

37. Floyd Warshall Algorithm: G-42

You are given a directed weighted graph with `V` vertices numbered from `0` to `V-1`.
The graph is represented using an adjacency matrix `matrix`, where `matrix[i][j]` is the weight of the edge from vertex `i` to vertex `j`.
If `matrix[i][j] = -1`, it means there is no direct edge between `i` and `j`.

Your task is to find the **shortest distance between every pair of vertices**.
If a vertex is unreachable from another vertex, the value should remain `-1`.

Example:
Input matrix:
0 2 -1 -1
1 0 3 -1
-1 -1 0 1
3 5 4 0

After applying Floyd Warshall, the shortest distance from vertex 0 to 2 becomes 5, and from 1 to 3 becomes 4.

Approach

Algorithm

Floyd Warshall is an **all-pairs shortest path algorithm**.

1. Treat the given adjacency matrix as the distance matrix.
2. Use three nested loops:
    - Outer loop picks an intermediate node `k`.
    - Inner loops try to improve the path from `i` to `j` using node `k`.
3. For every pair `(i, j)`:
    - If there is no path from `i` to `k` or `k` to `j`, skip.
    - If there is no direct edge from `i` to `j`, update it using `i → k → j`.
    - Otherwise, take the minimum of the current distance and the new distance via `k`.
4. After processing all intermediate nodes, the matrix contains the shortest distances.

This works because every shortest path can be built by gradually allowing more intermediate vertices.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    void shortest_distance(vector<vector<int>> &matrix){
        int n = matrix.size();

        for(int k = 0; k < n; k++){
            for(int i = 0; i < n; i++){
                for(int j = 0; j < n; j++){
                    if(matrix[i][k] == -1 || matrix[k][j] == -1)
                        continue;

                    if(matrix[i][j] == -1){
```

```
                    matrix[i][j] = matrix[i][k] + matrix[k][j];
                }
                else{
                    matrix[i][j] = min(matrix[i][j],
                                       matrix[i][k] +
matrix[k][j]);
                }
            }
        }
    }
};

int main(){
    vector<vector<int>> matrix = {
        {0, 2, -1, -1},
        {1, 0, 3, -1},
        {-1, -1, 0, 1},
        {3, 5, 4, 0}
    };

    Solution sol;
    sol.shortest_distance(matrix);

    for(int i = 0; i < matrix.size(); i++){
        for(int j = 0; j < matrix.size(); j++){
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(V^3)
Three nested loops over all vertices.

Space Complexity: `O(V^2)`
The adjacency matrix itself stores distances for all vertex pairs.

38. Find the City With the Smallest Number of Neighbours at a Threshold Distance: G-43

You are given `n` cities numbered from `0` to `n-1`. The cities are connected by **bidirectional weighted edges**.
Each edge `[u, v, w]` means the distance between city `u` and city `v` is `w`.

You are also given an integer `distanceThreshold`.

For every city, count how many cities (including itself) are reachable with **shortest distance ≤ distanceThreshold**.
You must return the city that has the **smallest number of reachable cities**.
If multiple cities have the same minimum count, return the city with the **largest city number**.

Example:
If city 0 and city 3 both can reach only 2 cities within the threshold, the answer is city 3.

Approach

Algorithm

This problem is solved using the **Floyd Warshall Algorithm** to compute all-pairs shortest paths.

1. Create a distance matrix `dist` of size `n x n`.
2. Initialize all distances as `INT_MAX`.
3. For every edge `[u, v, w]`:

- Set `dist[u][v] = w`
- Set `dist[v][u] = w` (because the graph is bidirectional)

4. Set `dist[i][i] = 0` for all cities.
5. Apply Floyd Warshall:
   - For every intermediate city k
   - For every pair `(i, j)`
   - Update
     `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`
6. For each city:
   - Count how many cities have distance ≤ `distanceThreshold`.
7. Keep track of the minimum count.
   - If the count is smaller or equal, update the answer city.
   - Equal is allowed because we want the **largest city number** in case of a tie.
8. Return the city number.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findCity(int n, int m, vector<vector<int>>& edges, int
distanceThreshold) {

        vector<vector<int>> dist(n, vector<int>(n, INT_MAX));
```

```
        for(auto it : edges){
            dist[it[0]][it[1]] = it[2];
            dist[it[1]][it[0]] = it[2];
        }

        for(int i = 0; i < n; i++)
            dist[i][i] = 0;

        for(int k = 0; k < n; k++){
            for(int i = 0; i < n; i++){
                for(int j = 0; j < n; j++){
                    if(dist[i][k] == INT_MAX || dist[k][j] == INT_MAX)
                        continue;
                    dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
                }
            }
        }

        int cntCity = n;
        int cityNo = -1;

        for(int city = 0; city < n; city++){
            int cnt = 0;
            for(int adjCity = 0; adjCity < n; adjCity++){
                if(dist[city][adjCity] <= distanceThreshold)
                    cnt++;
            }
            if(cnt <= cntCity){
                cntCity = cnt;
                cityNo = city;
            }
        }
        return cityNo;
    }
};

int main(){
```

```
    int n = 4, m = 4;
    vector<vector<int>> edges = {
        {0,1,3},{1,2,1},{1,3,4},{2,3,1}
    };
    int distanceThreshold = 4;

    Solution obj;
    cout << obj.findCity(n, m, edges, distanceThreshold);
    return 0;
}
```

Complexity Analysis

Time Complexity: `O(V^3)`
Three nested loops are used in Floyd Warshall, where `V` is the number of cities.

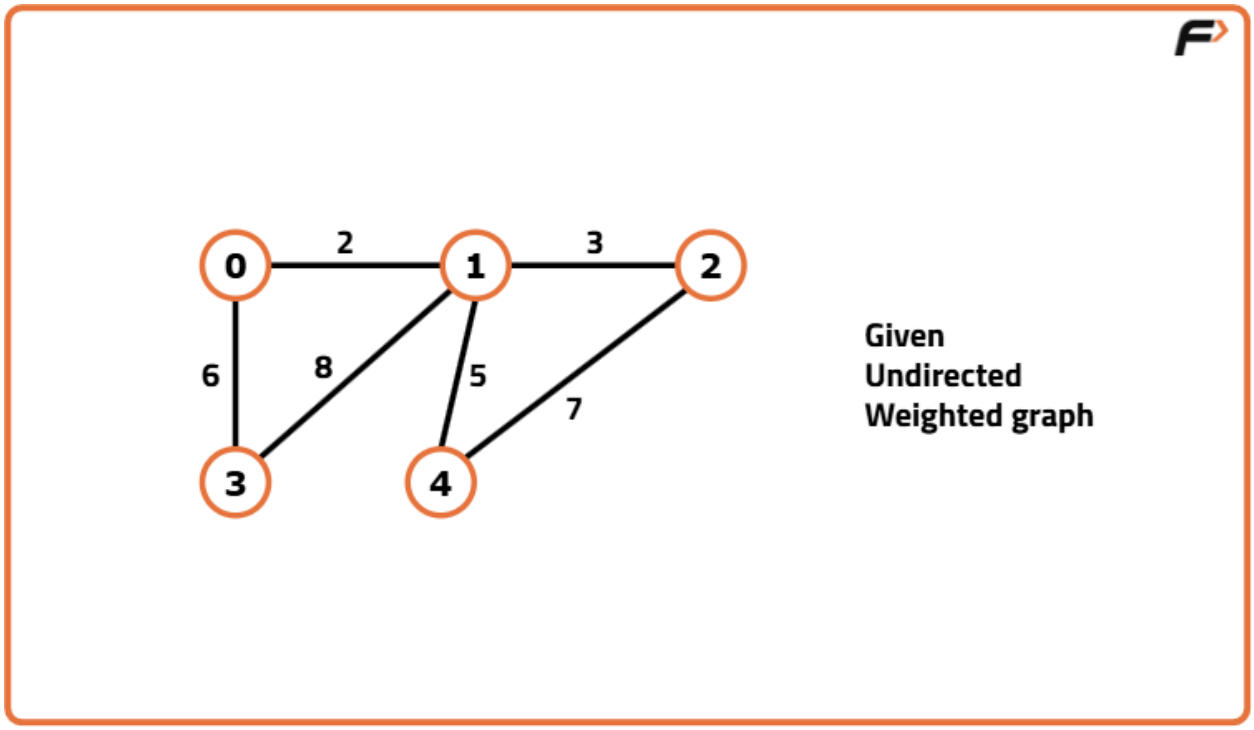
Space Complexity: `O(V^2)`
The distance matrix of size `V x V` is used to store shortest paths.

39. Minimum Spanning Tree - Theory: G-44

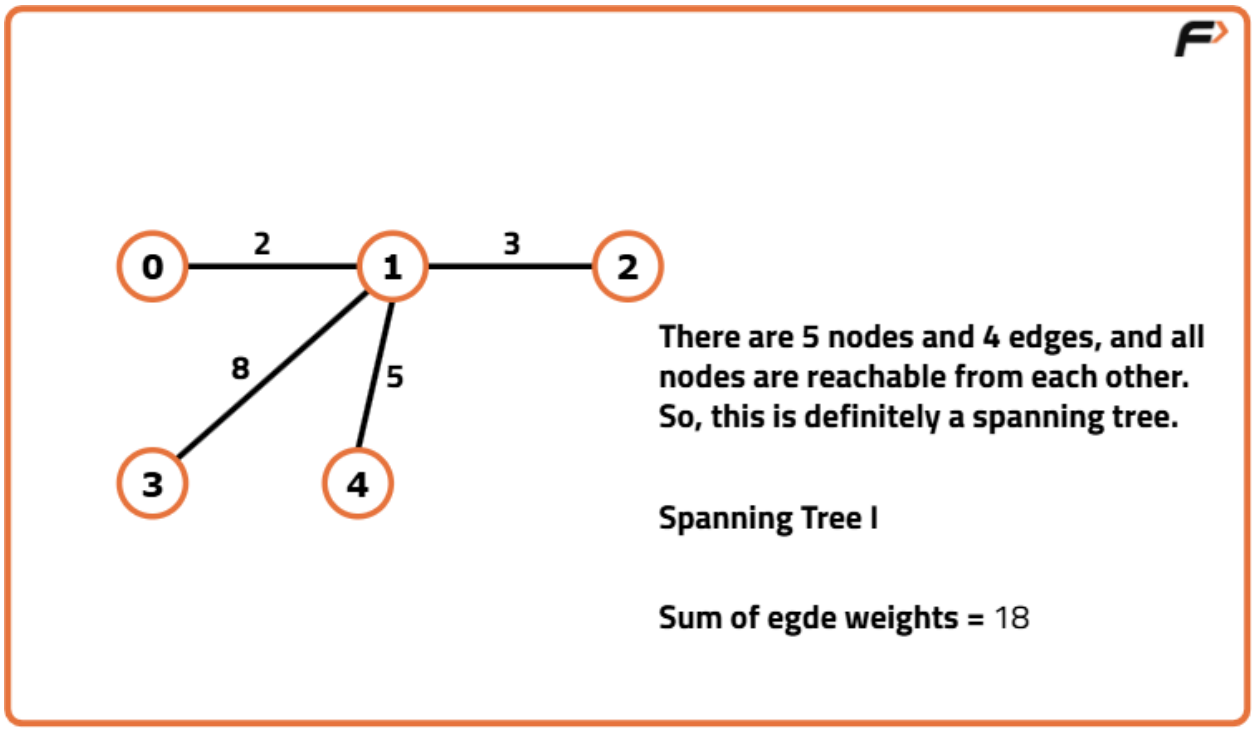
Spanning Tree:

A spanning tree is a <u>tree</u> in which we have N nodes(i.e. All the nodes present in the original graph) and N-1 edges and all nodes are reachable from each other.

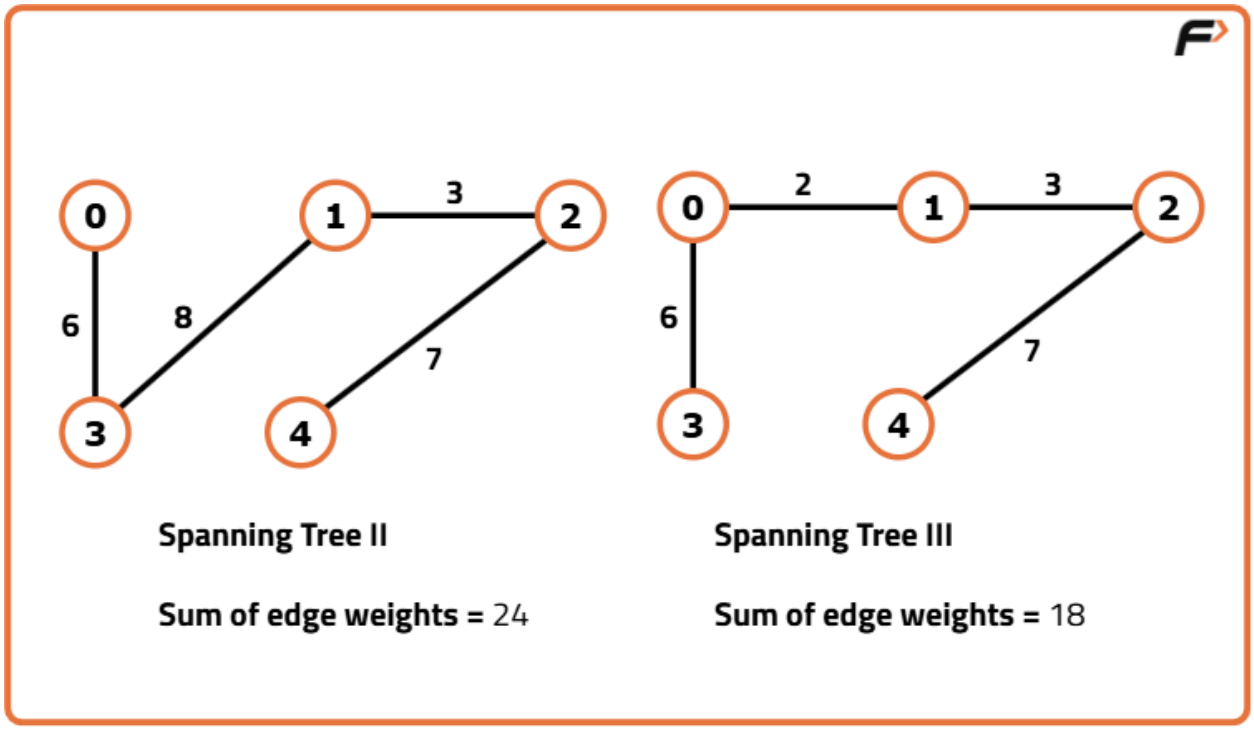
Let’s understand this using an example. Assume we are given an undirected weighted graph with N nodes and M edges. Here in this example, we have taken N as 5 and M as 6.



For the above graph, if we try to draw a spanning tree, the following illustration will be one:



We can draw more spanning trees for the given graph. Two of them are like the following:



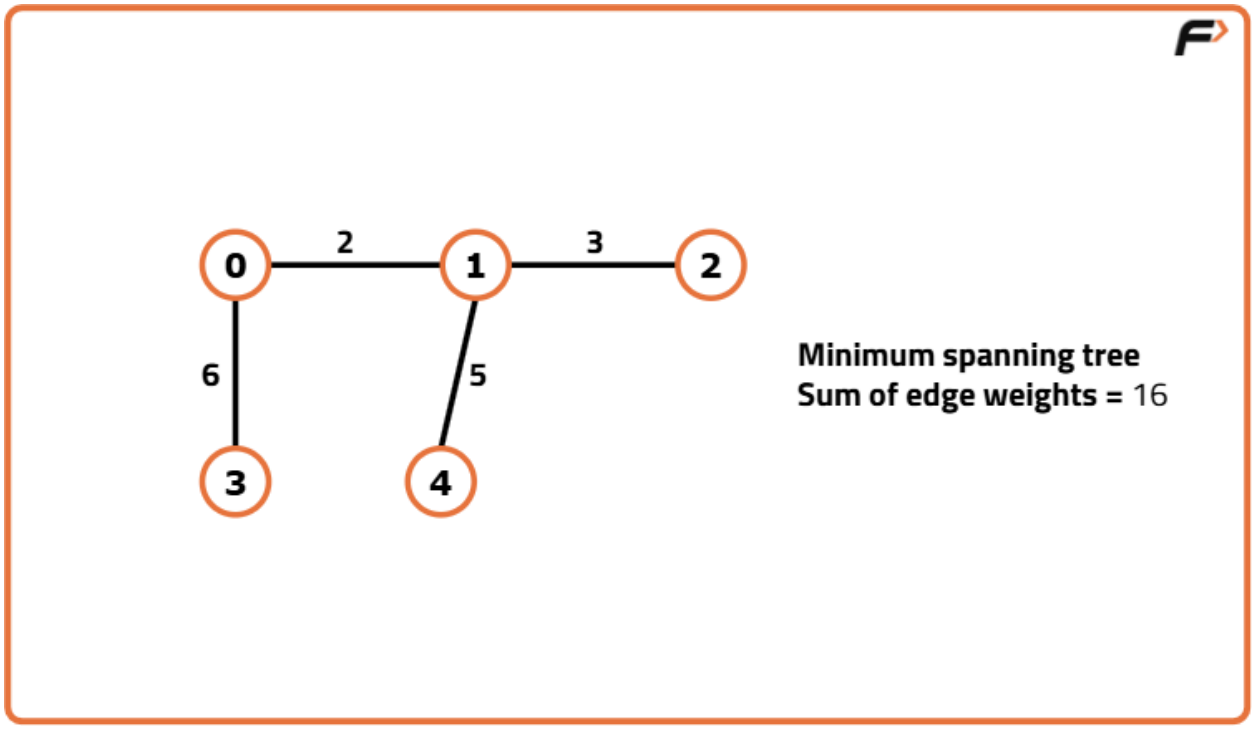
Note: *Point to remember is that a graph may have more than one spanning trees.*

All the above spanning trees contain some edge weights. For each of them, if we add the edge weights we can get the sum for that particular tree. Now, let's try to figure out the minimum spanning tree:

Minimum Spanning Tree:

Among all possible spanning trees of a graph, the minimum spanning tree is the one for which the sum of all the edge weights is the minimum.

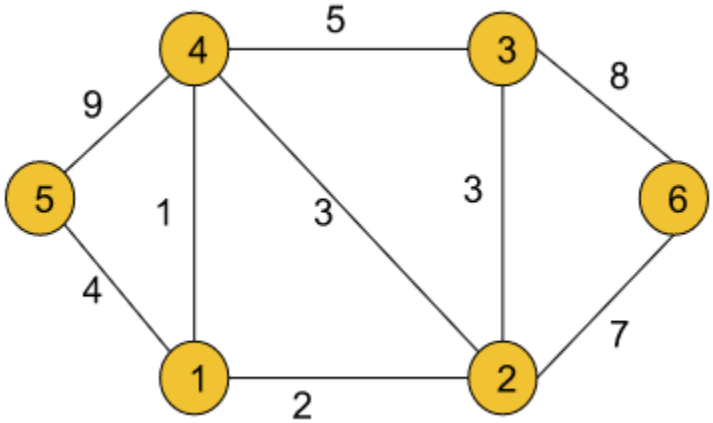
Let's understand the definition using the given graph drawn above. Until now, for the given graph we have drawn three spanning trees with the sum of edge weights 18, 24, and 18. If we can draw all possible spanning trees, we will find that the following spanning tree with the minimum sum of edge weights 16 is the ***minimum spanning tree*** for the given graph:



Note: *There may exist multiple minimum spanning trees for a graph like a graph may have multiple spanning trees.*

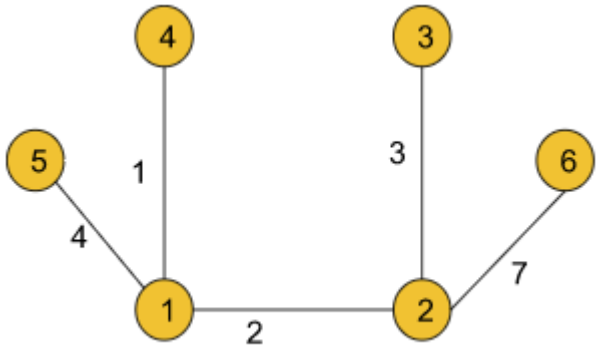
Practice Problem

Now, in case you have understood the concepts well, you can try to figure out the minimum spanning tree for the following undirected weighted graph:



Answer:Sum of edge

weights = 17The MST of the given Graph will look like



this:

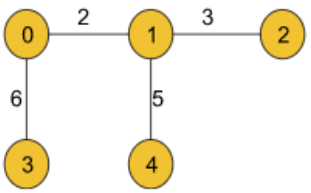
There are a couple of algorithms that help us to find the minimum spanning tree of a graph. One is **Prim's algorithm** and the other is **Kruskal's algorithm**. We will be discussing all of them in the upcoming articles.

40. Prim's Algorithm – Minimum Spanning Tree: G-45

You are given a **weighted, undirected, and connected graph** with V vertices and E edges.
Your task is to find the **sum of the weights of the edges in the Minimum Spanning Tree (MST)**.
Sometimes, you may also be asked to store the MST edges, but here the focus is on the **sum of weights**.

A **Minimum Spanning Tree** connects all vertices using exactly V-1 edges such that the total edge weight is minimum and no cycles are formed.

Example:
If the MST edges are `(0,1)`, `(0,3)`, `(1,2)`, `(1,4)` and their weights add up to 16, then the answer is 16.



Sum of edge weights = 16

Approach

Algorithm

Prim's Algorithm builds the MST **greedily**, starting from any node and always choosing the minimum weight edge that connects a visited node to an unvisited node.

Steps:

1. Use a **visited array** to mark nodes that are already part of the MST.
2. Use a **priority queue (min-heap)** that stores pairs `{edgeWeight, node}`.
3. Start from node `0` by pushing `{0, 0}` into the priority queue.
4. Initialize `sum = 0` to store the total weight of the MST.
5. While the priority queue is not empty:
    - Pop the element with the minimum edge weight.
    - If the node is already visited, skip it.
    - Otherwise:
        - Mark the node as visited.
        - Add the edge weight to `sum`.
        - Push all adjacent unvisited nodes into the priority queue with their edge weights.
6. When all nodes are visited, `sum` contains the total weight of the MST.

This works because at every step we **greedily pick the smallest edge** that expands the current tree without forming a cycle.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int spanningTree(int V, vector<vector<int>> adj[]) {

        priority_queue<pair<int,int>,
                       vector<pair<int,int>>,
                       greater<pair<int,int>>> pq;

        vector<int> vis(V,0);

        pq.push({0,0}); // {weight, node}
        int sum = 0;

        while(!pq.empty()){
            auto it = pq.top();
            pq.pop();
            int wt = it.first;
            int node = it.second;

            if(vis[node]) continue;

            vis[node] = 1;
            sum += wt;

            for(auto x : adj[node]){
                int adjNode = x[0];
                int edW = x[1];
                if(!vis[adjNode]){
                    pq.push({edW, adjNode});
                }
```

```
            }
        }
        return sum;
    }
};

int main(){
    int V = 5;
    vector<vector<int>> edges = {
        {0,1,2},{0,2,1},{1,2,1},
        {2,3,2},{3,4,1},{4,2,2}
    };

    vector<vector<int>> adj[V];
    for(auto it : edges){
        adj[it[0]].push_back({it[1], it[2]});
        adj[it[1]].push_back({it[0], it[2]});
    }

    Solution obj;
    cout << obj.spanningTree(V, adj);
    return 0;
}
```

Complexity Analysis

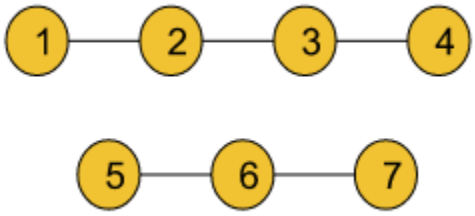
Time Complexity: `O(E log E)`
Each edge can be pushed into the priority queue, and push/pop operations take `log E` time.

Space Complexity: `O(E + V)`
`O(E)` for the priority queue and adjacency list, and `O(V)` for the visited array.

41. Disjoint Set | Union by Rank | Union by Size | Path Compression: G-46

A **Disjoint Set** (also called Union-Find) is a data structure used to manage a collection of **non-overlapping sets**.
It is mainly used to answer questions like:
"Do node u and node v belong to the same connected component?"

This is very useful in **graphs that change over time (dynamic graphs)**, where edges are added step by step and we need fast connectivity checks.



Why Disjoint Set is needed

Suppose we are given an undirected graph and asked whether node 1 and node 5 belong to the same component.

- Using DFS or BFS: `O(N + E)` time for every query.
- Using Disjoint Set: **almost constant time** per query.

That is why Disjoint Set is preferred for **multiple connectivity queries**.

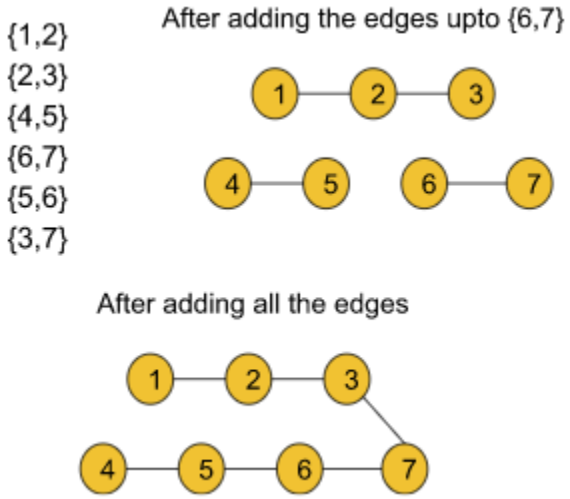
Dynamic Graph Concept

A dynamic graph is a graph where edges are added over time.

Example edge sequence:
`{(1,2), (2,3), (4,5), (6,7), (5,6), (3,7)}`

- After adding first 4 edges → node 1 and node 4 are in different components.
- After adding all edges → node 1 and node 4 become part of the same component.

Disjoint Set can answer such queries efficiently **after every update**.



Operations in Disjoint Set

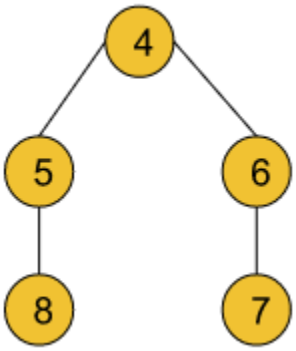
1. **findUPar(node)**
   Finds the **ultimate parent (root)** of a node.
2. **Union(u, v)**
   Merges the sets containing u and v.

Optimizations:

- Union by Rank
- Union by Size
- Path Compression

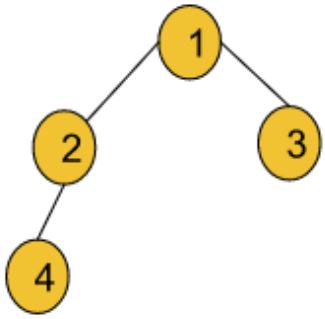
Important Terminology

Ultimate Parent
The topmost parent (root) of a component.



In this graph, the parent of 8 is 5 but the ultimate parent of 8 is 4

Rank
An approximate measure of the height of the tree.



Here the rank of node 1 is 2 as the distance between node 1 and the furthest leaf node 4 is 2.

Size
Number of nodes in a connected component.

Union by Rank

In union by rank:

- Each node has a `rank` value.
- While merging two components, attach the component with **smaller rank** under the one with **larger rank**.
- If ranks are equal, attach any one and increase the rank by 1.

This helps in keeping the tree shallow.

Why Ultimate Parent Matters

Two nodes may have different immediate parents but still belong to the same component.

Example:

- Node 5 → parent 6
- Node 7 → parent 6

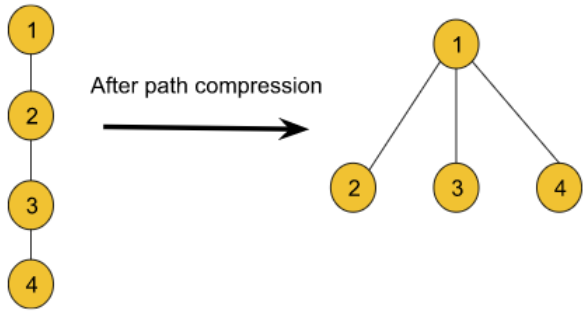
Immediate parents differ, but **ultimate parent is same**, so they are connected.

Hence, comparisons are always done using **ultimate parents**.

Path Compression

While finding the ultimate parent:

- We directly connect every node in the path to the ultimate parent.
- This flattens the structure.

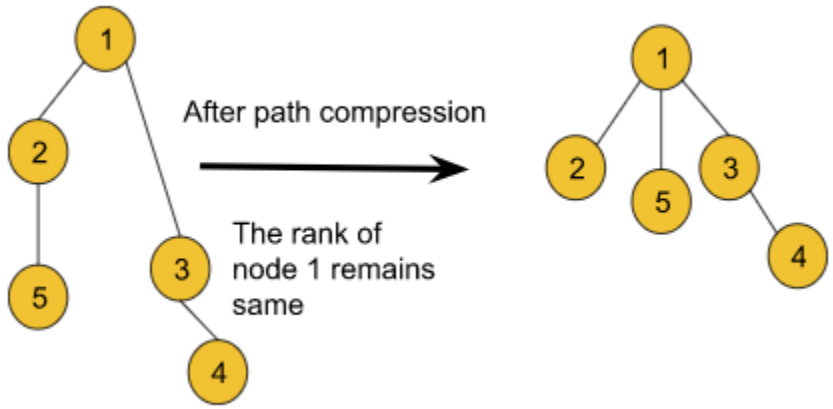


Before path compression:

- Finding parent may take `O(log N)` time.

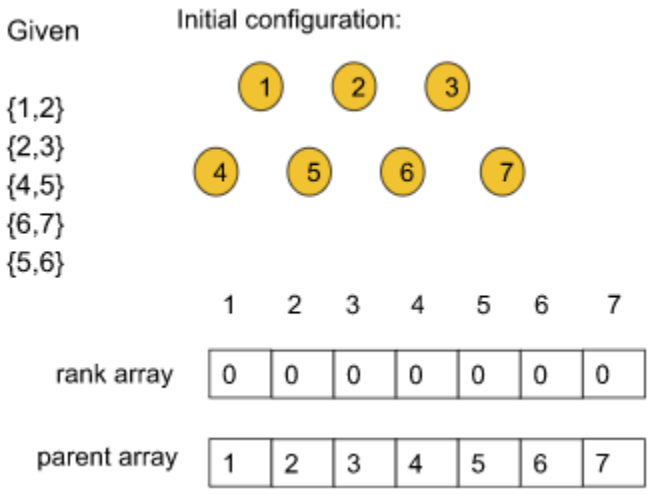
After path compression:

- Finding parent takes **almost constant time**.



Important:

- **Rank should not be updated during path compression** because it may get distorted.



Union by Rank Code (with Path Compression)

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
    vector<int> rank, parent;
public:
    DisjointSet(int n) {
        rank.resize(n+1, 0);
        parent.resize(n+1);
        for(int i=0;i<=n;i++)
            parent[i]=i;
    }

    int findUPar(int node){
        if(node==parent[node])
            return node;
        return parent[node]=findUPar(parent[node]);
    }

    void unionByRank(int u,int v){
        int pu=findUPar(u);
        int pv=findUPar(v);
        if(pu==pv) return;

        if(rank[pu]<rank[pv]){
            parent[pu]=pv;
        }
        else if(rank[pv]<rank[pu]){
            parent[pv]=pu;
        }
        else{
            parent[pv]=pu;
            rank[pu]++;
        }
    }
};
```

```
int main(){
    DisjointSet ds(7);
    ds.unionByRank(1,2);
    ds.unionByRank(2,3);
    ds.unionByRank(4,5);
    ds.unionByRank(6,7);
    ds.unionByRank(5,6);

    if(ds.findUPar(3)==ds.findUPar(7))
        cout<<"Same\n";
    else
        cout<<"Not same\n";

    ds.unionByRank(3,7);

    if(ds.findUPar(3)==ds.findUPar(7))
        cout<<"Same\n";
    else
        cout<<"Not same\n";
}
```

Union by Size

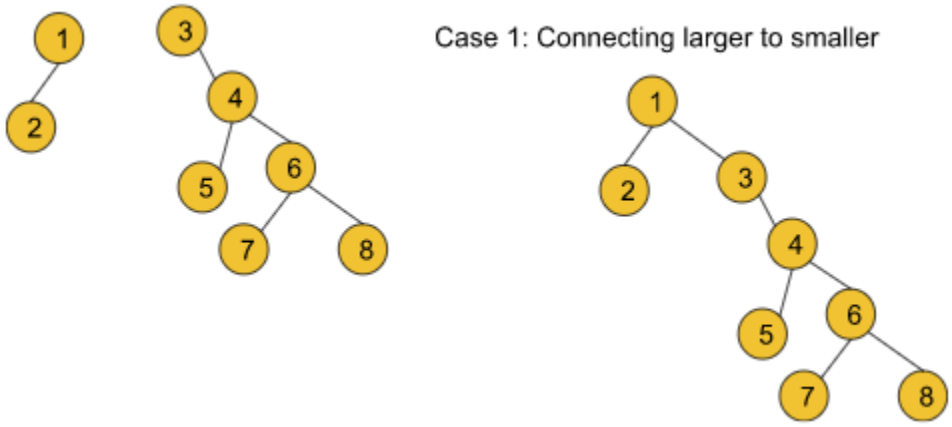
Union by size is similar to union by rank, but instead of rank:

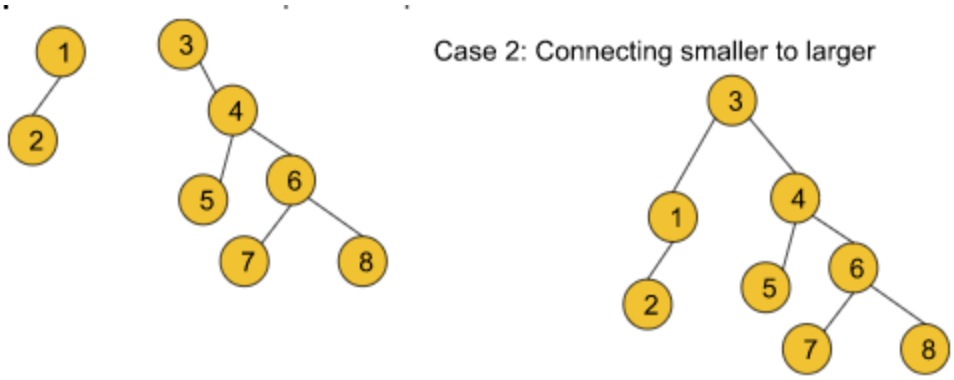
- We store the **size of each component**.
- Attach the smaller component under the larger one.
- Update the size accordingly.

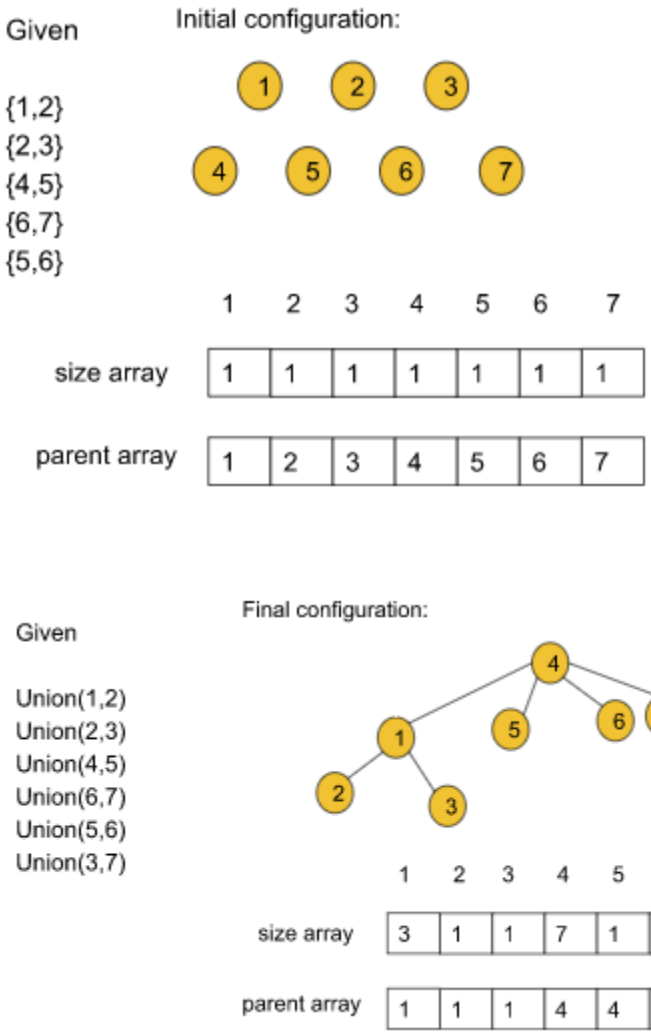
This method is more intuitive because **size does not get distorted** after path compression.

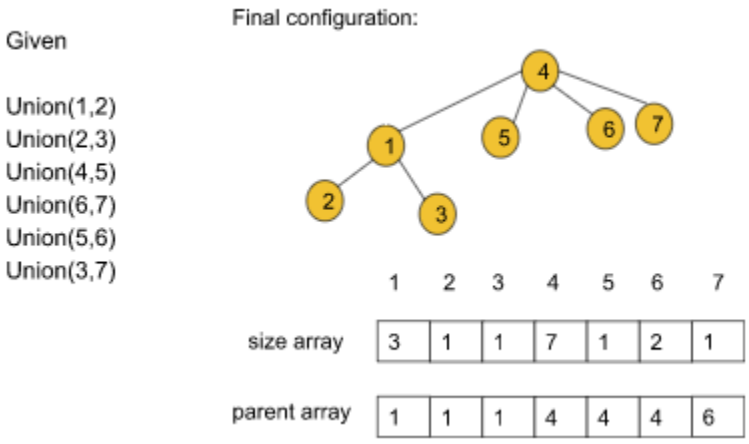
Union by Size Algorithm

1. Find ultimate parents of u and v.
2. Compare `size[pu]` and `size[pv]`.
3. Attach smaller size component to larger size component.
4. Update the size.









Union by Size Code

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
    vector<int> parent, size;
public:
    DisjointSet(int n){
        parent.resize(n+1);
        size.resize(n+1,1);
        for(int i=0;i<=n;i++)
            parent[i]=i;
```

```
    }

    int findUPar(int node){
        if(node==parent[node])
            return node;
        return parent[node]=findUPar(parent[node]);
    }

    void unionBySize(int u,int v){
        int pu=findUPar(u);
        int pv=findUPar(v);
        if(pu==pv) return;

        if(size[pu]<size[pv]){
            parent[pu]=pv;
            size[pv]+=size[pu];
        }
        else{
            parent[pv]=pu;
            size[pu]+=size[pv];
        }
    }
};

int main(){
    DisjointSet ds(7);
    ds.unionBySize(1,2);
    ds.unionBySize(2,3);
    ds.unionBySize(4,5);
    ds.unionBySize(6,7);
    ds.unionBySize(5,6);

    if(ds.findUPar(3)==ds.findUPar(7))
        cout<<"Same\n";
    else
        cout<<"Not same\n";

    ds.unionBySize(3,7);
```

```
    if(ds.findUPar(3)==ds.findUPar(7))
        cout<<"Same\n";
    else
        cout<<"Not same\n";
}
```

Time Complexity

- **findUPar + union operations:** `O(α(N))`
- `α(N)` (inverse Ackermann function) is very small
- Practically treated as **constant time**

42. Kruskal's Algorithm – Minimum Spanning Tree: G-47

You are given a **weighted, undirected, and connected graph** with `V` vertices and `E` edges. Your task is to find the **sum of weights of the edges in the Minimum Spanning Tree (MST)**.

A Minimum Spanning Tree connects all vertices using exactly `V-1` edges, has no cycles, and the total edge weight is minimum.

Example:
For edges `[0-1(1), 1-2(2), 2-3(3), 0-3(4)]`, the MST uses edges with weights `1, 2, 3`, so the answer is `6`.

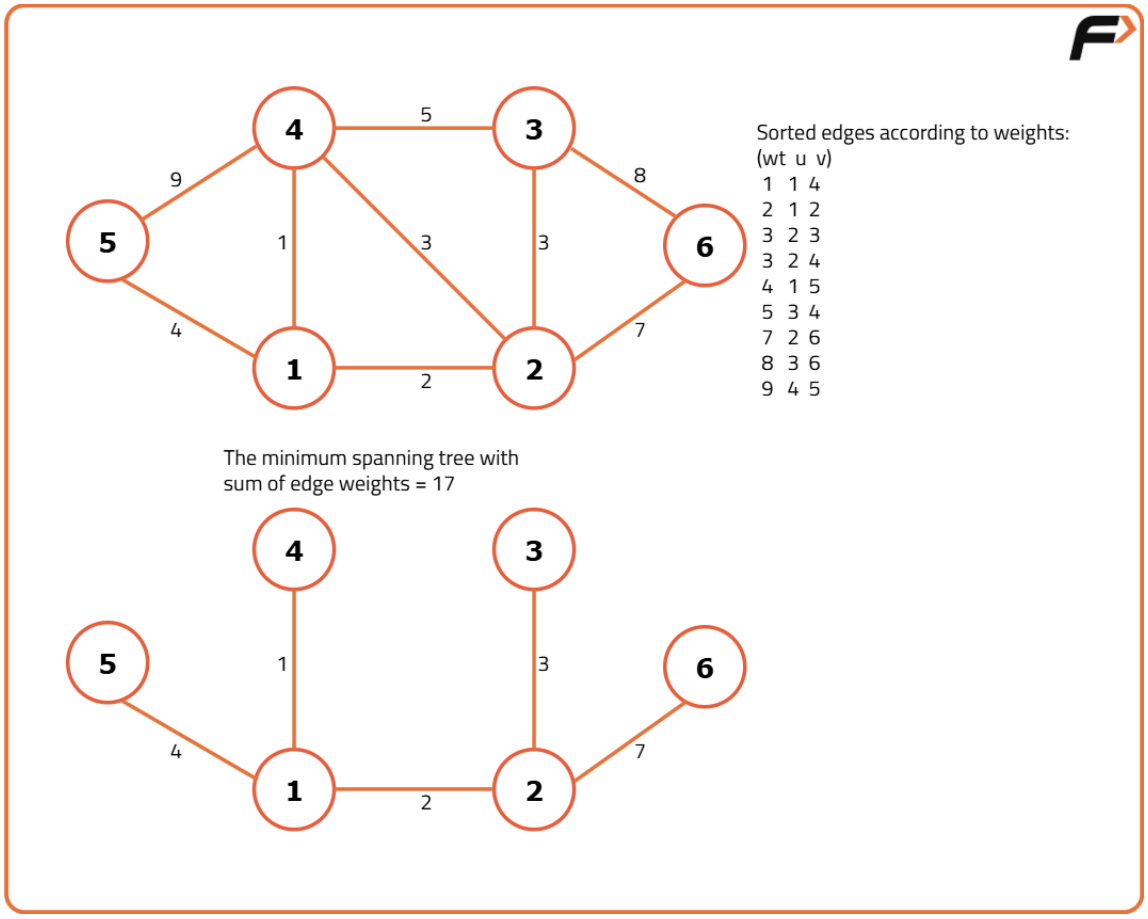
Approach

Algorithm

Kruskal's Algorithm builds the MST by **always choosing the smallest available edge** that does not form a cycle.

1. Store all edges in the form `(weight, u, v)`.
2. Sort all edges in **ascending order of weight**.
3. Create a **Disjoint Set (Union-Find)** to track connected components.
4. Iterate through the sorted edges:
    - If u and v belong to different components:
        - Add the edge weight to the MST sum.
        - Union the components of u and v.
    - Otherwise, skip the edge (it forms a cycle).
5. After processing all edges, the accumulated sum is the MST weight.

Cycle detection is efficiently handled using the Disjoint Set data structure.



Code

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
    vector<int> parent, size;
public:
    DisjointSet(int n) {
        parent.resize(n+1);
        size.resize(n+1,1);
        for(int i=0;i<=n;i++) parent[i]=i;
    }

    int findUPar(int node) {
        if(node==parent[node]) return node;
```

```
        return parent[node]=findUPar(parent[node]);
    }

    void unionBySize(int u,int v) {
        int pu=findUPar(u);
        int pv=findUPar(v);
        if(pu==pv) return;
        if(size[pu]<size[pv]) {
            parent[pu]=pv;
            size[pv]+=size[pu];
        } else {
            parent[pv]=pu;
            size[pu]+=size[pv];
        }
    }
};

class Solution {
public:
    int spanningTree(int V, vector<vector<int>> adj[]) {
        vector<pair<int,pair<int,int>>> edges;

        for(int i=0;i<V;i++){
            for(auto it: adj[i]){
                edges.push_back({it[1],{i,it[0]}});
            }
        }

        sort(edges.begin(),edges.end());

        DisjointSet ds(V);
        int sum=0;

        for(auto it: edges){
            int wt=it.first;
            int u=it.second.first;
            int v=it.second.second;
```

```
            if(ds.findUPar(u)!=ds.findUPar(v)){
                sum+=wt;
                ds.unionBySize(u,v);
            }
        }
        return sum;
    }
};

int main(){
    int V=4;
    vector<vector<int>> edges={{0,1,1},{1,2,2},{2,3,3},{0,3,4}};
    vector<vector<int>> adj[4];

    for(auto it: edges){
        adj[it[0]].push_back({it[1],it[2]});
        adj[it[1]].push_back({it[0],it[2]});
    }

    Solution obj;
    cout<<obj.spanningTree(V,adj);
    return 0;
}
```

Complexity Analysis

Time Complexity:
`O(E log E)`
Sorting the edges dominates. Disjoint set operations are almost constant time.

Space Complexity:
`O(E + V)`
Edges array uses `O(E)` space, and Disjoint Set uses `O(V)` space.

43. Number of Operations to Make Network Connected – DSU: G-49

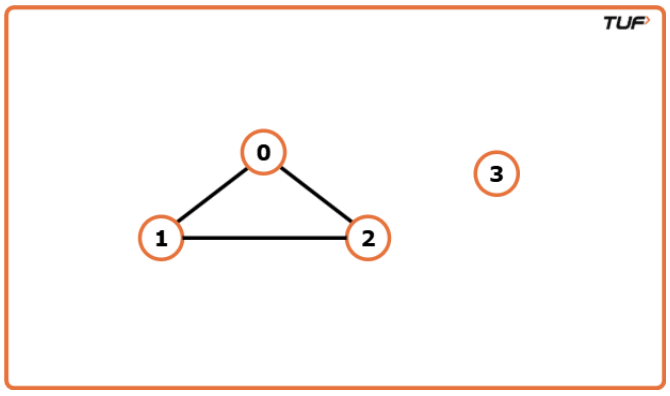
You are given a graph with `n` vertices and `m` edges.
In one operation, you can **remove any existing edge** and **add it between any two vertices**.

Your task is to find the **minimum number of operations** required to make the graph **connected**.
If it is not possible to make the graph connected, return `-1`.

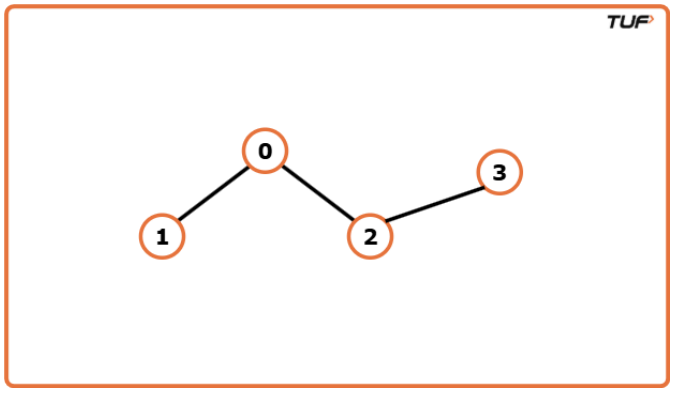
A graph is connected if there is a path between every pair of vertices.

Example:
If the graph has multiple disconnected components, we need to connect those components using available extra edges.

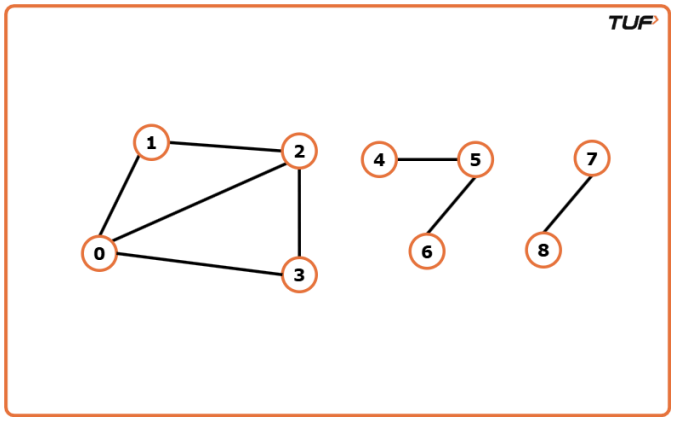
Input:



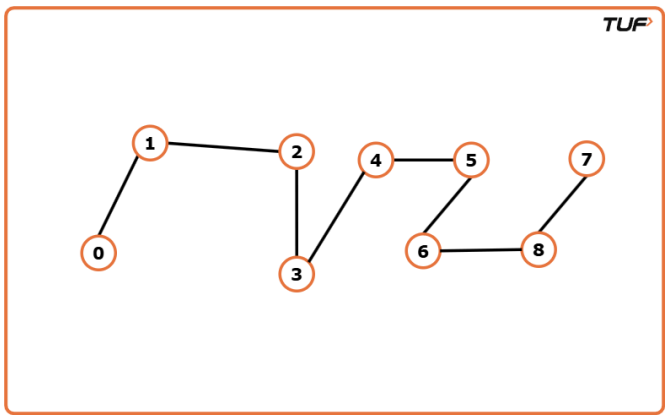
Output: 1



Input



Output: 1



We need a minimum of 2 operations to make the two components connected. We can remove the edge (0,2) and add the edge between node 3 and node 4 and we can remove the edge (0,3) and add it between nodes 6 and 8 like the following:

Approach

Algorithm

To make a graph connected:

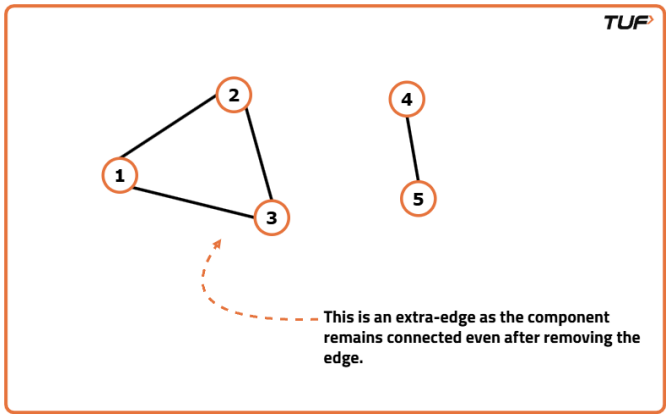
- If a graph has `components` connected components, then we need at least `components - 1` edges to connect them.
- We are not allowed to add new edges freely; we can only **reuse extra (redundant) edges**.

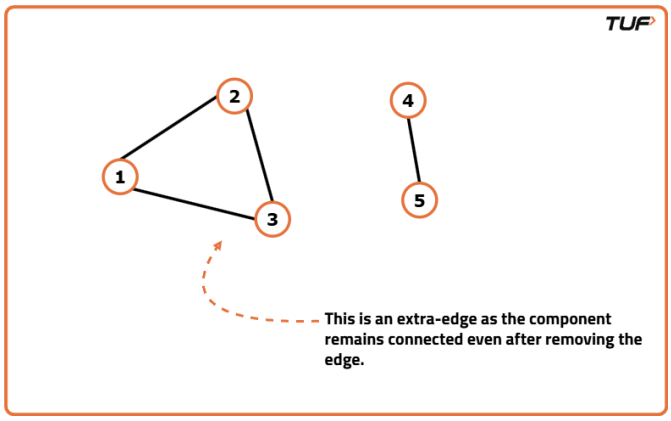
Steps:

1. Initialize a **Disjoint Set Union (DSU)** to track connected components.
2. Traverse all edges:
    - If an edge connects two nodes already in the same component, it is an **extra edge**.
    - Otherwise, union the two components.
3. After processing all edges, count the number of connected components.
4. To connect all components, we need `(components - 1)` edges.
5. If the number of extra edges is **at least** `(components - 1)`, return `(components - 1)`.
6. Otherwise, return `-1`.

Important check:

- If total edges `< n - 1`, it is **impossible** to connect the graph.





Code

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
public:
    vector<int> parent, rank;

    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for(int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
```

```
        if(parent[x] == x)
            return x;
        return parent[x] = find(parent[x]);
    }

    void unite(int x, int y) {
        int px = find(x);
        int py = find(y);

        if(px == py) return;

        if(rank[px] < rank[py])
            parent[px] = py;
        else if(rank[px] > rank[py])
            parent[py] = px;
        else {
            parent[py] = px;
            rank[px]++;
        }
    }
};

class Solution {
public:
    int makeConnected(int n, vector<vector<int>>& connections) {

        if(connections.size() < n - 1)
            return -1;

        DSU dsu(n);

        for(auto &edge : connections){
            dsu.unite(edge[0], edge[1]);
        }

        unordered_set<int> components;
        for(int i = 0; i < n; i++){
            components.insert(dsu.find(i));
```

```
        }

        return components.size() - 1;
    }
};

int main() {
    int n = 6;
    vector<vector<int>> connections = {{0,1},{0,2},{0,3},{1,4}};
    Solution obj;
    cout << obj.makeConnected(n, connections);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N + M \times \alpha(N))$
Each DSU operation is almost constant time, and we process all nodes and edges once.

Space Complexity: $O(N)$
Used for DSU parent and rank arrays.

44. Most Stones Removed with Same Row or Column – DSU: G-53

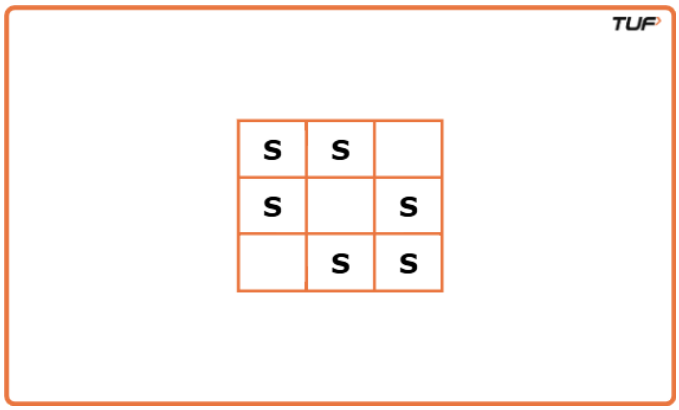
You are given n stones placed on a 2D plane. Each stone is placed at a unique coordinate (x, y).
A stone can be removed **only if there exists another stone in the same row or the same column that is not removed**.

Your task is to find the **maximum number of stones** that can be removed following this rule.

Key idea:
From a group of stones that are connected by same rows or columns, **we must leave at least one stone**, and all others can be removed.

Explanation with example

Example:
`stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]`



All stones are connected through rows or columns, forming **one single group**.
From one group of size 6, we can remove `6 - 1 = 5` stones.

Another example may form **multiple groups**.
From each group, we can remove `(group size - 1)` stones.
So overall:

Maximum stones removed = total stones − number of connected groups

---

Approach

Algorithm

We use **Disjoint Set Union (DSU)** to group stones that lie in the same row or same column.

1. Treat each **row index** and each **column index** as separate DSU nodes.

2. To avoid collision between row and column indices, add an offset to column indices.

3. For every stone `(x, y)`:
    - Union `x` (row) with `y + offset` (column).
4. After all unions:
    - Count how many **unique connected components** exist (based on ultimate parents of rows).
5. Since one stone must remain in each component:
    - `answer = total stones - number of components`

Code

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
public:
    unordered_map<int,int> parent;

    int find(int x){
        if(parent.find(x)==parent.end())
            parent[x]=x;
        if(parent[x]==x) return x;
        return parent[x]=find(parent[x]);
    }

    void unite(int x,int y){
        int px=find(x);
        int py=find(y);
        if(px!=py)
            parent[px]=py;
    }
};

class Solution {
public:
```

```
    int removeStones(vector<vector<int>>& stones) {
        DSU dsu;
        int offset = 10001;

        for(auto &stone: stones){
            dsu.unite(stone[0], stone[1] + offset);
        }

        unordered_set<int> components;
        for(auto &stone: stones){
            components.insert(dsu.find(stone[0]));
        }

        return stones.size() - components.size();
    }
};

int main(){
    vector<vector<int>> stones = {
        {0,0},{0,1},{1,0},{1,2},{2,1},{2,2}
    };

    Solution obj;
    cout << obj.removeStones(stones);
    return 0;
}
```

Complexity Analysis

Time Complexity:
$O(N \times \alpha(N))$
Each union and find operation is almost constant time.

Space Complexity:
$O(N)$
DSU stores parents for rows and columns involved.

45. Accounts Merge – DSU: G-50

You are given multiple user accounts.
Each account contains a **user name** followed by one or more **email addresses**.

Two accounts belong to the **same person** if they share **at least one common email**.
Your task is to **merge such accounts** and return the result where:

- Each merged account starts with the user's name
- Followed by **sorted, unique emails**
- Output order does not matter

Example Explanation

Example 1:
Two accounts of `"John"` share the email `johnsmith@mail.com`, so they belong to the same person and must be merged.
Accounts of `"Mary"` and another `"John"` do not share emails with_ud, so they stay separate.

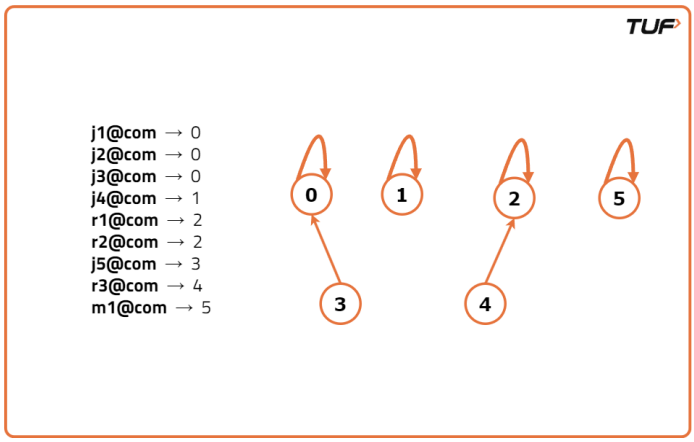
Algorithm

We treat **each account index as a node** and use **Disjoint Set Union (DSU)** to merge accounts that share emails.

1. Create a Disjoint Set for n accounts.
2. Create a map `email → account index`.
3. Traverse each account:
    - For every email:
        - If the email is not in the map, store `email → current index`

        - If it already exists, union the current account index with the stored index

4. After all unions:
    - Traverse all emails again
    - Find the **ultimate parent account** for each email
    - Group emails under their ultimate parent
5. For each group:
    - Sort emails
    - Add the account name at the front
6. Store all merged accounts in the result.



Code

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
    vector<int> parent, size;
public:
    DisjointSet(int n) {
        parent.resize(n);
        size.resize(n,1);
```

```
        for(int i=0;i<n;i++) parent[i]=i;
    }

    int findUPar(int node) {
        if(node==parent[node]) return node;
        return parent[node]=findUPar(parent[node]);
    }

    void unionBySize(int u,int v) {
        int pu=findUPar(u);
        int pv=findUPar(v);
        if(pu==pv) return;
        if(size[pu]<size[pv]) {
            parent[pu]=pv;
            size[pv]+=size[pu];
        } else {
            parent[pv]=pu;
            size[pu]+=size[pv];
        }
    }
};

class Solution {
public:
    vector<vector<string>> accountsMerge(vector<vector<string>>&
details) {
        int n=details.size();
        DisjointSet ds(n);
        unordered_map<string,int> mailToNode;

        for(int i=0;i<n;i++){
            for(int j=1;j<details[i].size();j++){
                string mail=details[i][j];
                if(mailToNode.find(mail)==mailToNode.end())
                    mailToNode[mail]=i;
                else
                    ds.unionBySize(i,mailToNode[mail]);
            }
```

```
        }

        vector<vector<string>> mergedMail(n);
        for(auto it: mailToNode){
            string mail=it.first;
            int node=ds.findUPar(it.second);
            mergedMail[node].push_back(mail);
        }

        vector<vector<string>> ans;
        for(int i=0;i<n;i++){
            if(mergedMail[i].empty()) continue;
            sort(mergedMail[i].begin(), mergedMail[i].end());
            vector<string> temp;
            temp.push_back(details[i][0]);
            for(auto &mail: mergedMail[i])
                temp.push_back(mail);
            ans.push_back(temp);
        }
        return ans;
    }
};

int main(){
    vector<vector<string>> accounts={
        {"John","j1@com","j2@com","j3@com"},
        {"John","j4@com"},
        {"Raj","r1@com","r2@com"},
        {"John","j1@com","j5@com"},
        {"Raj","r2@com","r3@com"},
        {"Mary","m1@com"}
    };

    Solution obj;
    auto res=obj.accountsMerge(accounts);

    for(auto &acc: res){
        for(auto &s: acc) cout<<s<<" ";
```

```
        cout<<endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity:
`O(N + E) + O(E log E)`

- Traversing emails and unions: `O(N + E)`
- Sorting emails for each merged account

Space Complexity:
`O(N + E)`

- DSU arrays
- Email map
- Merged email storage

46. Number of Islands – II (Online Queries) – DSU: G-51

You are given a grid of size `n × m` which is initially filled with water (`0`).
You are also given `k` operations. Each operation turns a water cell into land (`1`).

After **each operation**, you need to return the **current number of islands**.

An **island** is a group of land cells (1) connected **vertically or horizontally**.

Important points:

- Initially, there are **no islands**.
- The same cell can appear multiple times in the operations.
- You must return an answer **after every operation**.

Example Explanation

Example 1:
`n = 4, m = 5`
Operations: `{(1,1),(0,1),(3,3),(3,4)}`

After each operation, islands count becomes:
`1 1 2 2`

This happens because new land may either:

- Create a new island
- Or merge with existing nearby islands

Approach

This is an **online connectivity problem**, best solved using **Disjoint Set Union (DSU)**.

1. Treat each cell `(row, col)` as a node using mapping:
   `node = row * m + col`
2. Maintain a `visited` matrix to mark land cells.
3. Maintain a DSU for `n * m` possible cells.
4. Keep a counter `cnt` for number of islands.
5. For each operation `(row, col)`:

- If the cell is already land, push current `cnt` and continue.
- Otherwise:
    - Mark it as land.
    - Increase `cnt` by 1.
    - Check its 4 neighbors (up, right, down, left).
    - If any neighbor is land and belongs to a different DSU component:
        - Union them.
        - Decrease `cnt` by 1 (two islands merged).
- Push `cnt` into the answer array.

6. Return the answer array.

This works because:

- Each new land starts as a new island.
- DSU efficiently merges connected lands.
- Island count updates dynamically.

Code

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
    vector<int> parent, size;
public:
    DisjointSet(int n) {
        parent.resize(n);
        size.resize(n,1);
```

```
        for(int i=0;i<n;i++) parent[i]=i;
    }

    int findUPar(int node){
        if(node==parent[node]) return node;
        return parent[node]=findUPar(parent[node]);
    }

    void unionBySize(int u,int v){
        int pu=findUPar(u);
        int pv=findUPar(v);
        if(pu==pv) return;
        if(size[pu]<size[pv]){
            parent[pu]=pv;
            size[pv]+=size[pu];
        } else {
            parent[pv]=pu;
            size[pu]+=size[pv];
        }
    }
};

class Solution {
    bool isValid(int r,int c,int n,int m){
        return r>=0 && r<n && c>=0 && c<m;
    }
public:
    vector<int> numOfIslands(int n, int m, vector<vector<int>>&
operators) {

        DisjointSet ds(n*m);
        vector<vector<int>> vis(n,vector<int>(m,0));
        vector<int> ans;
        int cnt=0;

        int dr[]={-1,0,1,0};
        int dc[]={0,1,0,-1};
```

```
        for(auto &op: operators){
            int r=op[0], c=op[1];

            if(vis[r][c]){
                ans.push_back(cnt);
                continue;
            }

            vis[r][c]=1;
            cnt++;

            int node=r*m+c;

            for(int i=0;i<4;i++){
                int nr=r+dr[i];
                int nc=c+dc[i];
                if(isValid(nr,nc,n,m) && vis[nr][nc]){
                    int adjNode=nr*m+nc;
                    if(ds.findUPar(node)!=ds.findUPar(adjNode)){
                        cnt--;
                        ds.unionBySize(node,adjNode);
                    }
                }
            }
            ans.push_back(cnt);
        }
        return ans;
    }
};

int main(){
    int n=4,m=5;
    vector<vector<int>> ops={{0,0},{0,0},{1,1},{1,0},{0,1},

{0,3},{1,3},{0,4},{3,2},{2,2},{1,2},{0,2}};
    Solution obj;
    vector<int> res=obj.numOfIslands(n,m,ops);
    for(int x:res) cout<<x<<" ";
```

```
    return 0;
}
```

Complexity Analysis

Time Complexity:
`O(K × α(N×M))`
Each operation involves constant DSU operations.

Space Complexity:
`O(N × M)`
For visited matrix and DSU arrays.

47. Making a Large Island – DSU: G-52

You are given an `n x n` binary grid where `1` represents land and `0` represents water.
An island is a group of 1s connected **horizontally or vertically**.

You are allowed to change **at most one `0` into `1`**.
Your task is to return the **maximum possible size of an island** after this operation.

Question Explanation

Initially, the grid may contain multiple islands or none at all.
When we change a `0` to `1`, it may:

- Create a new island of size 1
- Connect multiple nearby islands into one large island

The goal is to choose the best `0` to flip so that the resulting island size is maximum.

If the grid already contains only 1s, then no change is needed and the answer is simply the total number of cells.

Example Explanation

Example 1
`grid = [[1,0],[0,1]]`

There are two separate islands of size 1.
If we change any one `0` to 1, it connects both islands.

Resulting island size = 3.

Example 2
`grid = [[1,1],[1,1]]`

All cells are already connected.
Largest island size = 4.

Approach

Algorithm

We use **Disjoint Set Union (DSU)** to efficiently manage island connections and sizes.

1. Treat each cell `(i, j)` as a node using:
   `node = i * n + j`

2. Create a DSU of size `n * n`.

3. Traverse the grid and union all adjacent land cells (1s) to form initial islands.

4. Now, for every `0` cell:

    - Look at its 4 neighbors.

- Collect **unique island parents** of neighboring 1s using a set (to avoid double counting).
- Sum the sizes of these unique islands.
- Add 1 for the flipped cell.
- Update the maximum island size.

5. Handle the edge case where the grid has no `0`:
   - The answer is the size of the largest existing island.
6. Return the maximum size found.

Code

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
public:
    vector<int> parent, size;

    DisjointSet(int n) {
        parent.resize(n);
        size.resize(n, 1);
        for(int i = 0; i < n; i++)
            parent[i] = i;
    }

    int findUPar(int node) {
        if(node == parent[node]) return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionBySize(int u, int v) {
        int pu = findUPar(u);
```

```
        int pv = findUPar(v);
        if(pu == pv) return;

        if(size[pu] < size[pv]) {
            parent[pu] = pv;
            size[pv] += size[pu];
        } else {
            parent[pv] = pu;
            size[pu] += size[pv];
        }
    }
};

class Solution {
    bool isValid(int r, int c, int n) {
        return r >= 0 && r < n && c >= 0 && c < n;
    }

public:
    int largestIsland(vector<vector<int>>& grid) {
        int n = grid.size();
        DisjointSet ds(n * n);

        int dr[4] = {-1, 0, 1, 0};
        int dc[4] = {0, 1, 0, -1};

        // Step 1: Union existing islands
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(grid[i][j] == 0) continue;
                for(int d = 0; d < 4; d++) {
                    int ni = i + dr[d];
                    int nj = j + dc[d];
                    if(isValid(ni, nj, n) && grid[ni][nj] == 1) {
                        int node1 = i * n + j;
                        int node2 = ni * n + nj;
                        ds.unionBySize(node1, node2);
                    }
```

```
                }
            }
        }

        int ans = 0;

        // Step 2: Try converting each 0 to 1
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(grid[i][j] == 1) continue;

                set<int> components;
                for(int d = 0; d < 4; d++) {
                    int ni = i + dr[d];
                    int nj = j + dc[d];
                    if(isValid(ni, nj, n) && grid[ni][nj] == 1) {
                        int node = ni * n + nj;
                        components.insert(ds.findUPar(node));
                    }
                }

                int totalSize = 1;
                for(auto comp : components) {
                    totalSize += ds.size[comp];
                }
                ans = max(ans, totalSize);
            }
        }

        // Edge case: all 1s
        for(int i = 0; i < n * n; i++) {
            ans = max(ans, ds.size[ds.findUPar(i)]);
        }

        return ans;
    }
};
```

```
int main() {
    vector<vector<int>> grid = {{1,0},{0,1}};
    Solution obj;
    cout << obj.largestIsland(grid);
    return 0;
}
```

Complexity Analysis

Time Complexity:
$O(n^2)$
We traverse the grid a constant number of times and DSU operations are nearly constant.

Space Complexity:
$O(n^2)$
DSU stores parent and size for all n^2 cells.

48. Swim in Rising Water

You are given an $n \times n$ grid where `grid[i][j]` represents the elevation of a cell.
Rain starts at time $t = 0$, and at time t, water level is t everywhere.

You start from the top-left cell $(0,0)$ and want to reach the bottom-right cell $(n-1,n-1)$.

You can move **up, down, left, right** between adjacent cells **only if both cells have elevation ≤ t**.

Your task is to return the **minimum time `t`** at which it is possible to reach the destination.

Example 1
`grid = [[0,2],[1,3]]`

- At `t = 0`, only cell `(0,0)` is accessible.
- At `t = 1`, cell `(1,0)` becomes accessible.
- At `t = 2`, cell `(0,1)` becomes accessible.
- At `t = 3`, cell `(1,1)` becomes accessible.

So the minimum time required is `3`.

Approach

Algorithm

This problem can be seen as finding a path from start to end such that the **maximum elevation on the path is minimized**.

We simulate the rising water using a **min-heap (priority queue)**:

1. Start from cell `(0,0)` and push it into a min-heap.
2. The heap always gives the cell with the **minimum required water level**.
3. Keep track of visited cells to avoid revisiting.
4. For each cell popped from the heap:
    - The current time needed is the maximum elevation seen so far on that path.
5. Push all valid, unvisited neighbors into the heap with:
    - `max(current_time, neighbor_elevation)`
6. As soon as we reach `(n-1,n-1)`, return the current time.

The first time we reach the destination is guaranteed to be the minimum possible time.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int swimInWater(vector<vector<int>>& grid) {
        int n = grid.size();

        // Min-heap storing {time, row, col}
        priority_queue<vector<int>, vector<vector<int>>,
greater<vector<int>>> pq;

        vector<vector<int>> vis(n, vector<int>(n, 0));

        // Start from (0,0)
        pq.push({grid[0][0], 0, 0});
        vis[0][0] = 1;

        int dr[4] = {0, 1, 0, -1};
        int dc[4] = {1, 0, -1, 0};

        while (!pq.empty()) {
            auto cur = pq.top();
            pq.pop();

            int time = cur[0];
            int r = cur[1];
            int c = cur[2];

            // Destination reached
            if (r == n - 1 && c == n - 1)
                return time;

            // Explore neighbors
            for (int i = 0; i < 4; i++) {
                int nr = r + dr[i];
                int nc = c + dc[i];
```

```
                if (nr >= 0 && nc >= 0 && nr < n && nc < n &&
!vis[nr][nc]) {
                    vis[nr][nc] = 1;
                    pq.push({max(time, grid[nr][nc]), nr, nc});
                }
            }
        }
        return -1;
    }
};

int main() {
    vector<vector<int>> grid = {{0,2},{1,3}};
    Solution obj;
    cout << obj.swimInWater(grid);
    return 0;
}
```

Complexity Analysis

Time Complexity:

$O(n^2 \log n^2)$

Each cell is pushed into the priority queue once, and heap operations take logarithmic time.

Space Complexity:

$O(n^2)$

Used for the visited matrix and the priority queue.

49. Bridges in Graph – Using Tarjan's Algorithm (Time In & Low Time): G-55

You are given an undirected graph with n nodes (servers) numbered from `0` to `n-1`.
Each connection `[a, b]` represents a bidirectional link between server a and server b.

A **critical connection (bridge)** is an edge such that **removing it disconnects the graph**, meaning some servers can no longer reach others.

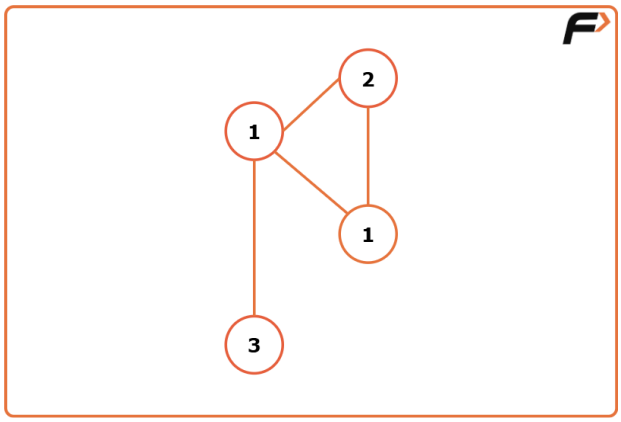
Your task is to return **all such critical connections**.

Question Explanation

In a connected network, some edges are very important.
If removing an edge increases the number of connected components, that edge is called a **bridge**.

For example, in
`connections = [[0,1],[1,2],[2,0],[1,3]]`

- The edge `[1,3]` is the only way to reach node 3
- Removing it disconnects node 3 from the rest
  So `[1,3]` is a bridge.



Approach

Algorithm (Tarjan's Algorithm using DFS)

We use **DFS with time of insertion (tin) and lowest reachable time (low)**.

Definitions:

- `tin[node]`: the time when the node is first visited
- `low[node]`: the minimum `tin` reachable from that node (including back edges)

Steps:

1. Build the adjacency list from the given connections.
2. Maintain:
    - `vis[]` to mark visited nodes
    - `tin[]` for discovery time
    - `low[]` for lowest reachable time
    - a global `timer`
3. Start DFS from node `0` (if graph is disconnected, DFS should be started from every unvisited node).
4. For each edge `(node → adjNode)`:
    - If `adjNode` is the parent, skip it.
    - If `adjNode` is unvisited:
        - DFS on it
        - Update `low[node] = min(low[node], low[adjNode])`
        - If `low[adjNode] > tin[node]`, then this edge is a **bridge**

- If `adjNode` is already visited (back edge):
    - Update `low[node] = min(low[node], tin[adjNode])`

5. Store all such edges that satisfy the bridge condition.

Example Explanation

For
`connections = [[0,1],[1,2],[2,0],[1,3]]`

- Nodes `0, 1, 2` form a cycle → no bridge inside the cycle
- Node 3 is connected only via 1
- `low[3] > tin[1]` → edge `[1,3]` is a bridge

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int timer = 1;

    void dfs(int node, int parent, vector<int> &vis,
             vector<int> adj[], int tin[], int low[],
             vector<vector<int>> &bridges) {

        vis[node] = 1;
        tin[node] = low[node] = timer;
        timer++;

        for (auto it : adj[node]) {
            if (it == parent) continue;
```

```
            if (!vis[it]) {
                dfs(it, node, vis, adj, tin, low, bridges);

                low[node] = min(low[node], low[it]);

                if (low[it] > tin[node]) {
                    bridges.push_back({node, it});
                }
            } else {
                low[node] = min(low[node], tin[it]);
            }
        }
    }

public:
    vector<vector<int>> criticalConnections(int n,
        vector<vector<int>>& connections) {

        vector<int> adj[n];
        for (auto &it : connections) {
            adj[it[0]].push_back(it[1]);
            adj[it[1]].push_back(it[0]);
        }

        vector<int> vis(n, 0);
        int tin[n], low[n];
        vector<vector<int>> bridges;

        dfs(0, -1, vis, adj, tin, low, bridges);

        return bridges;
    }
};

int main() {
    int n = 4;
    vector<vector<int>> connections = {
```

```
        {0,1},{1,2},{2,0},{1,3}
    };

    Solution obj;
    vector<vector<int>> ans = obj.criticalConnections(n, connections);

    for (auto &e : ans) {
        cout << "[" << e[0] << "," << e[1] << "] ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity:
`O(V + E)`
Each vertex and edge is visited once during DFS.

Space Complexity:
`O(V + E)`
Adjacency list plus arrays `vis`, `tin`, and `low`.

50. Articulation Point in Graph: G-56

You are given an undirected graph.
Your task is to find all **articulation points (cut vertices)**.

An articulation point is a vertex such that **removing it (along with its edges)** increases the number of connected components in the graph.

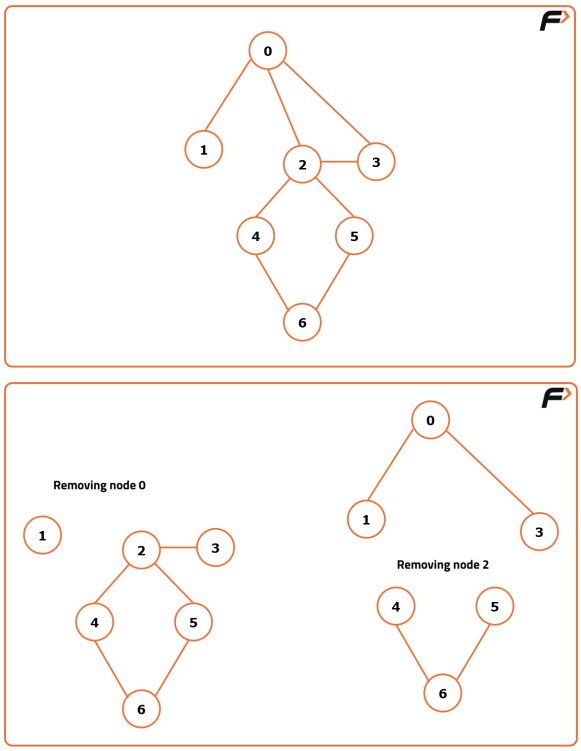
If no articulation point exists, return `-1`.

Question Explanation

In a graph, some vertices are very important for connectivity.
If removing a vertex breaks the graph into two or more disconnected parts, that vertex is called an **articulation point**.

For example:

- If removing node `0` disconnects the graph, then `0` is an articulation point.



- If removing node 2 also disconnects the graph, then 2 is also an articulation point.

Approach

Algorithm

We use **Tarjan's Algorithm** with **DFS**, using two arrays:

- `tin[node]` → time when the node is first visited
- `low[node]` → lowest discovery time reachable from that node

Steps:

1. Build the adjacency list of the graph.
2. Initialize:
    - `timer` to track discovery time
    - `vis[]` to mark visited nodes
    - `tin[]` and `low[]`
3. Run DFS for every unvisited node (graph may be disconnected).
4. During DFS:
    - Set `tin[node] = low[node] = timer++`
    - Maintain a `child` count for the current node
5. For each adjacent node:
    - If it is the parent, skip it
    - If unvisited:
        - DFS on it
        - Update `low[node] = min(low[node], low[child])`
        - If `low[child] >= tin[node]` and node is **not root**, mark node as articulation point
        - Increase `child` count
    - If already visited:
        - Update `low[node] = min(low[node], tin[adjNode])`
6. Special case for root:

- If root has more than one child, it is an articulation point

7. Collect all marked nodes as articulation points.
8. If none found, return `-1`.

Example Explanation

If a node has a child subtree that **cannot reach any ancestor** of the node using a back edge, then removing the node will disconnect that subtree. Hence, the node is an articulation point.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int timer = 1;

    void dfs(int node, int parent, vector<int> &vis,
             int tin[], int low[], vector<int> &mark,
             vector<int> adj[]) {

        vis[node] = 1;
        tin[node] = low[node] = timer++;
        int child = 0;

        for (auto it : adj[node]) {
            if (it == parent) continue;

            if (!vis[it]) {
                dfs(it, node, vis, tin, low, mark, adj);
                low[node] = min(low[node], low[it]);

                if (low[it] >= tin[node] && parent != -1) {
```

```
                    mark[node] = 1;
                }
                child++;
            }
            else {
                low[node] = min(low[node], tin[it]);
            }
        }

        if (parent == -1 && child > 1) {
            mark[node] = 1;
        }
    }

public:
    vector<int> articulationPoints(int n, vector<int> adj[]) {

        vector<int> vis(n, 0), mark(n, 0);
        int tin[n], low[n];

        for (int i = 0; i < n; i++) {
            if (!vis[i]) {
                dfs(i, -1, vis, tin, low, mark, adj);
            }
        }

        vector<int> ans;
        for (int i = 0; i < n; i++) {
            if (mark[i]) ans.push_back(i);
        }

        if (ans.size() == 0) return {-1};
        return ans;
    }
};

int main() {
    int n = 5;
```

```
    vector<vector<int>> edges = {
        {0,1},{1,4},{2,4},{2,3},{3,4}
    };

    vector<int> adj[n];
    for (auto e : edges) {
        adj[e[0]].push_back(e[1]);
        adj[e[1]].push_back(e[0]);
    }

    Solution sol;
    vector<int> res = sol.articulationPoints(n, adj);
    for (int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity:
`O(V + E)`
DFS visits every vertex and edge once.

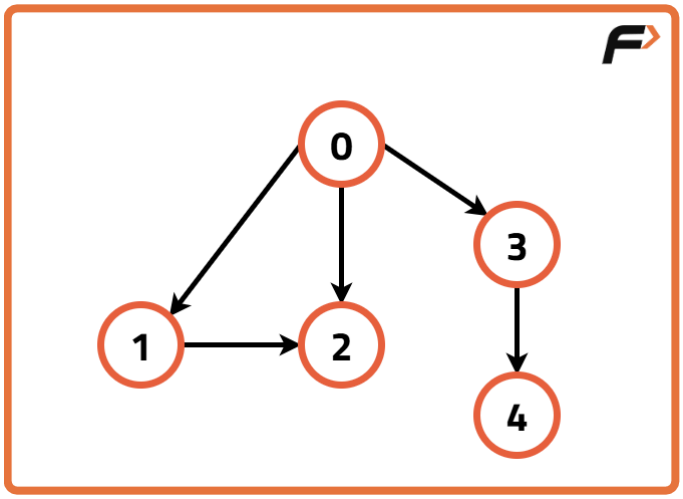
Space Complexity:
`O(V)`
Used for `vis`, `tin`, `low`, and recursion stack.

51. Strongly Connected Components – Kosaraju's Algorithm (G-54)

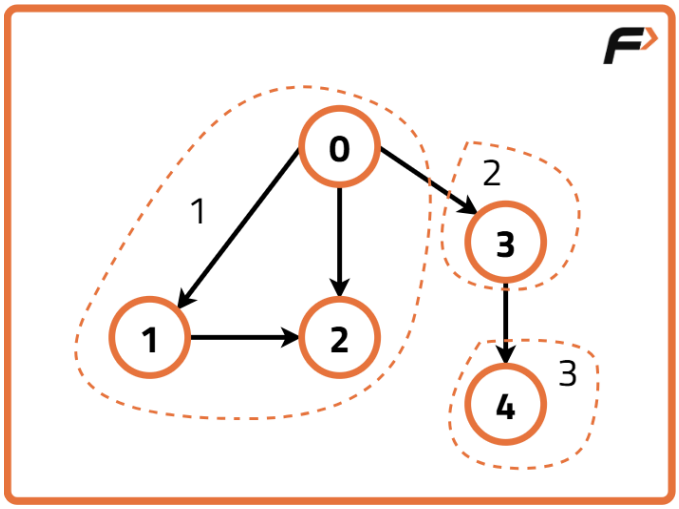
You are given a **directed graph** with `V` vertices (`0` to `V-1`) and `E` edges.
Your task is to **find the number of Strongly Connected Components (SCCs)**.

Strongly Connected Component (SCC):
A group of vertices where **every vertex is reachable from every other vertex** in the same group.

Input:



Result: 3



Key Idea (Intuition)

In a directed graph:

- Reachability is **direction dependent**

- Cycles form the backbone of SCCs

Kosaraju's Algorithm works by exploiting two observations:

1. Nodes that finish **later** in DFS are more "independent"
2. Reversing edges preserves SCC structure but changes reachability order

---

Algorithm (Kosaraju's – 3 Steps)

Step 1: DFS + Stack (Finishing Time Order)

- Perform DFS on the **original graph**
- Push nodes into a stack **after DFS completes** (based on finishing time)

Step 2: Transpose the Graph

- Reverse all edges
  If there is an edge u → v, make it v → u

Step 3: DFS on Transposed Graph

- Pop nodes from the stack
- For each unvisited node, perform DFS in the **transposed graph**
- Each DFS traversal gives **one SCC**

The **number of DFS calls in step 3 = number of SCCs**

Why This Works

- The first DFS orders nodes by how "deeply connected" they are
- The transpose graph ensures we only visit nodes inside the same SCC

- Processing nodes in decreasing finish time avoids cross-SCC traversal

---

Implementation (C++)

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    // DFS to fill stack by finishing time
    void dfs(int node, vector<int> &vis,
             vector<int> adj[], stack<int> &st) {

        vis[node] = 1;
        for (auto it : adj[node]) {
            if (!vis[it]) {
                dfs(it, vis, adj, st);
            }
        }
        // push after finishing DFS
        st.push(node);
    }

    // DFS on transposed graph
    void dfsTranspose(int node, vector<int> &vis,
                      vector<int> adjT[]) {

        vis[node] = 1;
        for (auto it : adjT[node]) {
            if (!vis[it]) {
                dfsTranspose(it, vis, adjT);
            }
        }
    }

public:
    int kosaraju(int V, vector<int> adj[]) {
```

```
        vector<int> vis(V, 0);
        stack<int> st;

        // Step 1: DFS on original graph
        for (int i = 0; i < V; i++) {
            if (!vis[i]) {
                dfs(i, vis, adj, st);
            }
        }

        // Step 2: Transpose graph
        vector<int> adjT[V];
        for (int i = 0; i < V; i++) {
            vis[i] = 0;
            for (auto it : adj[i]) {
                adjT[it].push_back(i);
            }
        }

        // Step 3: DFS using stack order
        int scc = 0;
        while (!st.empty()) {
            int node = st.top();
            st.pop();

            if (!vis[node]) {
                scc++;
                dfsTranspose(node, vis, adjT);
            }
        }
        return scc;
    }
};

int main() {
    int V = 5;
    vector<vector<int>> edges = {
```

```
        {1,0}, {0,2}, {2,1}, {0,3}, {3,4}
    };

    vector<int> adj[V];
    for (auto e : edges) {
        adj[e[0]].push_back(e[1]);
    }

    Solution sol;
    cout << "Number of SCCs: "
         << sol.kosaraju(V, adj) << endl;
    return 0;
}
```

Complexity Analysis

Metric	Complexity
Time	`O(V + E)`
Space	`O(V + E)`

- Each edge and vertex is processed at most twice
- Stack + transpose graph storage included